M.Sc. (IT & CA)
Saurashtra University
Effective from June - 2016

CS – 01: APPLICATION DEVELOPMENT USING ADVANCE JAVA

Objective:

- Learn how to download, setup and configure the Spring Framework
- Explore the Spring Container and Modules
- Understand dependency injection
- Learn aspect oriented programming and how it is used to provide cross cutting concerns
- Understand how Spring deals with transaction management and ORM
- Hibernate: Inheritance mapping collection mapping.
- Understand the HQL.

Pre-Requisites: Students must have strong background of Java programming knowledge and exposure to J2EE technology.

Unit No.	Topics	Details
1	**Basics of Spring, Spring with IDE And IOC container**	• What is Spring • Spring Modules • Spring Application • Spring in Myeclipse • Spring in Eclipse
	Dependency Injection	• Constructor Injection • CI Dependent Object • CI with collection • CI with Map • CI Inheriting Bean • Setter Injection • SI Dependent Object • SI with Collection • SI with Map • CI vs SI • Autowiring • Factory Method
	Spring AOP	• AOP Terminology • AOP Implementations • Pointcut • Advices
2	**Spring JDBC**	• JdbcTemplate Example • PreparedStatement • ResultSetExtractor • RowMapper

		• NamedParameter • SimpleJdbcTemplate
	Spring with ORM And SpEL	• Spring with Hibernate • Spring with JPA • SpEL Examples • Operators in SpEL • variable in SpEL
	Spring 3 MVC and Remoting with Spring	• Spring with RMI • Http Invoker • Hessian • Burlap • Spring with JMS
3	**OXM Frameworks, Spring Java Mail And Web Integration**	• Spring with JAXB • Spring with Xstream • Spring with Castor • Spring with Struts2 • Login and Logout Application
	Basics of Hibernate And Hibernate with IDE	• Hibernate Introduction • Hibernate Architecture • Understanding First Hibernate application • Hibernate in Eclipse • Hibernate in MyEclipse
	Hibernate Application And Hibernate Logging	• Hibernate with annotation • Hibernate Web application • Hibernate Generator classes • Hibernate Dialects • Hibernate with Log4j 1 • Hibernate with Log4j 2
4	**Inheritance Mapping**	• Table Per Hierarchy • Table Per Hierarchy using Annotation • Table Per Concrete • Table Per Concreteusing Annotation • Table Per Subclass • Table Per Subclass using Annotation
	Collection Mapping	• Mapping List • One-to-many by List using XML • Many to Many by List using XML • One To Many by List using Annotation • Mapping Bag

		• One-to-many by Bag • Mapping Set • One-to-many by Set • Mapping Map • Many-to-many by Map • Bidirectional • Lazy Collection
5	**Component Mapping, Association Mapping, Transaction Management, HQL and HCQL**	• One-to-one using Primary Key • One-to-one using Foreign Key
	Named Query, Hibernate Caching and Integration	• First Level Cache • Second Level Cache • Hibernate and Struts • Hibernate and Spring

Chapter-1.1

Basic of Spring with IDE and IoC container

☺ What is Spring?

This spring tutorial provides in-depth concepts of Spring Framework with simplified examples. It was **developed by Rod Johnson in 2003**. Spring framework makes the easy development of JavaEE application.

It is helpful for beginners and experienced persons.

Spring Framework

Spring is a *lightweight* framework. It can be thought of as a *framework of frameworks* because it provides support to various frameworks such as Struts, Hibernate, Tapestry, EJB, JSF etc. The framework, in broader sense, can be defined as a structure where we find solution of the various technical problems.

The Spring framework comprises several modules such as IOC, AOP, DAO, Context, ORM, WEB MVC etc. We will learn these modules in next page. Let's understand the IOC and Dependency Injection first.

Inversion Of Control (IOC) and Dependency Injection

These are the design patterns that are used to remove dependency from the programming code. They make the code easier to test and maintain. Let's understand this with the following code:

```
class Employee
{
Address address;

Employee()
{
address=new Address();
}
}
```

In such case, there is dependency between the Employee and Address (tight coupling). In the Inversion of Control scenario, we do this something like this:

```
class Employee{
Address address;

Employee(Address address)
{
this.address=address;
}
}
```

Thus, IOC makes the code loosely coupled. In such case, there is no need to modify the code if our logic is moved to new environment.

In Spring framework, IOC container is responsible to inject the dependency. We provide metadata to the IOC container either by XML file or annotation.

Advantage of Dependency Injection

- makes the code loosely coupled so easy to maintain
- makes the code easy to test

Advantages of Spring Framework

There are many advantages of Spring Framework. They are as follows:

1) Predefined Templates

Spring framework provides templates for JDBC, Hibernate, JPA etc. technologies. So there is no need to write too much code. It hides the basic steps of these technologies.

Let's take the example of JdbcTemplate, you don't need to write the code for exception handling, creating connection, creating statement, committing transaction, closing connection etc. You need to write the code of executing query only. Thus, it save a lot of JDBC code.

2) Loose Coupling

The Spring applications are loosely coupled because of dependency injection.

3) Easy to test

The Dependency Injection makes easier to test the application. The EJB or Struts application require server to run the application but Spring framework doesn't require server.

4) Lightweight

Spring framework is lightweight because of its POJO implementation. The Spring Framework doesn't force the programmer to inherit any class or implement any interface. That is why it is said non-invasive.

---

5) Fast Development

The Dependency Injection feature of Spring Framework and it support to various frameworks makes the easy development of JavaEE application.

---

6) Powerful abstraction

It provides powerful abstraction to JavaEE specifications such as JMS, JDBC, JPA and JTA.

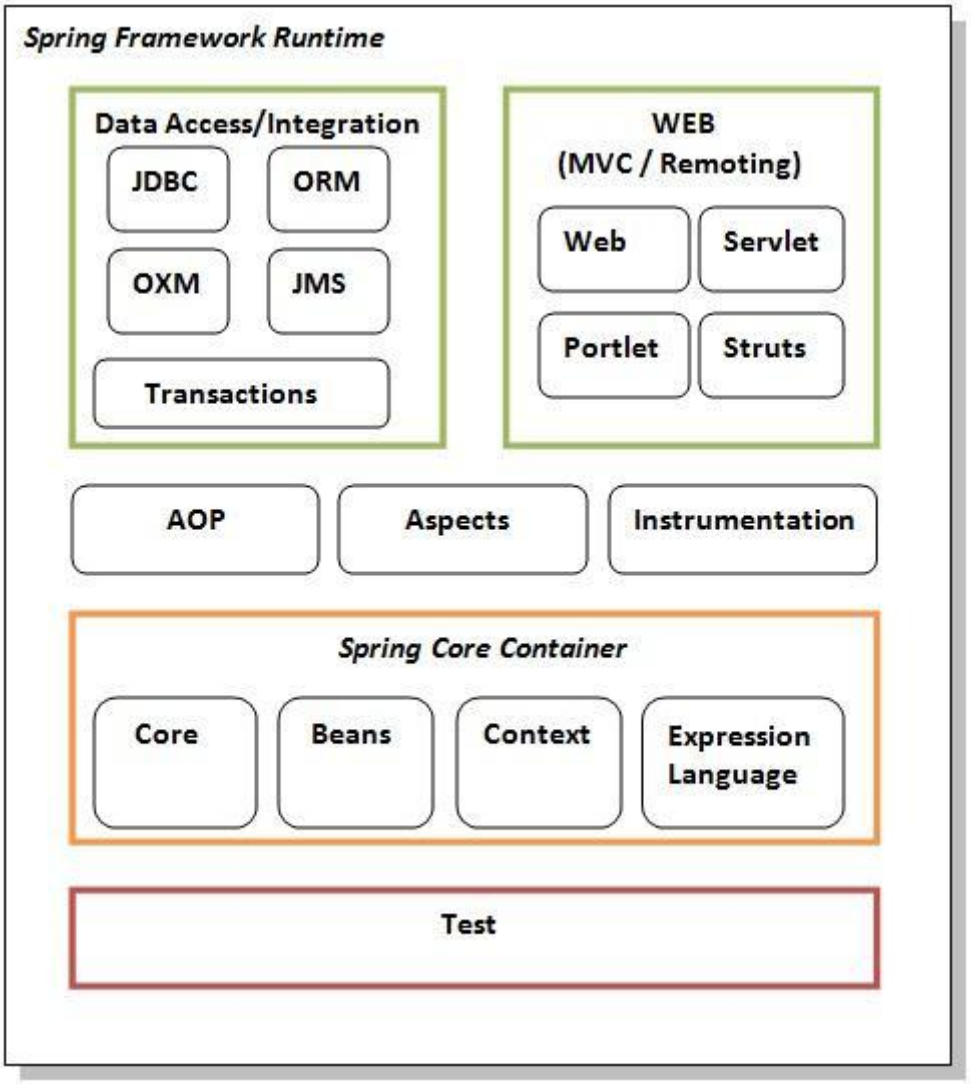
---

7) Declarative support

It provides declarative support for caching, validation, transactions and formatting.

☺ Spring Modules

The Spring framework comprises of many modules such as core, beans, context, expression language, AOP, Aspects, Instrumentation, JDBC, ORM, OXM, JMS, Transaction, Web, Servlet, Struts etc. These modules are grouped into Test, Core Container, AOP, Aspects, Instrumentation, Data Access / Integration, Web (MVC / Remoting) as displayed in the following diagram.



Test

This layer provides support of testing with JUnit and TestNG.

Spring Core Container

The Spring Core container contains core, beans, context and expression language (EL) modules.

Core and Beans

These modules provide IOC and Dependency Injection features.

Context

This module supports internationalization (I18N), EJB, JMS, Basic Remoting.

Expression Language

It is an extension to the EL defined in JSP. It provides support to setting and getting property values, method invocation, accessing collections and indexers, named variables, logical and arithmetic operators, retrieval of objects by name etc.

AOP, Aspects and Instrumentation

These modules support aspect oriented programming implementation where you can use Advices, Pointcuts etc. to decouple the code.

The aspects module provides support to integration with AspectJ.

The instrumentation module provides support to class instrumentation and classloader implementations.

Data Access / Integration

This group comprises of JDBC, ORM, OXM, JMS and Transaction modules. These modules basically provide support to interact with the database.

Web

This group comprises of Web, Web-Servlet, Web-Struts and Web-Portlet. These modules provide support to create web application.

☺ Spring Application

Steps to create spring application

Here, we are going to learn the simple steps to create the first spring application. To run this application, we are not using any IDE. We are simply using the command prompt. Let's see the simple steps to create the spring application

- **create the class**
- **create the xml file to provide the values**
- **create the test class**
- **Load the spring jar files**
- **Run the test class**

Steps to create spring application

Let's see the 5 steps to create the first spring application.

1) Create Java class

This is the simple java bean class containing the name property only.

```
package com.smgc;

public class Student
{
private String name;


public String getName()
{
   return name;
}

public void setName(String name)
{
   this.name = name;
}

public void displayInfo()
{
   System.out.println("Hello: " + name);
}
}
```

This is simple bean class, containing only one property name with its getters and setters method. This class contains one extra method named displayInfo() that prints the student name by the hello message.

2) Create the xml file

In case of myeclipse IDE, you don't need to create the xml file as myeclipse does this for yourselves. Open the applicationContext.xml file, and write the following code:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

<bean id="studentbean" class="com.smgc.Student">
      <property name="name" value="Piyush Patel"></property>
</bean>

</beans>
```

The **bean** element is used to define the bean for the given class. The **property** subelement of bean specifies the property of the Student class named name. The value specified in the property element will be set in the Student class object by the IOC container.

3) Create the test class

Create the java class e.g. Test. Here we are getting the object of Student class from the IOC container using the getBean() method of BeanFactory. Let's see the code of test class.

```
package com.smgc;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

public class Test {
public static void main(String[] args) {

Resource resource=new ClassPathResource("applicationContext.xml");
BeanFactory factory=new XmlBeanFactory(resource);

Student student=(Student)factory.getBean("studentbean");
student.displayInfo();
}
}
```

The **Resource** object represents the information of applicationContext.xml file. The Resource is the interface and the **ClassPathResource** is the implementation class of the Reource interface. The **BeanFactory** is responsible to return the bean. The **XmlBeanFactory** is the implementation class of the BeanFactory. There are many methods in the BeanFactory interface. One method is **getBean()**, which returns the object of the associated class.

4) Load the jar files required for spring framework

There are mainly three jar files required to run this application.

- **org.springframework.core-3.0.1.RELEASE-A**
- **com.springsource.org.apache.commons.logging-1.1.1**
- **org.springframework.beans-3.0.1.RELEASE-A**

For the future use, You can download the required jar files for spring core application.

download the core jar files for spring

download the all jar files for spring including core, web, aop, mvc, j2ee, remoting, oxm, jdbc, orm etc.

To run this example, you need to load only spring core jar files.

---

5) Run the test class

In command prompt

Javac –d . Student.java
Javac –d . Test.java
Java com.smgc.Test

Now run the Test class. You will get the output Hello: Piyush Patel.

☺ Spring In Myeclipse

Example of spring application in Myeclipse

Creating spring application in myeclipse IDE is simple. You don't need to be worried about the jar files required for spring application because myeclipse IDE takes care of it. Let's see the simple steps to create the spring application in myeclipse IDE.

- **create the java project**
- **add spring capabilities**
- **create the class**
- **create the xml file to provide the values**
- **create the test class**

Steps to create spring application in Myeclipse IDE

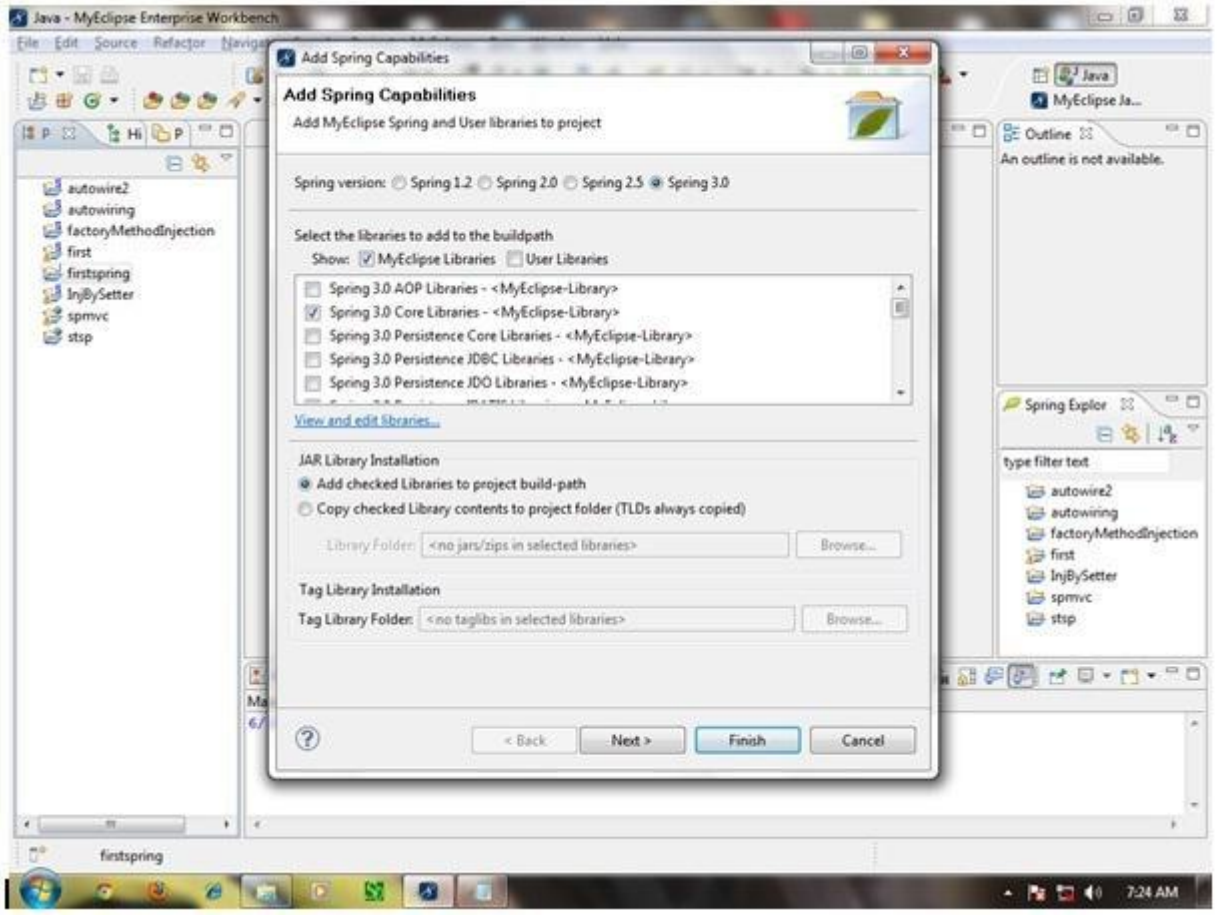
Let's see the 5 steps to create the first spring application using myeclipse IDE.

1) Create the Java Project

Go to **File** menu - **New** - **project** - **Java Project**. Write the project name e.g. firstspring - **Finish**. Now the java project is created.

2) Add spring capabilities

Go to **Myeclipse** menu - **Project Capabilities** - **Add spring capabilities** - **Finish**. Now the spring jar files will be added. For the simple application we need only core library i.e.selected by default.



3) Create Java class

In such case, we are simply creating the Student class have name property. The name of the student will be provided by the xml file. It is just a simple example not the actual use of spring. We will see the actual use in Dependency Injection chapter. To create the java class, **Right click on src** - **New** - **class** - **Write the class name e.g. Student** - **finish**. Write the following code:

```
1. package com.smgc;
2.
3. public class Student {
4. private String name;
5.
6. public String getName() {
7.     return name;
8. }
9.
10. public void setName(String name) {
11.     this.name = name;
12.}
13. public void displayInfo(){
14.     System.out.println("Hello: "+name);
15.}
16.}
```

This is simple bean class, containing only one property name with its getters and setters method. This class contains one extra method named displayInfo() that prints the student name by the hello message.

4) Create the xml file

In case of myeclipse IDE, you don't need to create the xml file as myeclipse does this for yourselves. Open the applicationContext.xml file, and write the following code:

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans
3.     xmlns="http://www.springframework.org/schema/beans"
4.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.     xmlns:p="http://www.springframework.org/schema/p"
6.     xsi:schemaLocation="http://www.springframework.org/schema/beans
7.             http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
8.
9. <bean id="studentbean" class="com.smgc.Student">
10. <property name="name" value="Piyush Patel"></property>
11. </bean>
12.
13. </beans>
```

The **bean** element is used to define the bean for the given class. The **property** subelement of bean specifies the property of the Student class named name. The value specified in the property element will be set in the Student class object by the IOC container.

5) Create the test class

Create the java class e.g. Test. Here we are getting the object of Student class from the IOC container using the getBean() method of BeanFactory. Let's see the code of test class.

```
1. package com.smgc;
2.
3. import org.springframework.beans.factory.BeanFactory;
4. import org.springframework.beans.factory.xml.XmlBeanFactory;
5. import org.springframework.core.io.ClassPathResource;
6. import org.springframework.core.io.Resource;
7.
8. public class Test {
9. public static void main(String[] args) {
10.    Resource resource=new ClassPathResource("applicationContext.xml");
11.    BeanFactory factory=new XmlBeanFactory(resource);
12.
13.    Student student=(Student)factory.getBean("studentbean");
14.    student.displayInfo();
15.}
16.}
```

The **Resource** object represents the information of applicationContext.xml file. The Resource is the interface and the **ClassPathResource** is the implementation class of the Reource interface. The **BeanFactory** is responsible to return the bean. The **XmlBeanFactory** is the implementation class of the BeanFactory. There are many methods in the BeanFactory interface. One method is **getBean()**, which returns the object of the associated class.

Now run the Test class. You will get the output Hello: Piyush Patel.

☺ Spring In Eclipse

Creating spring application in Eclipse IDE

Here, we are going to create a simple application of spring framework using eclipse IDE. Let's see the simple steps to create the spring application in Eclipse IDE.

- **create the java project**
- **add spring jar files**
- **create the class**
- **create the xml file to provide the values**
- **create the test class**

Steps to create spring application in Eclipse IDE

Let's see the 5 steps to create the first spring application using eclipse IDE.

1) Create the Java Project

Go to **File** menu - **New** - **project** - **Java Project**. Write the project name e.g. firstspring - **Finish**. Now the java project is created.

2) Add spring jar files

There are mainly three jar files required to run this application.

- **org.springframework.core-3.0.1.RELEASE-A**
- **com.springsource.org.apache.commons.logging-1.1.1**
- **org.springframework.beans-3.0.1.RELEASE-A**

For the future use, You can download the required jar files for spring core application.

To run this example, you need to load only spring core jar files.

To load the jar files in eclipse IDE, **Right click on your project** - **Build Path** - **Add external archives** - **select all the required jar files** - **finish.**.

3) Create Java class

In such case, we are simply creating the Student class have name property. The name of the student will be provided by the xml file. It is just a simple example not the actual use of spring. We will see the actual use in Dependency Injection chapter. To create the java class, **Right click on src** - **New** - **class** - **Write the class name e.g. Student** - **finish**. Write the following code:

```
package com.smgc;

public class Student {
private String name;

public String getName() {
return name;
}

public void setName(String name) {
this.name = name;
}

public void displayInfo(){
System.out.println("Hello: "+name);
}
}
```

This is simple bean class, containing only one property name with its getters and setters method. This class contains one extra method named displayInfo() that prints the student name by the hello message.

4) Create the xml file

To create the xml file click on src - new - file - give the file name such as applicationContext.xml - finish. Open the applicationContext.xml file, and write the following code:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
<bean id="studentbean" class="com.smgc.Student">
<property name="name" value="Piyush Patel"></property>
</bean>

</beans>
```

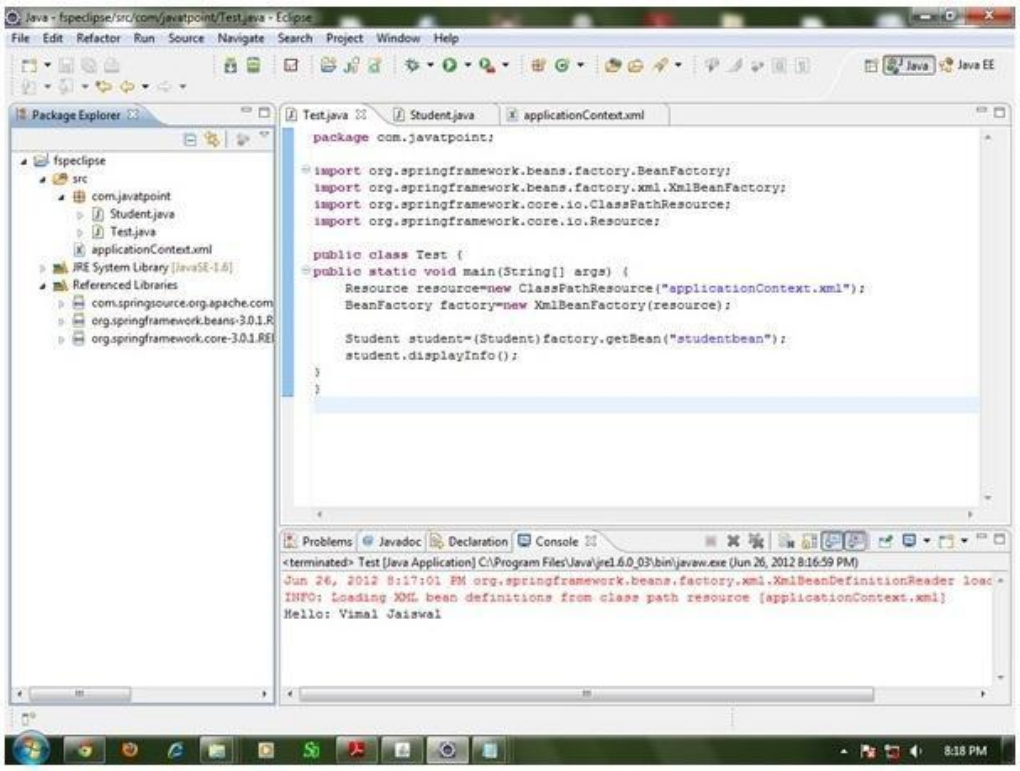
The **bean** element is used to define the bean for the given class. The **property** subelement of bean specifies the property of the Student class named name. The value specified in the property element will be set in the Student class object by the IOC container.

5) Create the test class

Create the java class e.g. Test. Here we are getting the object of Student class from the IOC container using the getBean() method of BeanFactory. Let's see the code of test class.

```
package com.smgc;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

public class Test {
public static void main(String[] args) {
Resource resource=new ClassPathResource("applicationContext.xml");
BeanFactory factory=new XmlBeanFactory(resource);

Student student=(Student)factory.getBean("studentbean");
student.displayInfo();
}
}
```

Now run this class. You will get the output Hello: Piyush Patel.



Chapter-1.2

Dependency Injection

☺ Constructor Injection

Dependency Injection by Constructor Example

We can inject the dependency by constructor. The **<constructor-arg>** subelement of **<bean>**
is used for constructor injection. Here we are going to inject

1. primitive and String-based values
2. Dependent object (contained object)
3. Collection values etc.

Injecting primitive and string-based values

Let's see the simple example to inject primitive and string-based values. We have created three files here:

- Employee.java
- applicationContext.xml
- Test.java

Employee.java

It is a simple class containing two fields id and name. There are four constructors and one method in this class.

```
package com.smgc;

public class Employee
{
private int id;
private String name;

public Employee()
{
    System.out.println("def cons");
}

public Employee(int id)
{
    this.id = id;
}

public Employee(String name)
{
    this.name = name;
```

```
}

public Employee(int id, String name)
{
    this.id = id;
    this.name = name;
}

void show()
{
    System.out.println(id+"  "+name);
}
}
```

applicationContext.xml

We are providing the information into the bean by this file. The constructor-arg element invokes the constructor. In such case, parameterized constructor of int type will be invoked. The value attribute of constructor-arg element will assign the specified value. The type attribute specifies that int parameter constructor will be invoked.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

<bean id="e" class=“com.smgc.Employee">
   <constructor-arg value="10" type="int"></constructor-arg>
</bean>

<bean id="e1" class=“com.smgc.Employee">
   <constructor-arg value="Piyush" type="String"></constructor-arg>
</bean>

<bean id="e2" class=“com.smgc.Employee">
   <constructor-arg value="11" type="int"></constructor-arg>
   <constructor-arg value="Piyush" type="String"></constructor-arg>
</bean>

</beans>
```

Test.java

This class gets the bean from the applicationContext.xml file and calls the show method.

```
package com.smgc;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.*;
```

```
public class Test {
public static void main(String[] args) {

        Resource r = new
        BeanFactory factory = new XmlBeanFactory(r);

        Employee s=(Employee)factory.getBean("e");
        s.show();
        }
        }
```

Output:10 null

Injecting string-based values

If you don't specify the type attribute in the constructor-arg element, by default string type constructor will be invoked.

```
1. ....
2. <bean id="e" class=“com.smgc.Employee">
3. <constructor-arg value="10"></constructor-arg>
4. </bean>
5. ....
```

If you change the bean element as given above, string parameter constructor will be invoked and the output will be 0 10.

Output:0 10

You may also pass the string literal as following:

```
1. ....
2. <bean id="e" class=“com.smgc.Employee">
3. <constructor-arg value="Piyush"></constructor-arg>
4. </bean>
5. ....
```

Output:0 Piyush

You may pass integer literal and string both as following

```
1. ....
2. <bean id="e" class=“com.smgc.Employee">
3. <constructor-arg value="10" type="int" ></constructor-arg>
4. <constructor-arg value="Piyush"></constructor-arg>
5. </bean>
6. ....
```

Output:10 Piyush

☺ CI Dependent Object

Constructor Injection with Dependent Object

If there is HAS-A relationship between the classes, we create the instance of dependent object (contained object) first then pass it as an argument of the main class constructor. Here, our scenario is Employee HAS-A Address. The Address class object will be termed as the dependent object. Let's see the Address class first:

Address.java

This class contains three properties, one constructor and toString() method to return the values of these object.

```
package com.smgc;

public class Address
{
private String city;
private String state;
private String country;

public Address(String city, String state, String country)
{
super();
this.city = city;
this.state = state;
this.country = country;
}

public String toString(){
return city+" "+state+" "+country;
}
}
```

Employee.java

It contains three properties id, name and address(dependent object) ,two constructors and show() method to show the records of the current object including the depedent object.

```
package com.smgc;

public class Employee {
private int id;
private String name;

private Address address;     //Aggregation

public Employee() {System.out.println("def cons");}
```

```
public Employee(int id, String name, Address address)
{
super();
this.id = id;
this.name = name;
this.address = address;
}

void show()
{
System.out.println(id+" "+name);
System.out.println(address.toString());
}

}
```

applicationContext.xml

The **ref** attribute is used to define the reference of another object, such way we are passing the dependent object as an constructor argument.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

<bean id="a1" class="com.smgc.Address">
   <constructor-arg value="Rajkot"></constructor-arg>
   <constructor-arg value="Gujarat"></constructor-arg>
   <constructor-arg value="India"></constructor-arg>
</bean>


<bean id="e" class="com.smgc.Employee">
   <constructor-arg value="11" type="int"></constructor-arg>
    <constructor-arg value="Piyush"></constructor-arg>
    <constructor-arg>    <ref bean="a1" />    </constructor-arg>
</bean>

</beans>
```

Test.java

This class gets the bean from the applicationContext.xml file and calls the show method.

```
package com.smgc;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.*;

public class Test
{
   public static void main(String[] args)
   {
        Resource r=new ClassPathResource("applicationContext.xml");
        BeanFactory factory=new XmlBeanFactory(r);

        Employee s=(Employee)factory.getBean("e");
        s.show();
   }
}
```

☺ CI with collection

Constructor Injection with Collection Example

We can inject collection values by constructor in spring framework. There can be used three elements inside the **constructor-arg** element.

It can be:

1. **list**
2. **set**
3. **map**

Each collection can have string based and non-string based values.

In this example, we are taking the example of Forum where **One question can have multiple answers**. There are three pages:

1. **Question.java**
2. **applicationContext.xml**
3. **Test.java**

In this example, we are using list that can have duplicate elements, you may use set that have only unique elements. But, you need to change list to set in the applicationContext.xml file and List to Set in the Question.java file.

Question.java

This class contains three properties, two constructors and displayInfo() method that prints the information. Here, we are using List to contain the multiple answers.

```
package com.smgc;

import java.util.Iterator;
import java.util.List;

public class Question
{
private int id;
private String name;
private List<String> answers;

public Question() {}

public Question(int id, String name, List<String> answers)
{
super();
this.id = id;
this.name = name;
this.answers = answers;
}
```

```
public void displayInfo()
{
   System.out.println(id+" "+name);
   System.out.println("answers are:");
   Iterator<String> itr = answers.iterator();
   while(itr.hasNext())
   {
            System.out.println(itr.next());
   }
}
}
```

applicationContext.xml

The list element of constructor-arg is used here to define the list.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

<bean id="q" class="com.smgc.Question">
   <constructor-arg value="111"></constructor-arg>
   <constructor-arg value="What is java?"></constructor-arg>
   <constructor-arg>
   <list>
      <value>Java is a programming language</value>
      <value>Java is a Platform</value>
      <value>Java is an Island of Indonasia</value>
   </list>
</constructor-arg>
</bean>

</beans>
```

Test.java

This class gets the bean from the applicationContext.xml file and calls the displayInfo method.

```
package com.smgc;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

public class Test {
public static void main(String[] args)
{
	Resource r=new ClassPathResource("applicationContext.xml");
	BeanFactory factory=new XmlBeanFactory(r);

	Question q = (Question) factory.getBean("q");
	q.displayInfo();
}
}
```

☺ CI with Map

Constructor Injection with Map Example

In this example, we are using **map** as the answer that have answer with posted username. Here, we are using key and value pair both as a string.

Like previous examples, it is the example of forum where **one question can have multiple answers**.

Question.java

This class contains three properties, two constructors and displayInfo() method to display the information.

```
package com.smgc;
import java.util.Iterator;
import java.util.Map;
import java.util.Set;
import java.util.Map.Entry;

public class Question {
private int id;
private String name;
private Map<String,String> answers;

public Question() {}

public Question(int id, String name, Map<String, String> answers) {
   super();
   this.id = id;
    this.name = name;
    this.answers = answers;
}

public void displayInfo(){
    System.out.println("question id: "+id);
    System.out.println("question name: "+name);
    System.out.println("Answers....");
    Set<Entry<String, String>> set=answers.entrySet();
    Iterator<Entry<String, String>> itr=set.iterator();
    while(itr.hasNext()){
        Entry<String,String> entry=itr.next();
        System.out.println("Answer: "+entry.getKey()+" Posted By: "+entry.getValue());
    }
}
}
```

applicationContext.xml

The **entry** attribute of **map** is used to define the key and value information.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

<bean id="q" class="com.smgc.Question">
<constructor-arg value="11"></constructor-arg>
<constructor-arg value="What is Java?"></constructor-arg>
<constructor-arg>
<map>
   <entry key="Java is a Programming Language"  value="Ajay Patil"></entry>
   <entry key="Java is a Platform" value="John Sinha"></entry>
   <entry key="Java is an Island" value="Raj Kumar"></entry>
</map>
</constructor-arg>
</bean>

</beans>
```

Test.java

This class gets the bean from the applicationContext.xml file and calls the displayInfo() method.

```
package com.smgc;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

public class Test {
public static void main(String[] args) {
Resource r=new ClassPathResource("applicationContext.xml");
BeanFactory factory=new XmlBeanFactory(r);

Question q=(Question)factory.getBean("q");
q.displayInfo();

}
}
```

☺ CI Inheriting Beans

Inheriting Bean in Spring

By using the **parent** attribute of **bean**, we can specify the inheritance relation between the beans. In such case, parent bean values will be inherited to the current bean.

Let's see the simple example to inherit the bean.

Employee.java

This class contains three properties, three constructor and show() method to display the values.

```
package com.smgc;

public class Employee {
   private int id;
   private String name;
   private Address address;

   public Employee() {}

public Employee(int id, String name) {
super();
this.id = id;
this.name = name;
}
public Employee(int id, String name, Address address) {
super();
this.id = id;
this.name = name;
this.address = address;
}

void show(){
System.out.println(id+" "+name);
System.out.println(address);
}

}
```

Address.java

```
package com.smgc;

public class Address {
private String addressLine1,city,state,country;

public Address(String addressLine1, String city, String state, String country) {
super();
this.addressLine1 = addressLine1;
this.city = city;
this.state = state;
this.country = country;
}

public String toString(){
return addressLine1+" "+city+" "+state+" "+country;
}

}
```

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

<bean id="e1" class=“com.smgc.Employee">
   <constructor-arg value="101"></constructor-arg>
   <constructor-arg  value="Sachin"></constructor-arg>
</bean>

<bean id="address1" class=“com.smgc.Address">
   <constructor-arg value="30,Uni. Road"></constructor-arg>
   <constructor-arg value="Rajkot"></constructor-arg>
   <constructor-arg value="Gujarat"></constructor-arg>
   <constructor-arg value="INDIA"></constructor-arg>
</bean>

<bean id="e2" class=“com.smgc.Employee" parent="e1">
   <constructor-arg ref="address1"></constructor-arg>
</bean>

</beans>
```

Test.java

This class gets the bean from the applicationContext.xml file and calls the show method.

```
package com.smgc;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

public class Test {
public static void main(String[] args) {
Resource r=new ClassPathResource("applicationContext.xml");
BeanFactory factory=new XmlBeanFactory(r);

Employee e1=(Employee)factory.getBean("e1");
e1.show();

e1=(Employee)factory.getBean("e2");
e1.show();

}
}
```

☺ Setter Injection

Dependency Injection by setter method

We can inject the dependency by setter method also. The **<property>** subelement of **<bean>** is used for setter injection. Here we are going to inject

1. primitive and String-based values
2. Dependent object (contained object)
3. Collection values etc.

Injecting primitive and string-based values by setter method

Let's see the simple example to inject primitive and string-based values by setter method. We have created three files here:

- Employee.java
- applicationContext.xml
- Test.java

Employee.java

It is a simple class containing three fields id, name and city with its setters and getters and a method to display these informations.

```
package com.smgc;

public class Employee {
private int id;
private String name;
private String city;

public int getId() {
return id;
}
public void setId(int id) {
this.id = id;
}
public String getName() {
return name;
}
public void setName(String name) {
this.name = name;
}

public String getCity() {
return city;
}
public void setCity(String city) {
```

```
this.city = city;
}

void display(){
System.out.println(id+" "+name+" "+city);
}

}
```

applicationContext.xml

We are providing the information into the bean by this file. The property element invokes the setter method. The value subelement of property will assign the specified value.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

<bean id="obj" class="com.smgc.Employee">
   <property name="id">
        <value>20</value>
   </property>

   <property name="name">
        <value>Piyush</value>
   </property>

   <property name="city">
        <value>Rajkot</value>
   </property>
</bean>

</beans>
```

Test.java

This class gets the bean from the applicationContext.xml file and calls the display method.

```
package com.smgc;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.*;

public class Test {
public static void main(String[] args) {

        Resource                                                r=new
        BeanFactory factory=new XmlBeanFactory(r);

        Employee e=(Employee)factory.getBean("obj");
        s.display();
```

Output:20 Piyush Rajkot

☺ SI Dependent Object

Setter Injection with Dependent Object Example

Like Constructor Injection, we can inject the dependency of another bean using setters. In such case, we use **property** element. Here, our scenario is **Employee HAS-A Address**. The Address class object will be termed as the dependent object. Let's see the Address class first:

Address.java

This class contains four properties, setters and getters and toString() method.

```
1. package com.smgc;
2.
3. public class Address {
4. private String addressLine1,city,state,country;
5.
6. //getters and setters
7.
8. public String toString(){
9.     return addressLine1+" "+city+" "+state+" "+country;
10.}
```

Employee.java

It contains three properties id, name and address(dependent object) , setters and getters with displayInfo() method.

```
1. package com.smgc;
2.
3. public class Employee {
4. private int id;
5. private String name;
6. private Address address;
7.
8. //setters and getters
9.
10.void displayInfo(){
11.    System.out.println(id+" "+name);
12.    System.out.println(address);
13.}
14.}
```

applicationContext.xml

The **ref** attribute of **property** elements is used to define the reference of another bean.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

<bean id="address1" class="com.smgc.Address">
<property name="addressLine1" value="30, Uni Road"></property>
<property name="city" value="Rajkot"></property>
<property name="state" value="Gujarat"></property>
<property name="country" value="India"></property>
</bean>

<bean id="obj" class="com.smgc.Employee">
<property name="id" value="1"></property>
<property name="name" value="Sachin Yadav"></property>
<property name="address" ref="address1"></property>
</bean>

</beans>
```

Test.java

This class gets the bean from the applicationContext.xml file and calls the displayInfo() method.

```
1.  package com.smgc;
2.
3.  import org.springframework.beans.factory.BeanFactory;
4.  import org.springframework.beans.factory.xml.XmlBeanFactory;
5.  import org.springframework.context.ApplicationContext;
6.  import org.springframework.context.support.ClassPathXmlApplicationContext;
7.  import org.springframework.core.io.ClassPathResource;
8.  import org.springframework.core.io.Resource;
9.
10. public class Test {
11. public static void main(String[] args) {
12.     Resource r=new ClassPathResource("applicationContext.xml");
13.     BeanFactory factory=new XmlBeanFactory(r);
14.
15.     Employee e=(Employee)factory.getBean("obj");
16.     e.displayInfo();
17. }
18. }
```

☺ SI with collection

Setter Injection with Collection Example

We can inject collection values by setter method in spring framework. There can be used three elements inside the **property** element.

It can be:

1. **list**
2. **set**
3. **map**

Each collection can have string based and non-string based values.

In this example, we are taking the example of Forum where **One question can have multiple answers**. There are three pages:

1. **Question.java**
2. **applicationContext.xml**
3. **Test.java**

In this example, we are using list that can have duplicate elements, you may use set that have only unique elements. But, you need to change list to set in the applicationContext.xml file and List to Set in the Question.java file.

Question.java

This class contains three properties with setters and getters and displayInfo() method that prints the information. Here, we are using List to contain the multiple answers.

```
1. package com.smgc;
2. import java.util.Iterator;
3. import java.util.List;
4.
5. public class Question {
6. private int id;
7. private String name;
8. private List<String> answers;
9.
10. //setters and getters
11.
12. public void displayInfo(){
13.     System.out.println(id+" "+name);
14.     System.out.println("answers are:");
15.     Iterator<String> itr=answers.iterator();
16.     while(itr.hasNext()){
17.         System.out.println(itr.next());
18.     }
19. }
20.
21. }
```

applicationContext.xml

The list element of constructor-arg is used here to define the list.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
    xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:p="http://www.springframework.org/schema/p"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

<bean id="q" class="com.smgc.Question">
<property name="id" value="1"></property>
<property name="name" value="What is Java?"></property>
<property name="answers">
<list>
<value>Java is a programming language</value>
<value>Java is a platform</value>
<value>Java is an Island</value>
</list>
</property>
</bean>

</beans>
```

Test.java

This class gets the bean from the applicationContext.xml file and calls the displayInfo method.

```
package com.smgc;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;

public class Test {
public static void main(String[] args) {
    Resource r=new ClassPathResource("applicationContext.xml");
    BeanFactory factory=new XmlBeanFactory(r);

    Question q=(Question)factory.getBean("q");
    q.displayInfo();
}
}
```

☺ SI with Map

Setter Injection with Map Example

In this example, we are using **map** as the answer for a question that have answer as the key and username as the value. Here, we are using key and value pair both as a string.

Like previous examples, it is the example of forum where **one question can have multiple answers**.

Question.java

This class contains three properties, getters & setters and displayInfo() method to display the information.

```
package com.smgc;
import java.util.Iterator;
import java.util.Map;
import java.util.Set;
import java.util.Map.Entry;
6.
public class Question {
private int id;
private String name;
private Map<String,String> answers;

 //getters and setters

 public void displayInfo(){
    System.out.println("question id:"+id);
    System.out.println("question name:"+name);
    System.out.println("Answers....");
    Set<Entry<String, String>> set=answers.entrySet();
    Iterator<Entry<String, String>> itr=set.iterator();
    while(itr.hasNext()){
       Entry<String,String> entry=itr.next();
       System.out.println("Answer:"+entry.getKey()+" Posted By:"+entry.getValue());
    }
}
}
```

applicationContext.xml

The **entry** attribute of **map** is used to define the key and value information.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
```

```
<bean id="q" class="com.smgc.Question">
<property name="id" value="1"></property>
<property name="name" value="What is Java?"></property>
<property name="answers">
<map>
<entry key="Java is a programming language"  value="Piyush Patel"></entry>
<entry key="Java is a Platform" value="Sachin Yadav"></entry>
</map>
</property>
</bean>

</beans>
```

Test.java

This class gets the bean from the applicationContext.xml file and calls the displayInfo() method.

```
1.  package com.smgc;
2.
3.  import org.springframework.beans.factory.BeanFactory;
4.  import org.springframework.beans.factory.xml.XmlBeanFactory;
5.  import org.springframework.core.io.ClassPathResource;
6.  import org.springframework.core.io.Resource;
7.
8.  public class Test {
9.  public static void main(String[] args) {
10.     Resource r=new ClassPathResource("applicationContext.xml");
11.     BeanFactory factory=new XmlBeanFactory(r);
12.
13.     Question q=(Question)factory.getBean("q");
14.     q.displayInfo();
15.
16. }
17. }
```

☺ CI vs SI

Difference between constructor and setter injection

There are many key differences between constructor injection and setter injection.

1. **Partial dependency**: can be injected using setter injection but it is not possible by constructor. Suppose there are 3 properties in a class, having 3 arg constructor and setters methods. In such case, if you want to pass information for only one property, it is possible by setter method only.

2. **Overriding**: Setter injection overrides the constructor injection. If we use both constructor and setter injection, IOC container will use the setter injection.

3. **Changes**: We can easily change the value by setter injection. It doesn't create a new bean instance always like constructor. So setter injection is flexible than constructor injection.

☺ Autowriting

Autowiring in Spring

Autowiring feature of spring framework enables you to inject the object dependency implicitly. It internally uses setter or constructor injection.

Autowiring can't be used to inject primitive and string values. It works with reference only.

Advantage of Autowiring

It requires the **less code** because we don't need to write the code to inject the dependency explicitly.

Disadvantage of Autowiring

No control of programmer.

It can't be used for primitive and string values.

Autowiring Modes

There are many autowiring modes:

No.	Mode	Description
1)	no	It is the default autowiring mode. It means no autowiring bydefault.
2)	byName	The byName mode injects the object dependency according to name of the bean. In such case, property name and bean name must be same. It internally calls setter method.
3)	byType	The byType mode injects the object dependency according to type. So property name and bean name can be different. It internally calls setter method.
4)	constructor	The constructor mode injects the dependency by calling the constructor of the class. It calls the constructor having large number of parameters.
5)	autodetect	It is deprecated since Spring 3.

Example of Autowiring

Let's see the simple code to use autowiring in spring. You need to use autowire attribute of bean element to apply the autowire modes.

```
1. <bean id="a" class="org.matru.A" autowire="byName"></bean>
```

Let's see the full example of autowiring in spring. To create this example, we have created 4 files.

1. **B.java**
2. **A.java**
3. **applicationContext.xml**
4. **Test.java**

B.java

This class contains a constructor and method only.

```
1. package org.matru;
2. public class B {
3. B(){System.out.println("b is created");}
4. void print(){System.out.println("hello b");}
5. }
```

A.java

This class contains reference of B class and constructor and method.

```
1. package org.matru;
2. public class A {
3. B b;
4. A(){System.out.println("a is created");}
5. public B getB() {
6.     return b;
7. }
8. public void setB(B b) {
9.     this.b = b;
10.}
11.void print(){System.out.println("hello a");}
12.void display(){
13.    print();
14.    b.print();
15.}
16.}
```

applicationContext.xml

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans
```

```
3.      xmlns="http://www.springframework.org/schema/beans"
4.      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.      xmlns:p="http://www.springframework.org/schema/p"
6.      xsi:schemaLocation="http://www.springframework.org/schema/beans
7.  http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
8.
9.  <bean id="b" class="org.matru.B"></bean>
10. <bean id="a" class="org.matru.A" autowire="byName"></bean>
11.
12. </beans>
```

Test.java

This class gets the bean from the applicationContext.xml file and calls the display method.

```
1.  package org.matru;
2.  import org.springframework.context.ApplicationContext;
3.  import org.springframework.context.support.ClassPathXmlApplicationContext;
4.  public class Test {
5.  public static void main(String[] args) {
6.      ApplicationContext context=new ClassPathXmlApplicationContext("applicationContex
    t.xml");
7.      A a=context.getBean("a",A.class);
8.      a.display();
9.  }
10. }
```

Output:

```
b is created
a is created
hello a
hello b
```

---

1) byName autowiring mode

In case of byName autowiring mode, bean id and reference name must be same.

It internally uses setter injection.

```
1.  <bean id="b" class="org.matru.B"></bean>
2.                  <bean          id="a"          class="org.matru.A"
```

autowire="byName"></bean> But, if you change the name of bean, it

will not inject the dependency.

Let's see the code where we are changing the name of the bean from b to b1.

```
1.  <bean id="b1" class="org.matru.B"></bean>
2.  <bean id="a" class="org.matru.A" autowire="byName"></bean>
```

2) byType autowiring mode

In case of byType autowiring mode, bean id and reference name may be different. But there must be only one bean of a type.

It internally uses setter injection.

```
1. <bean id="b1" class="org.matru.B"></bean>
2. <bean id="a" class="org.matru.A" autowire="byType"></bean>
```

In this case, it works fine because you have created an instance of B type. It doesn't matter that you have different bean name than reference name.

But, if you have multiple bean of one type, it will not work and throw exception.

Let's see the code where are many bean of type B.

```
1. <bean id="b1" class="org.matru.B"></bean>
2. <bean id="b2" class="org.matru.B"></bean>
3. <bean id="a" class="org.matru.A" autowire="byName"></bean>
```

In such case, it will throw exception.

3) constructor autowiring mode

In case of constructor autowiring mode, spring container injects the dependency by highest parameterized constructor.

If you have 3 constructors in a class, zero-arg, one-arg and two-arg then injection will be performed by calling the two-arg constructor.

```
1. <bean id="b" class="org.matru.B"></bean>
2. <bean id="a" class="org.matru.A" autowire="constructor"></bean>
```

4) no autowiring mode

In case of no autowiring mode, spring container doesn't inject the dependency by autowiring.

```
1. <bean id="b" class="org.matru.B"></bean>
2. <bean id="a" class="org.matru.A" autowire="no"></bean>
```

☺ Factory Method

Dependency Injection with Factory Method in Spring

Spring framework provides facility to inject bean using factory method. To do so, we can use two attributes of bean element.

1. **factory-method:** represents the factory method that will be invoked to inject the bean.
2. **factory-bean:** represents the reference of the bean by which factory method will be invoked. It is used if factory method is non-static.

A method that returns instance of a class is called **factory method**.

```
1. public class A {
2. public static A getA(){//factory method
3.     return new A();
4. }
5. }
```

Factory Method Types

There can be three types of factory method:

1) A **static factory method** that returns instance of **its own** class. It is used in singleton design pattern.

```
1. <bean id="a" class="com.smgc.A" factory-method="getA"></bean>
```

2) A **static factory method** that returns instance of **another** class. It is used instance is not known and decided at runtime.

```
1. <bean id="b" class="com.smgc.A" factory-method="getB"></bean>
```

3) A **non-static factory** method that returns instance of **another** class. It is used instance is not known and decided at runtime.

```
1. <bean id="a" class="com.smgc.A"></bean>
2.   <bean id="b" class="com.smgc.A" factory-method="getB" factory-
   bean="a"></bean>
```

Type 1

Let's see the simple code to inject the dependency by static factory method.

```
1. <bean id="a" class="com.smgc.A" factory-method="getA"></bean>
```

Let's see the full example to inject dependency using factory method in spring. To create this example, we have created 3 files.

1. **A.java**
2. **applicationContext.xml**
3. **Test.java**

A.java

This class is a singleton class.

```
1. package com.smgc;
2. public class A {
3. private static final A obj=new A();
4. private A(){System.out.println("private constructor");}
5. public static A getA(){
6.     System.out.println("factory method ");
7.     return obj;
8. }
9. public void msg(){
10.    System.out.println("hello user");
11.}
12.}
```

applicationContext.xml

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans
3.     xmlns="http://www.springframework.org/schema/beans"
4.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.     xmlns:p="http://www.springframework.org/schema/p"
6.     xsi:schemaLocation="http://www.springframework.org/schema/beans
7. http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
8.
9. <bean id="a" class="com.smgc.A" factory-method="getA"></bean>
10.
11.</beans>
```

Test.java

This class gets the bean from the applicationContext.xml file and calls the msg method.

```
1. package org.matru;
2. import org.springframework.context.ApplicationContext;
3. import org.springframework.context.support.ClassPathXmlApplicationContext;
4. public class Test {
5. public static void main(String[] args) {
6.     ApplicationContext context=new ClassPathXmlApplicationContext("applicationContex
   t.xml");
7.     A a=(A)context.getBean("a");
```

```
8.      a.msg();
9.  }
10. }
```

Output:

```
private  constructor
factory       method
hello user
```

Type 2

Let's see the simple code to inject the dependency by static factory method that returns the instance of another class.

To create this example, we have created 6 files.

1. **Printable.java**
2. **A.java**
3. **B.java**
4. **PrintableFactory.java**
5. **applicationContext.xml**
6. **Test.java**

Printable.java

```
1. package com.smgc;
2. public interface Printable {
3. void print();
4. }
```

A.java

```
1. package com.smgc;
2. public class A implements Printable{
3.     @Override
4.     public void print() {
5.         System.out.println("hello a");
6.     }
7.
8. }
```

B.java

```
1. package com.smgc;
2. public class B implements Printable{
3.     @Override
4.     public void print() {
5.         System.out.println("hello b");
```

```
6.      }
7.  }
```

PrintableFactory.java

```
1.  package com.smgc;
2.  public class PrintableFactory {
3.  public static Printable getPrintable(){
4.      //return new B();
5.          return new A();//return any one instance, either A or B
6.  }
7.  }
```

applicationContext.xml

```
1.  <?xml version="1.0" encoding="UTF-8"?>
2.  <beans
3.      xmlns="http://www.springframework.org/schema/beans"
4.      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.      xmlns:p="http://www.springframework.org/schema/p"
6.      xsi:schemaLocation="http://www.springframework.org/schema/beans
7.  http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
8.
9.     <bean id="p" class="com.smgc.PrintableFactory" factory-
    method="getPrintable"></bean>
10.
11. </beans>
```

Test.java

This class gets the bean from the applicationContext.xml file and calls the print() method.

```
1.  package org.matru;
2.  import org.springframework.context.ApplicationContext;
3.  import org.springframework.context.support.ClassPathXmlApplicationContext;
4.  public class Test {
5.  public static void main(String[] args) {
6.      ApplicationContext context=new ClassPathXmlApplicationContext("applicationContex
    t.xml");
7.      Printable p=(Printable)context.getBean("p");
8.      p.print();
9.  }
10. }
```

Output:

```
hello a
```

Type 3

Let's see the example to inject the dependency by non-static factory method that returns the instance of another class.

To create this example, we have created 6 files.

1. **Printable.java**
2. **A.java**
3. **B.java**
4. **PrintableFactory.java**
5. **applicationContext.xml**
6. **Test.java**

All files are same as previous, you need to change only 2 files: PrintableFactory and applicationContext.xml.

PrintableFactory.java

```
1. package com.smgc;
2. public class PrintableFactory {
3. //non-static factory method
4. public Printable getPrintable(){
5.     return new A();//return any one instance, either A or B
6. }
7. }
```

applicationContext.xml

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans
3.     xmlns="http://www.springframework.org/schema/beans"
4.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.     xmlns:p="http://www.springframework.org/schema/p"
6.     xsi:schemaLocation="http://www.springframework.org/schema/beans
7. http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
8.
9. <bean id="pfactory" class=“com.smgc.PrintableFactory"></bean>
10. <bean id="p" class=“com.smgc.PrintableFactory" factory-method="getPrintable"
11. factory-bean="pfactory"></bean>
12.
13. </beans>
```

Output:

```
hello a
```

Chapter-1.3

Spring AOP

☺ AOP Terminology

Spring AOP Tutorial

Aspect Oriented Programming (AOP) compliments OOPs in the sense that it also provides modularity. But the key unit of modularity is aspect than class.

AOP breaks the program logic into distinct parts (called concerns). It is used to increase modularity by **cross-cutting concerns**.

A **cross-cutting concern** is a concern that can affect the whole application and should be centralized in one location in code as possible, such as transaction management, authentication, logging, security etc.

Why use AOP?

It provides the pluggable way to dynamically add the additional concern before, after or around the actual logic. Suppose there are 10 methods in a class as given below:

```
1. class A{
2. public void m1(){...}
3. public void m2(){...}
4. public void m3(){...}
5. public void m4(){...}
6. public void m5(){...}
7. public void n1(){...}
8. public void n2(){...}
9. public void p1(){...}
10. public void p2(){...}
11. public void p3(){...}
12. }
```

There are 5 methods that starts from m, 2 methods that starts from n and 3 methods that starts from p.

Understanding Scenario I have to maintain log and send notification after calling methods that starts from m.

Problem without AOP We can call methods (that maintains log and sends notification) from the methods starting with m. In such scenario, we need to write the code in all the 5 methods.

But, if client says in future, I don't have to send notification, you need to change all the methods. It leads to the maintenance problem.

Solution with AOP We don't have to call methods from the method. Now we can define the additional concern like maintaining log, sending notification etc. in the method of a class. Its entry is given in the xml file.

In future, if client says to remove the notifier functionality, we need to change only in the xml file. So, maintenance is easy in AOP.

Where use AOP?

AOP is mostly used in following cases:

- to provide declarative enterprise services such as declarative transaction management.
- It allows users to implement custom aspects.

AOP Concepts and Terminology

AOP concepts and terminologies are as follows:

- Join point
- Advice
- Pointcut
- Introduction
- Target Object
- Aspect
- Interceptor
- AOP Proxy
- Weaving

Join point

Join point is any point in your program such as method execution, exception handling, field access etc. Spring supports only method execution join point.

Pointcut

It is an expression language of AOP that matches join points.

Introduction

It means introduction of additional method and fields for a type. It allows you to introduce new interface to any advised object.

Target Object

It is the object i.e. being advised by one or more aspects. It is also known as proxied object in spring because Spring AOP is implemented using runtime proxies.

Aspect

It is a class that contains advices, joinpoints etc.

Interceptor

It is an aspect that contains only one advice.

AOP Proxy

It is used to implement aspect contracts, created by AOP framework. It will be a JDK dynamic proxy or CGLIB proxy in spring framework.

Weaving

It is the process of linking aspect with other application types or objects to create an advised object. Weaving can be done at compile time, load time or runtime. Spring AOP performs weaving at runtime.

☺ AOP Implementations

AOP Implementations

AOP implementations are provided by:

1. AspectJ
2. Spring AOP
3. JBoss AOP

Spring AOP

Spring AOP can be used by 3 ways given below. But the widely used approach is Spring AspectJ Annotation Style. The 3 ways to use spring AOP are given below:

1. By Spring1.2 Old style (dtd based) (also supported in Spring3)
2. By AspectJ annotation-style
3. By Spring XML configuration-style(schema based)

1. Spring AOP Example

There are given examples of **Spring1.2 old style AOP** (dtd based) implementation.

Though it is supported in spring 3, but it is recommended to use spring aop with aspectJ that we are going to learn in next page.

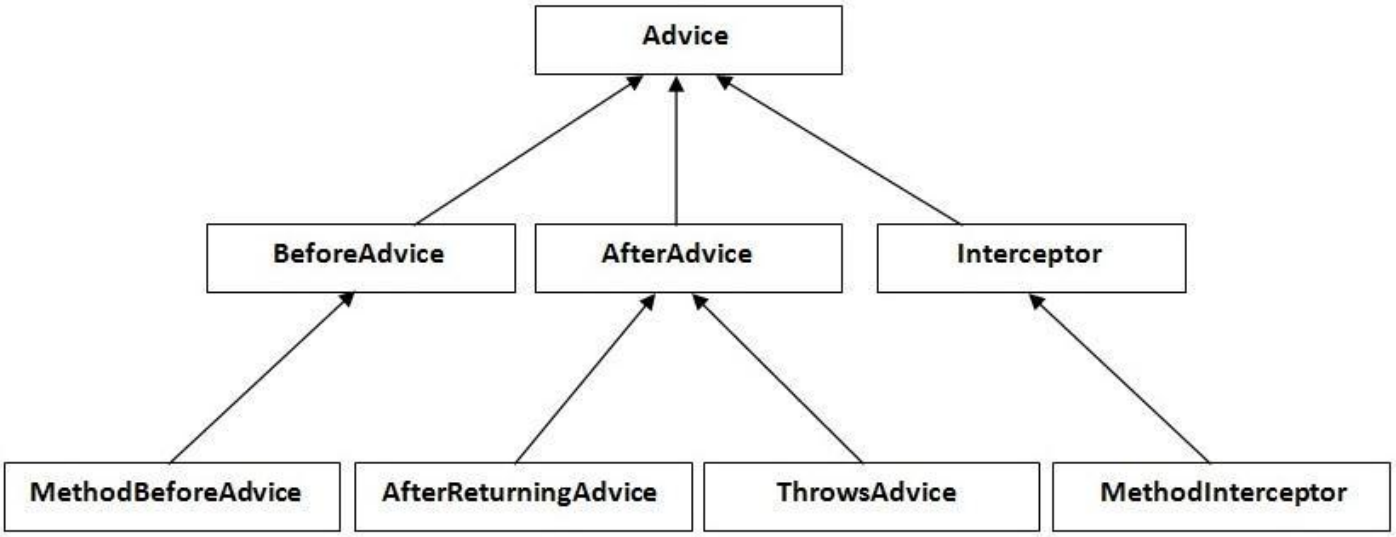
There are 4 types of advices supported in spring1.2 old style aop implementation.

1. **Before Advice** it is executed before the actual method call.
2. **After Advice** it is executed after the actual method call. If method returns a value, it is executed after returning value.
3. **Around Advice** it is executed before and after the actual method call.
4. **Throws Advice** it is executed if actual method throws exception.

To understand the basic concepts of Spring AOP, visit the previous page.

Understanding the hierarchy of advice interfaces

Let's understand the advice hierarchy by the diagram given below:



All are interfaces in aop.

MethodBeforeAdvice interface extends the **BeforeAdvice** interface.

AfterReturningAdvice interface extends the **AfterAdvice** interface.

ThrowsAdvice interface extends the **AfterAdvice** interface.

MethodInterceptor interface extends the **Interceptor** interface. It is used in around advice.

---

1) MethodBeforeAdvice Example

Create a class that contains actual business logic.

File: A.java

```
1. package com.smgc;
2. public class A {
3. public void m(){System.out.println("actual business logic");}
4. }
```

Now, create the advisor class that implements MethodBeforeAdvice interface.

File: BeforeAdvisor.java

```
1. package com.smgc;
2. import java.lang.reflect.Method;
3. import org.springframework.aop.MethodBeforeAdvice;
4. public class BeforeAdvisor implements MethodBeforeAdvice{
5.     @Override
6.     public void before(Method method, Object[] args, Object target)throws Throwable {
7.         System.out.println("additional concern before actual logic");
8.     }
```

```
9. }
```

In xml file, create 3 beans, one for A class, second for Advisor class and third for **ProxyFactoryBean** class.

File: applicationContext.xml

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans
3.     xmlns="http://www.springframework.org/schema/beans"
4.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.     xmlns:p="http://www.springframework.org/schema/p"
6.     xsi:schemaLocation="http://www.springframework.org/schema/beans
7. http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
8.
9. <bean id="obj" class="com.smgc.A"></bean>
10. <bean id="ba" class="com.smgc.BeforeAdvisor"></bean>
11.
12. <bean id="proxy" class="org.springframework.aop.framework.ProxyFactoryBean">
13. <property name="target" ref="obj"></property>
14. <property name="interceptorNames">
15. <list>
16. <value>ba</value>
17. </list>
18. </property>
19. </bean>
20.
21. </beans>
```

Understanding ProxyFactoryBean class:

The **ProxyFactoryBean** class is provided by Spring Famework. It contains 2 properties target and interceptorNames. The instance of A class will be considered as target object and the instance of advisor class as interceptor. You need to pass the advisor object as the list object as in the xml file given above.

The ProxyFactoryBean class is written something like this:

```
1. public class ProxyFactoryBean{
2. private Object target;
3. private List interceptorNames;
4. //getters and setters
5. }
```

Now, let's call the actual method.

File: Test.java

```
1. package com.smgc;
2. import org.springframework.beans.factory.BeanFactory;
3. import org.springframework.beans.factory.xml.XmlBeanFactory;
4. import org.springframework.core.io.ClassPathResource;
5. import org.springframework.core.io.Resource;
```

```
6.  public class Test {
7.  public static void main(String[] args) {
8.      Resource r=new ClassPathResource("applicationContext.xml");
9.      BeanFactory factory=new XmlBeanFactory(r);
10.
11.     A a=factory.getBean("proxy",A.class);
12.     a.m();
13. }
14. }
```

Output

```
1. additional concern before actual logic
2. actual business logic
```

Printing additional information in MethodBeforeAdvice

We can print additional information like method name, method argument, target object, target object class name, proxy class etc.

You need to change only two classes BeforeAdvisor.java and Test.java.

File: BeforeAdvisor.java

```
1.  package com.smgc;
2.  import java.lang.reflect.Method;
3.  import org.springframework.aop.MethodBeforeAdvice;
4.
5.  public class BeforeAdvisor implements MethodBeforeAdvice{
6.      @Override
7.      public void before(Method method, Object[] args, Object target)throws Throwable {
8.          System.out.println("additional concern before actual logic");
9.          System.out.println("method info:"+method.getName()+" "+method.getModifiers());
10.         System.out.println("argument info:");
11.         for(Object arg:args)
12.             System.out.println(arg);
13.         System.out.println("target Object:"+target);
14.         System.out.println("target object class name: "+target.getClass().getName());
15.     }
16. }
```

File: Test.java

```
1.  package com.smgc;
2.  import org.springframework.beans.factory.BeanFactory;
3.  import org.springframework.beans.factory.xml.XmlBeanFactory;
4.  import org.springframework.core.io.ClassPathResource;
5.  import org.springframework.core.io.Resource;
6.  public class Test {
7.  public static void main(String[] args) {
8.      Resource r=new ClassPathResource("applicationContext.xml");
9.      BeanFactory factory=new XmlBeanFactory(r);
10.
```

```
11.     A a=factory.getBean("proxy",A.class);
12.         System.out.println("proxy class name: "+a.getClass().getName());
13.     a.m();
14. }
15. }
```

Output

```
1. proxy class name: com.javatpoint.A$$EnhancerByCGLIB$$409872b1
2. additional concern before actual logic
3. method info:m 1
4. argument info:
5. target Object:com.javatpoint.A@11dba45
6. target object class name: com.javatpoint.A
7. actual business logic
```

---

2) AfterReturningAdvice Example

Create a class that contains actual business logic.

File: A.java

Same as in the previous example.

Now, create the advisor class that implements AfterReturningAdvice interface.

File: AfterAdvisor.java

```
1. package com.smgc;
2. import java.lang.reflect.Method;
3. import org.springframework.aop.AfterReturningAdvice;
4. public class AfterAdvisor implements AfterReturningAdvice{
5.     @Override
6.     public void afterReturning(Object returnValue, Method method,
7.          Object[] args, Object target) throws Throwable {
8.
9.         System.out.println("additional concern after returning advice");
10.    }
11.
12. }
```

Create the xml file as in the previous example, you need to change only the advisor class here.

File: applicationContext.xml

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans
3.     xmlns="http://www.springframework.org/schema/beans"
4.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
5.      xmlns:p="http://www.springframework.org/schema/p"
6.      xsi:schemaLocation="http://www.springframework.org/schema/beans
7.   http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
8.
9.   <bean id="obj" class="com.smgc.A"></bean>
10. <bean id="ba" class="com.smgc.AfterAdvisor"></bean>
11.
12. <bean id="proxy" class="org.springframework.aop.framework.ProxyFactoryBean">
13. <property name="target" ref="obj"></property>
14. <property name="interceptorNames">
15. <list>
16. <value>ba</value>
17. </list>
18. </property>
19. </bean>
20.
21.  </beans>
```

File: Test.java

Same as in the previous example.

Output

1. actual business logic
2. additional concern after returning advice

---

3) MethodInterceptor (AroundAdvice) Example

Create a class that contains actual business logic.

File: A.java

Same as in the previous example.

Now, create the advisor class that implements MethodInterceptor interface.

File: AroundAdvisor.java

```
1.  package com.smgc;
2.  import org.aopalliance.intercept.MethodInterceptor;
3.  import org.aopalliance.intercept.MethodInvocation;
4.  public class AroundAdvisor implements MethodInterceptor{
5.
6.      @Override
7.      public Object invoke(MethodInvocation mi) throws Throwable {
8.          Object obj;
```

```
9.          System.out.println("additional concern before actual logic");
10.         obj=mi.proceed();
11.         System.out.println("additional concern after actual logic");
12.         return obj;
13.     }
14.
15. }
```

Create the xml file as in the previous example, you need to change only the advisor class here.

File: applicationContext.xml

```
1.  <?xml version="1.0" encoding="UTF-8"?>
2.  <beans
3.      xmlns="http://www.springframework.org/schema/beans"
4.      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.      xmlns:p="http://www.springframework.org/schema/p"
6.      xsi:schemaLocation="http://www.springframework.org/schema/beans
7.  http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
8.
9.  <bean id="obj" class="com.smgc.A"></bean>
10. <bean id="ba" class="com.smgc.AroundAdvisor"></bean>
11.
12. <bean id="proxy" class="org.springframework.aop.framework.ProxyFactoryBean">
13. <property name="target" ref="obj"></property>
14. <property name="interceptorNames">
15. <list>
16. <value>ba</value>
17. </list>
18. </property>
19. </bean>
20.
21.  </beans>
```

File: Test.java

Same as in the previous example.

Output

1. additional concern before actual logic
2. actual business logic
3. additional concern after actual logic

---

4) ThrowsAdvice Example

Create a class that contains actual business logic.

File: Validator.java

```
1.  package com.smgc;
2.  public class Validator {
3.      public void validate(int age)throws Exception{
4.          if(age<18){
5.              throw new ArithmeticException("Not Valid Age");
6.          }
7.          else{
8.              System.out.println("vote confirmed");
9.          }
10.     }
11. }
```

Now, create the advisor class that implements ThrowsAdvice interface.

File: ThrowsAdvisor.java

```
1.  package com.smgc;
2.  import org.springframework.aop.ThrowsAdvice;
3.  public class ThrowsAdvisor implements ThrowsAdvice{
4.      public void afterThrowing(Exception ex){
5.          System.out.println("additional concern if exception occurs");
6.      }
7.  }
```

Create the xml file as in the previous example, you need to change only the Validator class and advisor class.

File: applicationContext.xml

```
1.  <?xml version="1.0" encoding="UTF-8"?>
2.  <beans
3.      xmlns="http://www.springframework.org/schema/beans"
4.      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.      xmlns:p="http://www.springframework.org/schema/p"
6.      xsi:schemaLocation="http://www.springframework.org/schema/beans
7.  http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
8.
9.  <bean id="obj" class="com.smgc.Validator"></bean>
10. <bean id="ba" class="com.smgc.ThrowsAdvisor"></bean>
11.
12. <bean id="proxy" class="org.springframework.aop.framework.ProxyFactoryBean">
13. <property name="target" ref="obj"></property>
14. <property name="interceptorNames">
15. <list>
16. <value>ba</value>
17. </list>
18. </property>
19. </bean>
20.
21. </beans>
```

File: Test.java

```
1. package com.smgc;
2.
3. import org.springframework.beans.factory.BeanFactory;
4. import org.springframework.beans.factory.xml.XmlBeanFactory;
5. import org.springframework.core.io.ClassPathResource;
6. import org.springframework.core.io.Resource;
7.
8. public class Test {
9. public static void main(String[] args) {
10.    Resource r=new ClassPathResource("applicationContext.xml");
11.    BeanFactory factory=new XmlBeanFactory(r);
12.
13.    Validator v=factory.getBean("proxy",Validator.class);
14.    try{
15.    v.validate(12);
16.    }catch(Exception e){e.printStackTrace();}
17. }
18. }
```

Output

```
1. java.lang.ArithmeticException: Not Valid Age
2.
3. additional concern if exception occurs
4.
5.      at com.javatpoint.Validator.validate(Validator.java:7)
6.      at com.javatpoint.Validator$$FastClassByCGLIB$$562915cf.invoke(<generated>)
7.      at net.sf.cglib.proxy.MethodProxy.invoke(MethodProxy.java:191)
8.      at org.springframework.aop.framework.Cglib2AopProxy$CglibMethodInvocation.invoke
9. Joinpoint(Cglib2AopProxy.java:692)
10.     at org.springframework.aop.framework.ReflectiveMethodInvocation.
11. proceed(ReflectiveMethodInvocation.java:150)
12.     at org.springframework.aop.framework.adapter.ThrowsAdviceInterceptor.
13. invoke(ThrowsAdviceInterceptor.java:124)
14.     at org.springframework.aop.framework.ReflectiveMethodInvocation.
15. proceed(ReflectiveMethodInvocation.java:172)
16.     at org.springframework.aop.framework.Cglib2AopProxy$DynamicAdvisedInterceptor.
17. intercept(Cglib2AopProxy.java:625)
18.     at com.javatpoint.Validator$$EnhancerByCGLIB$$4230ed28.validate(<generated>)
19.     at com.javatpoint.Test.main(Test.java:15)
```

2. **Spring AOP AspectJ Annotation Example**

1. @Before Example
2. @After Example
3. @AfterReturning Example
4. @Around Example
5. @AfterThrowing Example

The **Spring Framework** recommends you to use **Spring AspectJ AOP implementation** over the Spring 1.2 old style dtd based AOP implementation because it provides you more control and it is easy to use.

There are two ways to use Spring AOP AspectJ implementation:

1. By annotation: We are going to learn it here.
2. By xml configuration (schema based): We will learn it in next page.

Spring AspectJ AOP implementation provides many annotations:

1. @**Aspect** declares the class as aspect.
2. @**Pointcut** declares the pointcut expression.

The annotations used to create advices are given below:

1. @**Before** declares the before advice. It is applied before calling the actual method.
2. @**After** declares the after advice. It is applied after calling the actual method and before returning result.
3. @**AfterReturning** declares the after returning advice. It is applied after calling the actual method and before returning result. But you can get the result value in the advice.
4. @**Around** declares the around advice. It is applied before and after calling the actual method.
5. @**AfterThrowing** declares the throws advice. It is applied if actual method throws exception.

Understanding Pointcut

Pointcut is an expression language of Spring AOP.

The @**Pointcut** annotation is used to define the pointcut. We can refer the pointcut expression by name also. Let's see the simple example of pointcut expression.

```
1. @Pointcut("execution(* Operation.*(..))")
2. private void doSomething() {}
```

The name of the pointcut expression is doSomething(). It will be applied on all the methods of Operation class regardless of return type.

Understanding Pointcut Expressions

Let's try the understand the pointcut expressions by the examples given below:

1. @Pointcut("execution(public * *(..))")

It will be applied on all the public methods.

---

1. @Pointcut("execution(public Operation.*(..))")

It will be applied on all the public methods of Operation class.

---

1. @Pointcut("execution(* Operation.*(..))")

It will be applied on all the methods of Operation class.

---

1. @Pointcut("execution(public Employee.set*(..))")

It will be applied on all the public setter methods of Employee class.

---

1. @Pointcut("execution(int Operation.*(..))")

It will be applied on all the methods of Operation class that returns int value.

---

1) @Before Example

The AspectJ Before Advice is applied before the actual business logic method. You can perform any operation here such as conversion, authentication etc.

Create a class that contains actual business logic.

File: Operation.java

```
1. package com.smgc;
2. public  class Operation{
3.     public void msg(){System.out.println("msg method invoked");}
4.     public int m(){System.out.println("m method invoked");return 2;}
5.     public int k(){System.out.println("k method invoked");return 3;}
6. }
```

Now, create the aspect class that contains before advice.

File: TrackOperation.java

```
1. package com.smgc;
2.
3. import org.aspectj.lang.JoinPoint;
4. import org.aspectj.lang.annotation.Aspect;
5. import org.aspectj.lang.annotation.Before;
6. import org.aspectj.lang.annotation.Pointcut;
7.
8. @Aspect
9. public class TrackOperation{
10.    @Pointcut("execution(* Operation.*(..))")
11.    public void k(){}//pointcut name
12.
13.    @Before("k()")//applying pointcut on before advice
14.    public void myadvice(JoinPoint jp)//it is advice (before advice)
15.    {
16.        System.out.println("additional concern");
17.        //System.out.println("Method Signature: "  + jp.getSignature());
18.    }
19. }
```

Now create the applicationContext.xml file that defines beans.

File: applicationContext.xml

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans xmlns="http://www.springframework.org/schema/beans"
3.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.     xmlns:aop="http://www.springframework.org/schema/aop"
5.       xsi:schemaLocation="http://www.springframework.org/schema/beans
6.       http://www.springframework.org/schema/beans/spring-beans.xsd
7.       http://www.springframework.org/schema/aop
8.       http://www.springframework.org/schema/aop/spring-aop.xsd">
9.
10.
11.    <bean id="opBean" class="com.smgc.Operation">   </bean>
12.    <bean id="trackMyBean" class="com.smgc.TrackOperation"></bean>
13.
14.    <bean   class="org.springframework.aop.aspectj.annotation.AnnotationAwareAspectJAutoProx
    yCreator"></bean>
15.
16. </beans>
```

Now, let's call the actual method.

File: Test.java

```
1. package com.smgc;
2.
3. import org.springframework.context.ApplicationContext;
4. import org.springframework.context.support.ClassPathXmlApplicationContext;
5. public class Test{
6.     public static void main(String[] args){
```

```
7.          ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
8.          Operation e = (Operation) context.getBean("opBean");
9.          System.out.println("calling msg...");
10.         e.msg();
11.         System.out.println("calling m...");
12.         e.m();
13.         System.out.println("calling k...");
14.         e.k();
15.     }
16. }
```

Output

```
1. calling msg...
2. additional concern
3. msg() method invoked
4. calling m...
5. additional concern
6. m() method invoked
7. calling k...
8. additional concern
9. k() method invoked
```

As you can see, additional concern is printed before msg(), m() and k() method is invoked.

Now if you change the pointcut expression as given below:

```
1. @Pointcut("execution(* Operation.m*(..))")
```

Now additional concern will be applied for the methods starting with m in Operation class. Output will be as this:

```
1. calling msg...
2. additional concern
3. msg() method invoked
4. calling m...
5. additional concern
6. m() method invoked
7. calling k...
8. k() method invoked
```

Now you can see additional concern is not printed before k() method invoked.

---

2) @After Example

The AspectJ after advice is applied after calling the actual business logic methods. It can be used to maintain log, security, notification etc.

Here, We are assuming that **Operation.java**, **applicationContext.xml** and **Test.java** files are same as given in @Before example.

Create the aspect class that contains after advice.

File: TrackOperation.java

```
1.  package com.smgc;
2.
3.  import org.aspectj.lang.JoinPoint;
4.  import org.aspectj.lang.annotation.Aspect;
5.  import org.aspectj.lang.annotation.After;
6.  import org.aspectj.lang.annotation.Pointcut;
7.
8.  @Aspect
9.  public class TrackOperation{
10.     @Pointcut("execution(* Operation.*(..))")
11.     public void k(){}//pointcut name
12.
13.     @After("k()")//applying pointcut on after advice
14.     public void myadvice(JoinPoint jp)//it is advice (after advice)
15.     {
16.         System.out.println("additional concern");
17.         //System.out.println("Method Signature: "  + jp.getSignature());
18.     }
19. }
```

Output

```
1.  calling msg...
2.  msg() method invoked
3.  additional concern
4.  calling m...
5.  m() method invoked
6.  additional concern
7.  calling k...
8.  k() method invoked
9.  additional concern
```

You can see that additional concern is printed after calling msg(), m() and k() methods.

---

3) @AfterReturning Example

By using after returning advice, we can get the result in the advice.

Create the class that contains business logic.

File: Operation.java

```
1. package com.smgc;
2. public  class Operation{
3.     public int m(){System.out.println("m() method invoked");return 2;}
4.     public int k(){System.out.println("k() method invoked");return 3;}
5. }
```

Create the aspect class that contains after returning advice.

File: TrackOperation.java

```
1. package com.smgc;
2.
3. import org.aspectj.lang.JoinPoint;
4. import org.aspectj.lang.annotation.AfterReturning;
5. import org.aspectj.lang.annotation.Aspect;
6.
7. @Aspect
8. public class TrackOperation{
9.     @AfterReturning(
10.             pointcut = "execution(* Operation.*(..))",
11.             returning= "result")
12.
13.    public void myadvice(JoinPoint jp,Object result)//it is advice (after returning advice)
14.    {
15.        System.out.println("additional concern");
16.        System.out.println("Method Signature: "  + jp.getSignature());
17.        System.out.println("Result in advice: "+result);
18.        System.out.println("end of after returning advice...");
19.    }
20. }
```

File: applicationContext.xml

It is same as given in @Before advice example

File: Test.java

Now create the Test class that calls the actual methods.

```
1. package com.smgc;
2.
3. import org.springframework.context.ApplicationContext;
4. import org.springframework.context.support.ClassPathXmlApplicationContext;
5. public class Test{
6.     public static void main(String[] args){
7.         ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
8.         Operation e = (Operation) context.getBean("opBean");
9.         System.out.println("calling m...");
10.        System.out.println(e.m());
11.        System.out.println("calling k...");
12.        System.out.println(e.k());
13.    }
```

```
14. }
```

Output

```
1. calling m...
2. m() method invoked
3. additional concern
4. Method Signature: int com.javatpoint.Operation.m()
5. Result in advice: 2
6. end of after returning advice...
7. 2
8. calling k...
9. k() method invoked
10. additional concern
11. Method Signature: int com.javatpoint.Operation.k()
12. Result in advice: 3
13. end of after returning advice...
14. 3
```

You can see that return value is printed two times, one is printed by TrackOperation class and second by Test class.

---

4) @Around Example

The AspectJ around advice is applied before and after calling the actual business logic methods.

Here, we are assuming that **applicationContext.xml** file is same as given in @Before example.

Create a class that contains actual business logic.

File: Operation.java

```
1. package com.smgc;
2. public  class Operation{
3.     public void msg(){System.out.println("msg() is invoked");}
4.     public void display(){System.out.println("display() is invoked");}
5. }
```

Create the aspect class that contains around advice.

You need to pass the **PreceedingJoinPoint** reference in the advice method, so that we can proceed the request by calling the proceed() method.

File: TrackOperation.java

```
1. package com.smgc;
2. import org.aspectj.lang.ProceedingJoinPoint;
3. import org.aspectj.lang.annotation.Around;
```

```
4.  import org.aspectj.lang.annotation.Aspect;
5.  import org.aspectj.lang.annotation.Pointcut;
6.
7.  @Aspect
8.  public class TrackOperation
9.  {
10.     @Pointcut("execution(* Operation.*(..))")
11.     public void abcPointcut(){}
12.
13.     @Around("abcPointcut()")
14.     public Object myadvice(ProceedingJoinPoint pjp) throws Throwable
15.     {
16.         System.out.println("Additional Concern Before calling actual method");
17.         Object obj=pjp.proceed();
18.         System.out.println("Additional Concern After calling actual method");
19.         return obj;
20.     }
21. }
```

File: Test.java

Now create the Test class that calls the actual methods.

```
1.  package com.smgc;
2.  import org.springframework.context.ApplicationContext;
3.  import org.springframework.context.support.ClassPathXmlApplicationContext;
4.  public class Test{
5.      public static void main(String[] args){
6.          ApplicationContext context = new classPathXmlApplicationContext("applicationContext.xml
    ");
7.
8.          Operation op = (Operation) context.getBean("opBean");
9.          op.msg();
10.         op.display();
11.     }
12. }
```

Output

```
1.  Additional Concern Before calling actual method
2.  msg() is invoked
3.  Additional Concern After calling actual method
4.  Additional Concern Before calling actual method
5.  display() is invoked
6.  Additional Concern After calling actual method
```

You can see that additional concern is printed before and after calling msg() and display methods.

---

5) @AfterThrowing Example

By using after throwing advice, we can print the exception in the TrackOperation class. Let's see the example of AspectJ AfterThrowing advice.

Create the class that contains business logic.

File: Operation.java

```
1. package com.smgc;
2. public  class Operation{
3.     public void validate(int age)throws Exception{
4.     if(age<18){
5.         throw new ArithmeticException("Not valid age");
6.     }
7.     else{
8.         System.out.println("Thanks for vote");
9.     }
10.    }
11.
12. }
```

Create the aspect class that contains after throwing advice.

Here, we need to pass the Throwable reference also, so that we can intercept the exception here.

File: TrackOperation.java

```
1. package com.smgc;
2. import org.aspectj.lang.JoinPoint;
3. import org.aspectj.lang.annotation.AfterThrowing;
4. import org.aspectj.lang.annotation.Aspect;
5. @Aspect
6. public class TrackOperation{
7.     @AfterThrowing(
8.             pointcut = "execution(* Operation.*(..))",
9.             throwing= "error")
10.
11.    public void myadvice(JoinPoint jp,Throwable error)//it is advice
12.    {
13.        System.out.println("additional concern");
14.        System.out.println("Method Signature: "  + jp.getSignature());
15.        System.out.println("Exception is: "+error);
16.        System.out.println("end of after throwing advice...");
17.    }
18. }
```

File: applicationContext.xml

It is same as given in @Before advice example

File: Test.java

Now create the Test class that calls the actual methods.

```
1. package com.smgc;
2.
3. import org.springframework.context.ApplicationContext;
4. import org.springframework.context.support.ClassPathXmlApplicationContext;
5. public class Test{
6.     public static void main(String[] args){
7.         ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
8.         Operation op = (Operation) context.getBean("opBean");
9.         System.out.println("calling validate...");
10.        try{
11.            op.validate(19);
12.        }catch(Exception e){System.out.println(e);}
13.        System.out.println("calling validate again...");
14.
15.        try{
16.            op.validate(11);
17.        }catch(Exception e){System.out.println(e);}
18.    }
19. }
```

Output

```
1. calling validate...
2. Thanks for vote
3. calling validate again...
4. additional concern
5. Method Signature: void com.javatpoint.Operation.validate(int)
6. Exception is: java.lang.ArithmeticException: Not valid age
7. end of after throwing advice...
8. java.lang.ArithmeticException: Not valid age
```

---

3. **Spring AOP AspectJ Xml Configuration Example**

1. aop:before example
2. aop:after example
3. aop:after-returning example
4. aop:around example
5. aop:after-throwing example

Spring enables you to define the aspects, advices and pointcuts in xml file.

In the previous page, we have seen the aop examples using annotations. Now we are going to see same examples by the xml configuration file.

Let's see the xml elements that are used to define advice.

1. **aop:before** It is applied before calling the actual business logic method.
2. **aop:after** It is applied after calling the actual business logic method.
3. **aop:after-returning** it is applied after calling the actual business logic method. It can be used to intercept the return value in advice.
4. **aop:around** It is applied before and after calling the actual business logic method.
5. **aop:after-throwing** It is applied if actual business logic method throws exception.

---

1) aop:before Example

The AspectJ Before Advice is applied before the actual business logic method. You can perform any operation here such as conversion, authentication etc.

Create a class that contains actual business logic.

File: Operation.java

```
1. package com.smgc;
2. public  class Operation{
3.     public void msg(){System.out.println("msg method invoked");}
4.     public int m(){System.out.println("m method invoked");return 2;}
5.     public int k(){System.out.println("k method invoked");return 3;}
6. }
```

Now, create the aspect class that contains before advice.

File: TrackOperation.java

```
1. package com.smgc;
2. import org.aspectj.lang.JoinPoint;
3. public class TrackOperation{
4.     public void myadvice(JoinPoint jp)//it is advice
5.     {
6.         System.out.println("additional concern");
7.         //System.out.println("Method Signature: "  + jp.getSignature());
8.     }
9. }
```

Now create the applicationContext.xml file that defines beans.

File: applicationContext.xml

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans xmlns="http://www.springframework.org/schema/beans"
3.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.     xmlns:aop="http://www.springframework.org/schema/aop"
5.     xsi:schemaLocation="http://www.springframework.org/schema/beans
6.     http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
7.     http://www.springframework.org/schema/aop
8.     http://www.springframework.org/schema/aop/spring-aop-3.0.xsd ">
9.
10. <aop:aspectj-autoproxy />
```

```
11.
12. <bean id="opBean" class="com.smgc.Operation">   </bean>
13. <bean id="trackAspect" class="com.smgc.TrackOperation"></bean>
14.
15. <aop:config>
16.   <aop:aspect id="myaspect" ref="trackAspect" >
17.     <!-- @Before -->
18.     <aop:pointcut id="pointCutBefore"   expression="execution(* com.javatpoint.Operation.*(..))"
    />
19.     <aop:before method="myadvice" pointcut-ref="pointCutBefore" />
20.   </aop:aspect>
21. </aop:config>
22.
23. </beans>
```

Now, let's call the actual method.

File: Test.java

```
1.  package com.smgc;
2.
3.  import org.springframework.context.ApplicationContext;
4.  import org.springframework.context.support.ClassPathXmlApplicationContext;
5.  public class Test{
6.      public static void main(String[] args){
7.          ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xm
    l");
8.          Operation e = (Operation) context.getBean("opBean");
9.          System.out.println("calling msg...");
10.         e.msg();
11.         System.out.println("calling m...");
12.         e.m();
13.         System.out.println("calling k...");
14.         e.k();
15.     }
16. }
```

Output

```
1.  calling msg...
2.  additional concern
3.  msg() method invoked
4.  calling m...
5.  additional concern
6.  m() method invoked
7.  calling k...
8.  additional concern
9.  k() method invoked
```

As you can see, additional concern is printed before msg(), m() and k() method is invoked.

---

2) aop:after example

The AspectJ after advice is applied after calling the actual business logic methods. It can be used to maintain log, security, notification etc.

Here, We are assuming that **Operation.java**, **TrackOperation.java** and **Test.java** files are same as given in aop:before example.

Now create the applicationContext.xml file that defines beans.

File: applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:aop="http://www.springframework.org/schema/aop"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
    http://www.springframework.org/schema/aop
    http://www.springframework.org/schema/aop/spring-aop-3.0.xsd ">

<aop:aspectj-autoproxy />

<bean id="opBean" class="com.smgc.Operation">  </bean>
<bean id="trackAspect" class="com.smgc.TrackOperation"></bean>

<aop:config>
  <aop:aspect id="myaspect" ref="trackAspect" >
    <!-- @After -->
    <aop:pointcut id="pointCutAfter"    expression="execution(* com.javatpoint.Operation.*(..))" />
    <aop:after method="myadvice" pointcut-ref="pointCutAfter" />
  </aop:aspect>
</aop:config>

</beans>
```

Output

```
calling msg...
msg() method invoked
additional concern
calling m...
m() method invoked
additional concern
calling k...
k() method invoked
additional concern
```

You can see that additional concern is printed after calling msg(), m() and k() methods.

---

3) aop:after-returning example

By using after returning advice, we can get the result in the advice.

Create the class that contains business logic.

File: Operation.java

```
1. package com.smgc;
2. public  class Operation{
3.     public int m(){System.out.println("m() method invoked");return 2;}
4.     public int k(){System.out.println("k() method invoked");return 3;}
5. }
```

Create the aspect class that contains after returning advice.

File: TrackOperation.java

```
1. package com.smgc;
2.
3. import org.aspectj.lang.JoinPoint;
4.
5. public class TrackOperation{
6.     public void myadvice(JoinPoint jp,Object result)//it is advice (after advice)
7.     {
8.         System.out.println("additional concern");
9.         System.out.println("Method Signature: "  + jp.getSignature());
10.        System.out.println("Result in advice: "+result);
11.        System.out.println("end of after returning advice...");
12.    }
13. }
```

File: applicationContext.xml

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans xmlns="http://www.springframework.org/schema/beans"
3.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.     xmlns:aop="http://www.springframework.org/schema/aop"
5.     xsi:schemaLocation="http://www.springframework.org/schema/beans
6.     http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
7.     http://www.springframework.org/schema/aop
8.     http://www.springframework.org/schema/aop/spring-aop-3.0.xsd ">
9.
10. <aop:aspectj-autoproxy />
11.
12. <bean id="opBean" class="com.smgc.Operation">   </bean>
13.
14. <bean id="trackAspect" class="com.smgc.TrackOperation"></bean>
15.
16. <aop:config>
17.   <aop:aspect id="myaspect" ref="trackAspect" >
18.     <!-- @AfterReturning -->
```

```
19.     <aop:pointcut id="pointCutAfterReturning"    expression="execution(* com.javatpoint.Operatio
    n.*(..))" />
20.     <aop:after-returning method="myadvice" returning="result" pointcut-
    ref="pointCutAfterReturning" />
21.   </aop:aspect>
22. </aop:config>
23.
24.  </beans>
```

File: Test.java

Now create the Test class that calls the actual methods.

```
1.  package com.smgc;
2.
3.  import org.springframework.context.ApplicationContext;
4.  import org.springframework.context.support.ClassPathXmlApplicationContext;
5.  public class Test{
6.      public static void main(String[] args){
7.          ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xm
    l");
8.          Operation e = (Operation) context.getBean("opBean");
9.          System.out.println("calling m...");
10.         System.out.println(e.m());
11.         System.out.println("calling k...");
12.         System.out.println(e.k());
13.     }
14. }
```

Output

```
1.  calling m...
2.  m() method invoked
3.  additional concern
4.  Method Signature: int com.javatpoint.Operation.m()
5.  Result in advice: 2
6.  end of after returning advice...
7.  2
8.  calling k...
9.  k() method invoked
10. additional concern
11. Method Signature: int com.javatpoint.Operation.k()
12. Result in advice: 3
13. end of after returning advice...
14. 3
```

You can see that return value is printed two times, one is printed by TrackOperation class and second by Test class.

---

4) aop:around example

The AspectJ around advice is applied before and after calling the actual business logic methods.

Create a class that contains actual business logic.

File: Operation.java

```
1. package com.smgc;
2. public  class Operation{
3.     public void msg(){System.out.println("msg() is invoked");}
4.     public void display(){System.out.println("display() is invoked");}
5. }
```

Create the aspect class that contains around advice.

You need to pass the **PreceedingJoinPoint** reference in the advice method, so that we can proceed the request by calling the proceed() method.

File: TrackOperation.java

```
1. package com.smgc;
2. import org.aspectj.lang.ProceedingJoinPoint;
3. public class TrackOperation
4. {
5.     public Object myadvice(ProceedingJoinPoint pjp) throws Throwable
6.     {
7.         System.out.println("Additional Concern Before calling actual method");
8.         Object obj=pjp.proceed();
9.         System.out.println("Additional Concern After calling actual method");
10.        return obj;
11.    }
12. }
```

File: applicationContext.xml

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans xmlns="http://www.springframework.org/schema/beans"
3.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.     xmlns:aop="http://www.springframework.org/schema/aop"
5.     xsi:schemaLocation="http://www.springframework.org/schema/beans
6.     http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
7.     http://www.springframework.org/schema/aop
8.     http://www.springframework.org/schema/aop/spring-aop-3.0.xsd ">
9.
10. <aop:aspectj-autoproxy />
11.
12. <bean id="opBean" class="com.smgc.Operation">   </bean>
13.
14. <bean id="trackAspect" class="com.smgc.TrackOperation"></bean>
15.
16. <aop:config>
```

```
17.   <aop:aspect id="myaspect" ref="trackAspect" >
18.     <!-- @Around -->
19.     <aop:pointcut id="pointCutAround"   expression="execution(* com.javatpoint.Operation.*(..))"
      />
20.     <aop:around method="myadvice" pointcut-ref="pointCutAround" />
21.   </aop:aspect>
22. </aop:config>
23.
24. </beans>
```

File: Test.java

Now create the Test class that calls the actual methods.

```
1.  package com.smgc;
2.  import org.springframework.context.ApplicationContext;
3.  import org.springframework.context.support.ClassPathXmlApplicationContext;
4.  public class Test{
5.      public static void main(String[] args){
6.          ApplicationContext context = new classPathXmlApplicationContext("applicationContext.xml
    ");
7.
8.          Operation op = (Operation) context.getBean("opBean");
9.          op.msg();
10.         op.display();
11.     }
12. }
```

Output

```
1.  Additional Concern Before calling actual method
2.  msg() is invoked
3.  Additional Concern After calling actual method
4.  Additional Concern Before calling actual method
5.  display() is invoked
6.  Additional Concern After calling actual method
```

You can see that additional concern is printed before and after calling msg() and display methods.

---

5) aop:after-throwing example

By using after throwing advice, we can print the exception in the TrackOperation class. Let's see the example of AspectJ AfterThrowing advice.

Create the class that contains business logic.

File: Operation.java

```
1. package com.smgc;
2. public  class Operation{
3.     public void validate(int age)throws Exception{
4.     if(age<18){
5.         throw new ArithmeticException("Not valid age");
6.     }
7.     else{
8.         System.out.println("Thanks for vote");
9.     }
10.    }
11.
12. }
```

Create the aspect class that contains after throwing advice.

Here, we need to pass the Throwable reference also, so that we can intercept the exception here.

File: TrackOperation.java

```
1. package com.smgc;
2. import org.aspectj.lang.JoinPoint;
3. public class TrackOperation{
4.
5.     public void myadvice(JoinPoint jp,Throwable error)//it is advice
6.     {
7.         System.out.println("additional concern");
8.         System.out.println("Method Signature: "  + jp.getSignature());
9.         System.out.println("Exception is: "+error);
10.        System.out.println("end of after throwing advice...");
11.    }
12. }
```

File: applicationContext.xml

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans xmlns="http://www.springframework.org/schema/beans"
3.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.     xmlns:aop="http://www.springframework.org/schema/aop"
5.     xsi:schemaLocation="http://www.springframework.org/schema/beans
6.     http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
7.     http://www.springframework.org/schema/aop
8.     http://www.springframework.org/schema/aop/spring-aop-3.0.xsd ">
9.  <aop:aspectj-autoproxy />
10. <bean id="opBean" class="com.smgc.Operation">   </bean>
11. <bean id="trackAspect" class="com.smgc.TrackOperation"></bean>
12.
13. <aop:config>
14.   <aop:aspect id="myaspect" ref="trackAspect" >
15.     <!-- @AfterThrowing -->
16.     <aop:pointcut id="pointCutAfterThrowing"     expression="execution(* com.javatpoint.Operatio
    n.*(..))" />
17.     <aop:after-throwing method="myadvice" throwing="error" pointcut-
    ref="pointCutAfterThrowing" />
```

```
18.   </aop:aspect>
19. </aop:config>
20.  </beans>
```

File: Test.java

Now create the Test class that calls the actual methods.

```
1.  package com.smgc;
2.
3.  import org.springframework.context.ApplicationContext;
4.  import org.springframework.context.support.ClassPathXmlApplicationContext;
5.  public class Test{
6.      public static void main(String[] args){
7.          ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
8.          Operation op = (Operation) context.getBean("opBean");
9.          System.out.println("calling validate...");
10.         try{
11.             op.validate(19);
12.         }catch(Exception e){System.out.println(e);}
13.         System.out.println("calling validate again...");
14.
15.         try{
16.             op.validate(11);
17.         }catch(Exception e){System.out.println(e);}
18.     }
19. }
```

Output

```
1.  calling validate...
2.  Thanks for vote
3.  calling validate again...
4.  additional concern
5.  Method Signature: void com.javatpoint.Operation.validate(int)
6.  Exception is: java.lang.ArithmeticException: Not valid age
7.  end of after throwing advice...
8.  java.lang.ArithmeticException: Not valid age
```

☺ Pointcut

- As per above section.

☺ Advices

Advice represents an action taken by an aspect at a particular join point. There are different types of advices:

- **Before Advice**: it executes before a join point.
- **After Returning Advice**: it executes after a joint point completes normally.
- **After Throwing Advice**: it executes if method exits by throwing an exception.
- **After (finally) Advice**: it executes after a join point regardless of join point exit whether normally or exceptional return.
- **Around Advice**: It executes before and after a join point.

Chapter-2.1

Spring JDBC

☺ JDBC Template Example

Spring JdbcTemplate Tutorial

1. Spring JDBC Template
2. Understanding the need for Spring JDBC Template
3. Advantage of Spring JDBC Template
4. JDBC Template classes
5. Example of JdbcTemplate class

Spring **JdbcTemplate** is a powerful mechanism to connect to the database and execute SQL queries. It internally uses JDBC api, but eliminates a lot of problems of JDBC API.

Problems of JDBC API

The problems of JDBC API are as follows:

- We need to write a lot of code before and after executing the query, such as creating connection, statement, closing resultset, connection etc.
- We need to perform exception handling code on the database logic.
- We need to handle transaction.
- Repetition of all these codes from one to another database logic is a time consuming task.

Advantage of Spring JdbcTemplate

Spring JdbcTemplate eliminates all the above mentioned problems of JDBC API. It provides you methods to write the queries directly, so it saves a lot of work and time.

Spring Jdbc Approaches

Spring framework provides following approaches for JDBC database access:

- JdbcTemplate
- NamedParameterJdbcTemplate
- SimpleJdbcTemplate
- SimpleJdbcInsert and SimpleJdbcCall

JdbcTemplate class

It is the central class in the Spring JDBC support classes. It takes care of creation and release of resources such as creating and closing of connection object etc. So it will not lead to any problem if you forget to close the connection.

It handles the exception and provides the informative exception messages by the help of excepion classes defined in the **org.springframework.dao** package.

We can perform all the database operations by the help of JdbcTemplate class such as insertion, updation, deletion and retrieval of the data from the database.

Let's see the methods of spring JdbcTemplate class.

No.	Method	Description
1)	public int update(String query)	is used to insert, update and delete records.
2)	public int update(String query,Object... args)	is used to insert, update and delete records using PreparedStatement using given arguments.
3)	public void execute(String query)	is used to execute DDL query.
4)	public T execute(String sql, PreparedStatementCallback action)	executes the query by using PreparedStatement callback.
5)	public T query(String sql, ResultSetExtractor rse)	is used to fetch records using ResultSetExtractor.
6)	public List query(String sql, RowMapper rse)	is used to fetch records using RowMapper.

Example of Spring JdbcTemplate

We are assuming that you have created the following table inside the Oracle10g database.

```
1. create table employee(
2. id number(10),
3. name varchar2(100),
4. salary number(10)
5. );
```

Employee.java

This class contains 3 properties with constructors and setter and getters.

```
1. package com.smgc;
2.
3. public class Employee {
4. private int id;
```

```
5. private String name;
6. private float salary;
7. //no-arg and parameterized constructors
8. //getters and setters
9. }
```

EmployeeDao.java

It contains one property jdbcTemplate and three methods saveEmployee(), updateEmployee and deleteEmployee().

```
1. package com.smgc;
2. import org.springframework.jdbc.core.JdbcTemplate;
3.
4. public class EmployeeDao {
5. private JdbcTemplate jdbcTemplate;
6.
7. public void setJdbcTemplate(JdbcTemplate jdbcTemplate) {
8.     this.jdbcTemplate = jdbcTemplate;
9. }
10.
11. public int saveEmployee(Employee e){
12.     String query="insert into employee values(
13.     '"+e.getId()+"','"+e.getName()+"','"+e.getSalary()+"')";
14.     return jdbcTemplate.update(query);
15. }
16. public int updateEmployee(Employee e){
17.     String query="update employee set
18.     name='"+e.getName()+"',salary='"+e.getSalary()+"' where id='"+e.getId()+"' ";
19.     return jdbcTemplate.update(query);
20. }
21. public int deleteEmployee(Employee e){
22.     String query="delete from employee where id='"+e.getId()+"' ";
23.     return jdbcTemplate.update(query);
24. }
25.
26. }
```

applicationContext.xml

The **DriverManagerDataSource** is used to contain the information about the database such as driver class name, connnection URL, username and password.

There are a property named **datasource** in the JdbcTemplate class of DriverManagerDataSource type. So, we need to provide the reference of DriverManagerDataSource object in the JdbcTemplate class for the datasource property.

Here, we are using the JdbcTemplate object in the EmployeeDao class, so we are passing it by the setter method but you can use constructor also.

```
1. <?xml version="1.0" encoding="UTF-8"?>
```

```
<beans
    xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:p="http://www.springframework.org/schema/p"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

<bean id="ds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
<property name="driverClassName" value="oracle.jdbc.driver.OracleDriver" />
<property name="url" value="jdbc:oracle:thin:@localhost:1521:xe" />
<property name="username" value="system" />
<property name="password" value="oracle" />
</bean>

<bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
<property name="dataSource" ref="ds"></property>
</bean>

<bean id="edao" class="com.smgc.EmployeeDao">
<property name="jdbcTemplate" ref="jdbcTemplate"></property>
</bean>

</beans>
```

Test.java

This class gets the bean from the applicationContext.xml file and calls the saveEmployee() method. You can also call updateEmployee() and deleteEmployee() method by uncommenting the code as well.

```
package com.smgc;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class Test {

public static void main(String[] args) {
    ApplicationContext ctx=new ClassPathXmlApplicationContext("applicationContext.xml");

    EmployeeDao dao=(EmployeeDao)ctx.getBean("edao");
    int status=dao.saveEmployee(new Employee(102,"Amit",35000));
    System.out.println(status);

    /*int status=dao.updateEmployee(new Employee(102,"Piyush",15000));
    System.out.println(status);
    */

    /*Employee e=new Employee();
    e.setId(102);
    int status=dao.deleteEmployee(e);
    System.out.println(status);*/

}
}
```

☺ PreparedStatements

Example of PreparedStatement in Spring JdbcTemplate

1. PreparedStatement in Spring JDBC Template
2. PreparedStatementCallback interface
3. Example of using PreparedStatement in Spring

We can execute parameterized query using Spring JdbcTemplate by the help of **execute()** method of JdbcTemplate class. To use parameterized query, we pass the instance of **PreparedStatementCallback** in the execute method.

Syntax of execute method to use parameterized query

```
1. public T execute(String sql,PreparedStatementCallback<T>);
```

PreparedStatementCallback interface

It processes the input parameters and output results. In such case, you don't need to care about single and double quotes.

Method of PreparedStatementCallback interface

It has only one method doInPreparedStatement. Syntax of the method is given below:

```
1. public T doInPreparedStatement(PreparedStatement ps)throws SQLException, DataAccessExc
   eption
```

Example of using PreparedStatement in Spring

We are assuming that you have created the following table inside the Oracle10g database.

```
1. create table employee(
2. id number(10),
3. name varchar2(100),
4. salary number(10)
5. );
```

Employee.java

This class contains 3 properties with constructors and setter and getters.

```
1. package com.smgc;
2.
3. public class Employee {
4. private int id;
5. private String name;
6. private float salary;
```

```
7. //no-arg and parameterized constructors
8. //getters and setters
9. }
```

EmployeeDao.java

It contains one property jdbcTemplate and one method saveEmployeeByPreparedStatement. You must understand the concept of annonymous class to understand the code of the method.

```
1. package com.smgc;
2. import java.sql.PreparedStatement;
3. import java.sql.SQLException;
4.
5. import org.springframework.dao.DataAccessException;
6. import org.springframework.jdbc.core.JdbcTemplate;
7. import org.springframework.jdbc.core.PreparedStatementCallback;
8.
9. public class EmployeeDao {
10. private JdbcTemplate jdbcTemplate;
11.
12. public void setJdbcTemplate(JdbcTemplate jdbcTemplate) {
13.     this.jdbcTemplate = jdbcTemplate;
14. }
15.
16. public Boolean saveEmployeeByPreparedStatement(final Employee e){
17.     String query="insert into employee values(?,?,?)";
18.     return jdbcTemplate.execute(query,new PreparedStatementCallback<Boolean>(){
19.     @Override
20.     public Boolean doInPreparedStatement(PreparedStatement ps)
21.             throws SQLException, DataAccessException {
22.
23.         ps.setInt(1,e.getId());
24.         ps.setString(2,e.getName());
25.         ps.setFloat(3,e.getSalary());
26.
27.         return ps.execute();
28.
29.     }
30.     });
31. }
32.
33.
34. }
```

applicationContext.xml

The **DriverManagerDataSource** is used to contain the information about the database such as driver class name, connnection URL, username and password.

There are a property named **datasource** in the JdbcTemplate class of DriverManagerDataSource type. So, we need to provide the reference of DriverManagerDataSource object in the JdbcTemplate class for the datasource property.

Here, we are using the JdbcTemplate object in the EmployeeDao class, so we are passing it by the setter method but you can use constructor also.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
    xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:p="http://www.springframework.org/schema/p"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

<bean id="ds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
<property name="driverClassName" value="oracle.jdbc.driver.OracleDriver" />
<property name="url" value="jdbc:oracle:thin:@localhost:1521:xe" />
<property name="username" value="system" />
<property name="password" value="oracle" />
</bean>

<bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
<property name="dataSource" ref="ds"></property>
</bean>

<bean id="edao" class="com.smgc.EmployeeDao">
<property name="jdbcTemplate" ref="jdbcTemplate"></property>
</bean>


</beans>
```

Test.java

This class gets the bean from the applicationContext.xml file and calls the saveEmployeeByPreparedStatement() method.

```
package com.smgc;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class Test {

public static void main(String[] args) {
    ApplicationContext ctx=new ClassPathXmlApplicationContext("applicationContext.xml");

    EmployeeDao dao=(EmployeeDao)ctx.getBean("edao");
    dao.saveEmployeeByPreparedStatement(new Employee(108,"Amit",35000));
}
}
```

☺ ResultSetExtractor

ResultSetExtractor Example | Fetching Records by Spring JdbcTemplate

We can easily fetch the records from the database using **query()** method of **JdbcTemplate** class where we need to pass the instance of ResultSetExtractor.

Syntax of query method using ResultSetExtractor

```
1. public T query(String sql,ResultSetExtractor<T> rse)
```

ResultSetExtractor Interface

ResultSetExtractor interface can be used to fetch records from the database. It accepts a ResultSet and returns the list.

Method of ResultSetExtractor interface

It defines only one method extractData that accepts ResultSet instance as a parameter. Syntax of the method is given below:

```
1. public T extractData(ResultSet rs)throws SQLException,DataAccessException
```

Example of ResultSetExtractor Interface to show all the records of the table

We are assuming that you have created the following table inside the Oracle10g database.

```
1. create table employee(
2. id number(10),
3. name varchar2(100),
4. salary number(10)
5. );
```

Employee.java

This class contains 3 properties with constructors and setter and getters. It defines one extra method toString().

```
1. package com.smgc;
2.
3. public class Employee {
4. private int id;
```

```
5.  private String name;
6.  private float salary;
7.  //no-arg and parameterized constructors
8.  //getters and setters
9.
10. public String toString(){
11.     return id+" "+name+" "+salary;
12. }
13. }
```

EmployeeDao.java

It contains on property jdbcTemplate and one method getAllEmployees.

```
1.  package com.smgc;
2.  import java.sql.ResultSet;
3.  import java.sql.SQLException;
4.  import java.util.ArrayList;
5.  import java.util.List;
6.  import org.springframework.dao.DataAccessException;
7.  import org.springframework.jdbc.core.JdbcTemplate;
8.  import org.springframework.jdbc.core.ResultSetExtractor;
9.
10. public class EmployeeDao {
11. private JdbcTemplate template;
12.
13. public void setTemplate(JdbcTemplate template) {
14.     this.template = template;
15. }
16.
17. public List<Employee> getAllEmployees(){
18.  return template.query("select * from employee",new ResultSetExtractor<List<Employee>>(){
19.     @Override
20.      public List<Employee> extractData(ResultSet rs) throws SQLException,
21.             DataAccessException {
22.         List<Employee> list=new ArrayList<Employee>();
23.         while(rs.next()){
24.         Employee e=new Employee();
25.         e.setId(rs.getInt(1));
26.         e.setName(rs.getString(2));
27.         e.setSalary(rs.getInt(3));
28.         list.add(e);
29.         }
30.         return list;
31.         }
32.     });
33.   }
34. }
```

applicationContext.xml

The **DriverManagerDataSource** is used to contain the information about the database such as driver class name, connnection URL, username and password.

There are a property named **datasource** in the JdbcTemplate class of DriverManagerDataSource type. So, we need to provide the reference of DriverManagerDataSource object in the JdbcTemplate class for the datasource property.

Here, we are using the JdbcTemplate object in the EmployeeDao class, so we are passing it by the setter method but you can use constructor also.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
    xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:p="http://www.springframework.org/schema/p"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
 http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

<bean id="ds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
<property name="driverClassName" value="oracle.jdbc.driver.OracleDriver" />
<property name="url" value="jdbc:oracle:thin:@localhost:1521:xe" />
<property name="username" value="system" />
<property name="password" value="oracle" />
</bean>

<bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
<property name="dataSource" ref="ds"></property>
</bean>

<bean id="edao" class="com.smgc.EmployeeDao">
<property name="jdbcTemplate" ref="jdbcTemplate"></property>
</bean>

</beans>
```

Test.java

This class gets the bean from the applicationContext.xml file and calls the getAllEmployees() method of EmployeeDao class.

```
package com.smgc;
import java.util.List;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class Test {
public static void main(String[] args) {
    ApplicationContext ctx=new ClassPathXmlApplicationContext("applicationContext.xml");
    EmployeeDao dao=(EmployeeDao)ctx.getBean("edao");
    List<Employee> list=dao.getAllEmployees();
    for(Employee e:list)
        System.out.println(e);
    }
}
```

☺ RowMapper

RowMapper Example | Fetching records by Spring JdbcTemplate

1. RowMapper
2. RowMapper Interface
3. Method of RowMapper Interface
4. Example of RowMapper Interface

Like ResultSetExtractor, we can use RowMapper interface to fetch the records from the database using **query()** method of **JdbcTemplate** class. In the execute of we need to pass the instance of RowMapper now.

Syntax of query method using RowMapper

```
1. public T query(String sql,RowMapper<T> rm)
```

RowMapper Interface

RowMapper interface allows to map a row of the relations with the instance of user-defined class. It iterates the ResultSet internally and adds it into the collection. So we don't need to write a lot of code to fetch the records as ResultSetExtractor.

Advantage of RowMapper over ResultSetExtractor

RowMapper saves a lot of code becuase it internally adds the data of ResultSet into the collection.

Method of RowMapper interface

It defines only one method mapRow that accepts ResultSet instance and int as the parameter list. Syntax of the method is given below:

```
1. public T mapRow(ResultSet rs, int rowNumber)throws SQLException
```

Example of RowMapper Interface to show all the records of the table

We are assuming that you have created the following table inside the Oracle10g database.

```
1. create table employee(
2. id number(10),
3. name varchar2(100),
4. salary number(10)
5. );
```

Employee.java

This class contains 3 properties with constructors and setter and getters and one extra method toString().

```
1. package com.smgc;
2.
3. public class Employee {
4. private int id;
5. private String name;
6. private float salary;
7. //no-arg and parameterized constructors
8. //getters and setters
9. public String toString(){
10.     return id+" "+name+" "+salary;
11. }
12. }
```

EmployeeDao.java

It contains on property jdbcTemplate and one method getAllEmployeesRowMapper.

```
1. package com.smgc;
2. import java.sql.ResultSet;
3. import java.sql.SQLException;
4. import java.util.ArrayList;
5. import java.util.List;
6. import org.springframework.dao.DataAccessException;
7. import org.springframework.jdbc.core.JdbcTemplate;
8. import org.springframework.jdbc.core.ResultSetExtractor;
9. import org.springframework.jdbc.core.RowMapper;
10.
11. public class EmployeeDao {
12. private JdbcTemplate template;
13.
14. public void setTemplate(JdbcTemplate template) {
15.     this.template = template;
16. }
17.
18. public List<Employee> getAllEmployeesRowMapper(){
19.  return template.query("select * from employee",new RowMapper<Employee>(){
20.     @Override
21.     public Employee mapRow(ResultSet rs, int rownumber) throws SQLException {
22.         Employee e=new Employee();
23.         e.setId(rs.getInt(1));
24.         e.setName(rs.getString(2));
25.         e.setSalary(rs.getInt(3));
26.         return e;
27.     }
28.     });
29. }
30. }
```

applicationContext.xml

The **DriverManagerDataSource** is used to contain the information about the database such as driver class name, connnection URL, username and password.

There are a property named **datasource** in the JdbcTemplate class of DriverManagerDataSource type. So, we need to provide the reference of DriverManagerDataSource object in the JdbcTemplate class for the datasource property.

Here, we are using the JdbcTemplate object in the EmployeeDao class, so we are passing it by the setter method but you can use constructor also.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
    xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:p="http://www.springframework.org/schema/p"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

<bean id="ds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
<property name="driverClassName" value="oracle.jdbc.driver.OracleDriver" />
<property name="url" value="jdbc:oracle:thin:@localhost:1521:xe" />
<property name="username" value="system" />
<property name="password" value="oracle" />
</bean>

<bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
<property name="dataSource" ref="ds"></property>
</bean>

<bean id="edao" class="com.smgc.EmployeeDao">
<property name="jdbcTemplate" ref="jdbcTemplate"></property>
</bean>

</beans>
```

Test.java

This class gets the bean from the applicationContext.xml file and calls the getAllEmployeesRowMapper() method of EmployeeDao class.

```
package com.smgc;

import java.util.List;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class Test {
public static void main(String[] args) {
    ApplicationContext ctx=new ClassPathXmlApplicationContext("applicationContext.xml");
    EmployeeDao dao=(EmployeeDao)ctx.getBean("edao");
    List<Employee> list=dao.getAllEmployeesRowMapper();
```

```
12.
13.     for(Employee e:list)
14.         System.out.println(e);
15. }
16. }
```

☺ NamedParameter

Spring NamedParameterJdbcTemplate Example

1. NamedParameterJdbcTemplate
2. Method of NamedParameterJdbcTemplate
3. Spring NamedParameterJdbcTemplate Example

Spring provides another way to insert data by named parameter. In such way, we use names instead of ?(question mark). So it is better to remember the data for the column.

Simple example of named parameter query

```
1. insert into employee values (:id,:name,:salary)
```

Method of NamedParameterJdbcTemplate class

In this example,we are going to call only the execute method of NamedParameterJdbcTemplate class. Syntax of the method is as follows:

```
1. pubic T execute(String sql,Map map,PreparedStatementCallback psc)
```

Example of NamedParameterJdbcTemplate class

We are assuming that you have created the following table inside the Oracle10g database.

```
1. create table employee(
2. id number(10),
3. name varchar2(100),
4. salary number(10)
5. );
```

Employee.java

This class contains 3 properties with constructors and setter and getters.

```
1. package com.smgc;
2.
3. public class Employee {
4. private int id;
5. private String name;
6. private float salary;
7. //no-arg and parameterized constructors
8. //getters and setters
9. }
```

EmployeeDao.java

It contains on property jdbcTemplate and one method save.

```
1.  package com.smgc;
2.
3.  import java.sql.PreparedStatement;
4.  import java.sql.SQLException;
5.  import org.springframework.dao.DataAccessException;
6.  import org.springframework.jdbc.core.PreparedStatementCallback;
7.  import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;
8.  import java.util.*;
9.
10. public class EmpDao {
11. NamedParameterJdbcTemplate template;
12.
13. public EmpDao(NamedParameterJdbcTemplate template) {
14.         this.template = template;
15. }
16. public  void save (Emp e){
17. String query="insert into employee values (:id,:name,:salary)";
18.
19. Map<String,Object> map=new HashMap<String,Object>();
20. map.put("id",e.getId());
21. map.put("name",e.getName());
22. map.put("salary",e.getSalary());
23.
24. template.execute(query,map,new PreparedStatementCallback() {
25.     @Override
26.     public Object doInPreparedStatement(PreparedStatement ps)
27.             throws SQLException, DataAccessException {
28.         return ps.executeUpdate();
29.     }
30. });
31. }
32. }
```

applicationContext.xml

The **DriverManagerDataSource** is used to contain the information about the database such as driver class name, connnection URL, username and password.

There are a property named **datasource** in the NamedParameterJdbcTemplate class of DriverManagerDataSource type. So, we need to provide the reference of DriverManagerDataSource object in the NamedParameterJdbcTemplate class for the datasource property.

Here, we are using the NamedParameterJdbcTemplate object in the EmployeeDao class, so we are passing it by the constructor but you can use setter method also.

```
1.  <?xml version="1.0" encoding="UTF-8"?>
2.  <beans
```

```
3.      xmlns="http://www.springframework.org/schema/beans"
4.      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.      xmlns:p="http://www.springframework.org/schema/p"
6.      xsi:schemaLocation="http://www.springframework.org/schema/beans
7.   http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
8.
9.   <bean id="ds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
10. <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver" />
11. <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe" />
12. <property name="username" value="system" />
13. <property name="password" value="oracle" />
14. </bean>
15.
16. <bean id="jtemplate"
17.  class="org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate">
18. <constructor-arg ref="ds"></constructor-arg>
19. </bean>
20.
21. <bean id="edao" class="com.smgc.EmpDao">
22. <constructor-arg>
23. <ref bean="jtemplate"/>
24. </constructor-arg>
25. </bean>
26.
27. </beans>
```

SimpleTest.java

This class gets the bean from the applicationContext.xml file and calls the save method.

```
1.   package com.smgc;
2.
3.   import org.springframework.beans.factory.BeanFactory;
4.   import org.springframework.beans.factory.xml.XmlBeanFactory;
5.   import org.springframework.core.io.ClassPathResource;
6.   import org.springframework.core.io.Resource;
7.
8.   public class SimpleTest {
9.     public static void main(String[] args) {
10.
11.     Resource r=new ClassPathResource("applicationContext.xml");
12.     BeanFactory factory=new XmlBeanFactory(r);
13.
14.     EmpDao dao=(EmpDao)factory.getBean("edao");
15.     dao.save(new Emp(23,"Piyush",50000));
16.
17.   }
18. }
```

☺ SimpleJdbcTemplate

Spring SimpleJdbcTemplate Example

1. Spring SimpleJdbcTemplate
2. Example of SimpleJdbcTemplate

Spring 3 JDBC supports the java 5 feature var-args (variable argument) and autoboxing by the help of SimpleJdbcTemplate class.

SimpleJdbcTemplate class wraps the JdbcTemplate class and provides the update method where we can pass arbitrary number of arguments.

Syntax of update method of SimpleJdbcTemplate class

```
1. int update(String sql,Object... parameters)
```

We should pass the parameter values in the update method in the order they are defined in the parameterized query.

Example of SimpleJdbcTemplate class

We are assuming that you have created the following table inside the Oracle10g database.

```
1. create table employee(
2. id number(10),
3. name varchar2(100),
4. salary number(10)
5. );
```

Employee.java

This class contains 3 properties with constructors and setter and getters.

```
1. package com.smgc;
2.
3. public class Employee {
4. private int id;
5. private String name;
6. private float salary;
7. //no-arg and parameterized constructors
8. //getters and setters
9. }
```

EmployeeDao.java

It contains one property SimpleJdbcTemplate and one method update. In such case, update method will update only name for the corresponding id. If you want to update the name and salary both, comment the above two lines of code of the update method and uncomment the 2 lines of code given below.

```
1.  package com.smgc;
2.
3.  import org.springframework.jdbc.core.simple.SimpleJdbcTemplate;
4.  public class EmpDao {
5.  SimpleJdbcTemplate template;
6.
7.  public EmpDao(SimpleJdbcTemplate template) {
8.          this.template = template;
9.  }
10. public int update (Emp e){
11. String query="update employee set name=? where id=?";
12. return template.update(query,e.getName(),e.getId());
13.
14. //String query="update employee set name=?,salary=? where id=?";
15. //return template.update(query,e.getName(),e.getSalary(),e.getId());
16. }
17.
18. }
```

applicationContext.xml

The **DriverManagerDataSource** is used to contain the information about the database such as driver class name, connnection URL, username and password.

There are a property named **datasource** in the SimpleJdbcTemplate class of DriverManagerDataSource type. So, we need to provide the reference of DriverManagerDataSource object in the SimpleJdbcTemplate class for the datasource property.

Here, we are using the SimpleJdbcTemplate object in the EmployeeDao class, so we are passing it by the constructor but you can use setter method also.

```
1.  <?xml version="1.0" encoding="UTF-8"?>
2.  <beans
3.     xmlns="http://www.springframework.org/schema/beans"
4.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.     xmlns:p="http://www.springframework.org/schema/p"
6.     xsi:schemaLocation="http://www.springframework.org/schema/beans
7.  http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
8.
9.  <bean id="ds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
10. <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver" />
11. <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe" />
12. <property name="username" value="system" />
13. <property name="password" value="oracle" />
14. </bean>
```

```
15.
16. <bean id="jtemplate" class="org.springframework.jdbc.core.simple.SimpleJdbcTemplate">
17. <constructor-arg ref="ds"></constructor-arg>
18. </bean>
19.
20. <bean id="edao" class="com.smgc.EmpDao">
21. <constructor-arg>
22. <ref bean="jtemplate"/>
23. </constructor-arg>
24. </bean>
25.
26. </beans>
```

SimpleTest.java

This class gets the bean from the applicationContext.xml file and calls the update method of EmpDao class.

```
1.  package com.smgc;
2.
3.  import org.springframework.beans.factory.BeanFactory;
4.  import org.springframework.beans.factory.xml.XmlBeanFactory;
5.  import org.springframework.core.io.ClassPathResource;
6.  import org.springframework.core.io.Resource;
7.
8.  public class SimpleTest {
9.  public static void main(String[] args) {
10.
11.    Resource r=new ClassPathResource("applicationContext.xml");
12.    BeanFactory factory=new XmlBeanFactory(r);
13.
14.    EmpDao dao=(EmpDao)factory.getBean("edao");
15.    int status=dao.update(new Emp(23,"Tarun",35000));
16.    System.out.println(status);
17. }
18. }
```

Chapter-2.2

Spring with ORM And SpEL

☺ Spring with Hibernate

Hibernate and Spring Integration

We can simply integrate **hibernate application with spring application**.

In hibernate framework, we provide all the database information hibernate.cfg.xml file.

But if we are going to integrate the hibernate application with spring, we don't need to create the hibernate.cfg.xml file. We can provide all the information in the applicationContext.xml file.

Advantage of Spring framework with hibernate

The Spring framework provides **HibernateTemplate** class, so you don't need to follow so many steps like create Configuration, BuildSessionFactory, Session, beginning and committing transaction etc.

So **it saves a lot of code**.

Understanding problem without using spring:

Let's understand it by the code of hibernate given below:

```
1. //creating configuration
2. Configuration cfg=new Configuration();
3. cfg.configure("hibernate.cfg.xml");
4.
5. //creating seession factory object
6. SessionFactory factory=cfg.buildSessionFactory();
7.
8. //creating session object
9. Session session=factory.openSession();
10.
11. //creating transaction object
12. Transaction t=session.beginTransaction();
13.
14. Employee e1=new Employee(111,"arun",40000);
15. session.persist(e1);//persisting the object
16.
17. t.commit();//transaction is commited
18. session.close();
```

As you can see in the code of sole hibernate, you have to follow so many steps.

Solution by using HibernateTemplate class of Spring Framework:

Now, you don't need to follow so many steps. You can simply write this:

```
1. Employee e1=new Employee(111,"arun",40000);
2. hibernateTemplate.save(e1);
```

Methods of HibernateTemplate class

Let's see a list of commonly used methods of HibernateTemplate class.

No.	Method	Description
1)	void persist(Object entity)	persists the given object.
2)	Serializable save(Object entity)	persists the given object and returns id.
3)	void saveOrUpdate(Object entity)	persists or updates the given object. If id is found, it updates the record otherwise saves the record.
4)	void update(Object entity)	updates the given object.
5)	void delete(Object entity)	deletes the given object on the basis of id.
6)	Object get(Class entityClass, Serializable id)	returns the persistent object on the basis of given id.
7)	Object load(Class entityClass, Serializable id)	returns the persistent object on the basis of given id.
8)	List loadAll(Class entityClass)	returns the all the persistent objects.

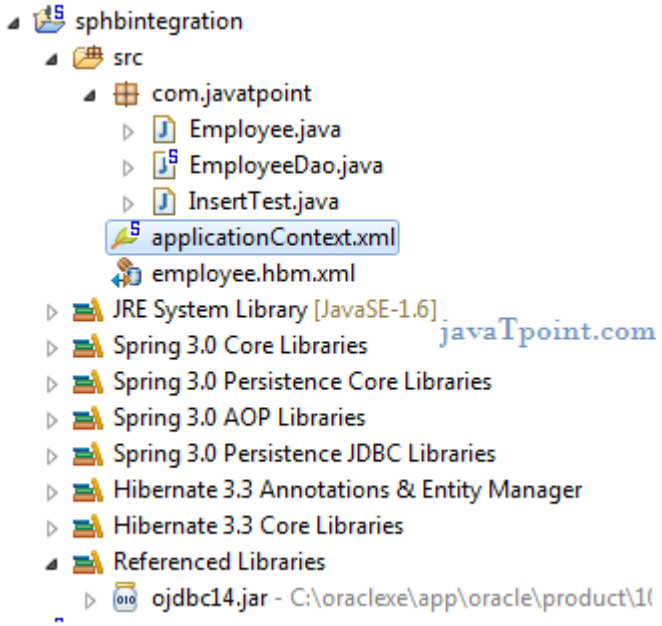
Steps

Let's see what are the simple steps for hibernate and spring integration:

1. **create table in the database** It is optional.
2. **create applicationContext.xml file** It contains information of DataSource, SessionFactory etc.
3. **create Employee.java file** It is the persistent class
4. **create employee.hbm.xml file** It is the mapping file.
5. **create EmployeeDao.java file** It is the dao class that uses HibernateTemplate.
6. **create InsertTest.java file** It calls methods of EmployeeDao class.

Example of Hibernate and spring integration

In this example, we are going to integrate the hibernate application with spring. Let's see the **directory structure** of spring and hibernate example.



---

1) create the table in the database

In this example, we are using the Oracle as the database, but you may use any database. Let's create the table in the oracle database

```
1. CREATE TABLE  "EMP558"
2.    (    "ID" NUMBER(10,0) NOT NULL ENABLE,
3.     "NAME" VARCHAR2(255 CHAR),
4.     "SALARY" FLOAT(126),
5.      PRIMARY KEY ("ID") ENABLE
6.    )
7. /
```

---

2) Employee.java

It is a simple POJO class. Here it works as the persistent class for hibernate.

```
1. package com.smgc;
2. public class Employee {
3. private int id;
4. private String name;
5. private float salary;
6. //getters and setters
7. }
```

3) employee.hbm.xml

This mapping file contains all the information of the persistent class.

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
<class name="com.smgc.Employee" table="emp558">
        <id name="id">
        <generator class="assigned"></generator>
        </id>

        <property name="name"></property>
        <property name="salary"></property>
</class>

</hibernate-mapping>
```

4) EmployeeDao.java

It is a java class that uses the **HibernateTemplate** class method to persist the object of Employee class.

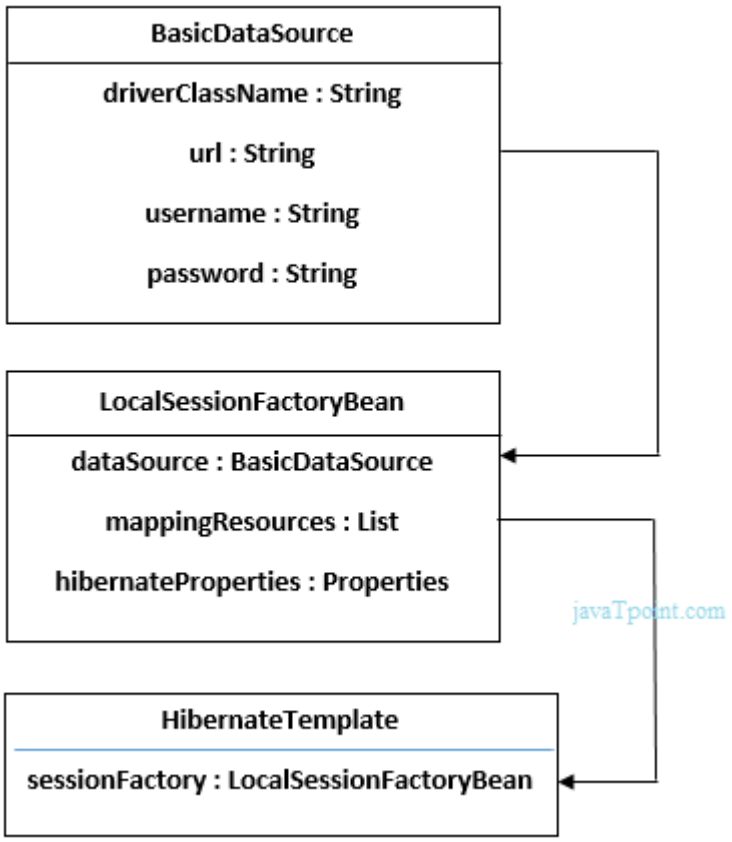
```
package com.smgc;
import org.springframework.orm.hibernate3.HibernateTemplate;
import java.util.*;
public class EmployeeDao {
HibernateTemplate template;
public void setTemplate(HibernateTemplate template) {
    this.template = template;
}
//method to save employee
public void saveEmployee(Employee e){
    template.save(e);
}
//method to update employee
public void updateEmployee(Employee e){
    template.update(e);
}
//method to delete employee
public void deleteEmployee(Employee e){
    template.delete(e);
}
//method to return one employee of given id
public Employee getById(int id){
    Employee e=(Employee)template.get(Employee.class,id);
    return e;
```

```
25. }
26. //method to return all employees
27. public List<Employee> getEmployees(){
28.     List<Employee> list=new ArrayList<Employee>();
29.     list=template.loadAll(Employee.class);
30.     return list;
31. }
32. }
```

---

5) applicationContext.xml

In this file, we are providing all the informations of the database in the **BasicDataSource** object. This object is used in the **LocalSessionFactoryBean** class object, containing some other informations such as mappingResources and hibernateProperties. The object of **LocalSessionFactoryBean** class is used in the HibernateTemplate class. Let's see the code of applicationContext.xml file.



File: applicationContext.xml

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans
3.     xmlns="http://www.springframework.org/schema/beans"
4.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.     xmlns:p="http://www.springframework.org/schema/p"
6.     xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
7.          http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
8.
9.
10.    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
11.       <property name="driverClassName"  value="oracle.jdbc.driver.OracleDriver"></property>
12.       <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"></property>
13.       <property name="username" value="system"></property>
14.       <property name="password" value="oracle"></property>
15.    </bean>
16.
17.    <bean id="mysessionFactory"  class="org.springframework.orm.hibernate3.LocalSessionFactoryBean"
    >
18.       <property name="dataSource" ref="dataSource"></property>
19.
20.       <property name="mappingResources">
21.       <list>
22.       <value>employee.hbm.xml</value>
23.       </list>
24.       </property>
25.
26.       <property name="hibernateProperties">
27.          <props>
28.             <prop key="hibernate.dialect">org.hibernate.dialect.Oracle9Dialect</prop>
29.             <prop key="hibernate.hbm2ddl.auto">update</prop>
30.             <prop key="hibernate.show_sql">true</prop>
31.
32.          </props>
33.       </property>
34.    </bean>
35.
36.    <bean id="template" class="org.springframework.orm.hibernate3.HibernateTemplate">
37.    <property name="sessionFactory" ref="mysessionFactory"></property>
38.    </bean>
39.
40.    <bean id="d" class="com.smgc.EmployeeDao">
41.    <property name="template" ref="template"></property>
42.    </bean>
43.
44.
45.    </beans>
```

---

6) InsertTest.java

This class uses the EmployeeDao class object and calls its saveEmployee method by passing the object of Employee class.

```
1.  package com.smgc;
2.
3.  import org.springframework.beans.factory.BeanFactory;
4.  import org.springframework.beans.factory.xml.XmlBeanFactory;
```

```
5.  import org.springframework.core.io.ClassPathResource;
6.  import org.springframework.core.io.Resource;
7.
8.  public class InsertTest {
9.  public static void main(String[] args) {
10.
11.    Resource r=new ClassPathResource("applicationContext.xml");
12.    BeanFactory factory=new XmlBeanFactory(r);
13.
14.    EmployeeDao dao=(EmployeeDao)factory.getBean("d");
15.
16.    Employee e=new Employee();
17.    e.setId(114);
18.    e.setName("varun");
19.    e.setSalary(50000);
20.
21.    dao.saveEmployee(e);
22.
23. }
24. }
```

Now, if you see the table in the oracle database, record is inserted successfully.

Enabling automatic table creation, showing sql queries etc.

You can enable many hibernate properties like automatic table creation by hbm2ddl.auto etc. in applicationContext.xml file. Let's see the code:

```
1.  <property name="hibernateProperties">
2.          <props>
3.             <prop key="hibernate.dialect">org.hibernate.dialect.Oracle9Dialect</prop>
4.             <prop key="hibernate.hbm2ddl.auto">update</prop>
5.             <prop key="hibernate.show_sql">true</prop>
6.
7.          </props>
```

If you write this code, you don't need to create table because table will be created automatically.

☺ Spring with JPA

Spring Data JPA Tutorial

Spring Data JPA API provides JpaTemplate class to integrate spring application with JPA.

JPA (Java Persistent API) is the sun specification for persisting objects in the enterprise application. It is currently used as the replacement for complex entity beans.

The implementation of JPA specification are provided by many vendors such as:

- Hibernate
- Toplink
- iBatis
- OpenJPA etc.

Advantage of Spring JpaTemplate

You don't need to write the before and after code for persisting, updating, deleting or searching object such as creating Persistence instance, creating EntityManagerFactory instance, creating EntityTransaction instance, creating EntityManager instance, commiting EntityTransaction instance and closing EntityManager.

So, it **save a lot of code**.

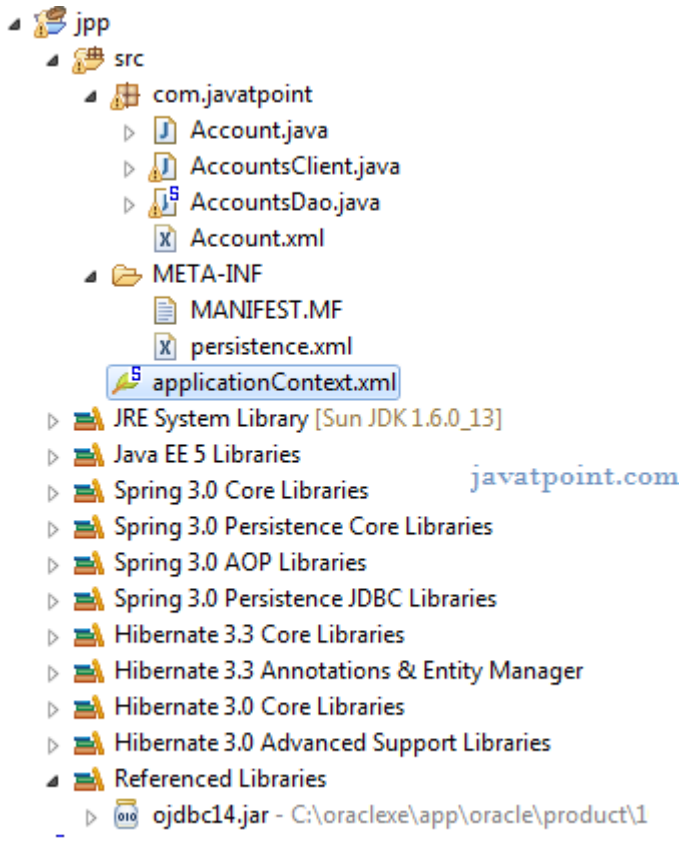
In this example, we are going to use hibernate for the implementation of JPA.

Example of Spring and JPA Integration

Let's see the simple steps to integration spring application with JPA:

1. **create Account.java file**
2. **create Account.xml file**
3. **create AccountDao.java file**
4. **create persistence.xml file**
5. **create applicationContext.xml file**
6. **create AccountsClient.java file**

In this example, we are going to integrate the hibernate application with spring. Let's see the **directory structure** of jpa example with spring.



---

1) Account.java

It is a simple POJO class.

```
1. package com.smgc;
2. 
3. public class Account {
4.     private int accountNumber;
5.     private String owner;
6.     private double balance;
7.         //no-arg and parameterized constructor
8.         //getters and setters
9. }
```

---

2) Account.xml

This mapping file contains all the information of the persistent class.

```
1. <entity-mappings version="1.0"
2.         xmlns="http://java.sun.com/xml/ns/persistence/orm"
3.         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.         xsi:schemaLocation="http://java.sun.com/xml/ns/persistence/orm
5.         http://java.sun.com/xml/ns/persistence/orm_1_0.xsd ">
6. 
```

```
7.    <entity class="com.smgc.Account">
8.       <table name="account100"></table>
9.       <attributes>
10.         <id name="accountNumber">
11.            <column name="accountnumber"/>
12.         </id>
13.         <basic name="owner">
14.            <column name="owner"/>
15.         </basic>
16.         <basic name="balance">
17.            <column name="balance"/>
18.         </basic>
19.      </attributes>
20.   </entity>
21. </entity-mappings>
```

---

3) AccountDao.java

```
1.  package com.smgc;
2.  import java.util.List;
3.  import org.springframework.orm.jpa.JpaTemplate;
4.  import org.springframework.transaction.annotation.Transactional;
5.  @Transactional
6.  public class AccountsDao{
7.     JpaTemplate template;
8.     public void setTemplate(JpaTemplate template) {
9.        this.template = template;
10.    }
11.    public void createAccount(int accountNumber,String owner,double balance){
12.       Account account = new Account(accountNumber,owner,balance);
13.       template.persist(account);
14.    }
15.    public void updateBalance(int accountNumber,double newBalance){
16.       Account account = template.find(Account.class, accountNumber);
17.       if(account != null){
18.          account.setBalance(newBalance);
19.       }
20.       template.merge(account);
21.    }
22.    public void deleteAccount(int accountNumber){
23.       Account account = template.find(Account.class, accountNumber);
24.       if(account != null)
25.          template.remove(account);
26.    }
27.    public List<Account> getAllAccounts(){
28.       List<Account> accounts =template.find("select acc from Account acc");
29.       return accounts;
30.    }
31. }
```

4) persistence.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="http://java.sun.com/xml/ns/persistence"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
   http://java.sun.com/xml/ns/persistence/persistence_1_0.xsd" version="1.0">

   <persistence-unit name="ForAccountsDB">
      <mapping-file>com/javatpoint/Account.xml</mapping-file>
      <class>com.javatpoint.Account</class>
   </persistence-unit>
</persistence>
```

5) applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xmlns:tx="http://www.springframework.org/schema/tx"
   xsi:schemaLocation="http://www.springframework.org/schema/beans
   http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
   http://www.springframework.org/schema/tx
   http://www.springframework.org/schema/tx/spring-tx-3.0.xsd">

    <tx:annotation-driven transaction-manager="jpaTxnManagerBean" proxy-target-
   class="true"/>

    <bean id="dataSourceBean" class="org.springframework.jdbc.datasource.DriverManagerDat
   aSource">
      <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"></property>
      <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"></property>
      <property name="username" value="system"></property>
      <property name="password" value="oracle"></property>
   </bean>

    <bean id="hbAdapterBean" class="org.springframework.orm.jpa.vendor.HibernateJpaVendo
   rAdapter">
      <property name="showSql" value="true"></property>
      <property name="generateDdl" value="true"></property>
      <property name="databasePlatform" value="org.hibernate.dialect.OracleDialect"></propert
   y>
    </bean>

   <bean id="emfBean" class="org.springframework.orm.jpa.LocalContainerEntityManagerFact
   oryBean">
      <property name="dataSource" ref="dataSourceBean"></property>
      <property name="jpaVendorAdapter" ref="hbAdapterBean"></property>
```

```
28.     </bean>
29.
30.     <bean id="jpaTemplateBean" class="org.springframework.orm.jpa.JpaTemplate">
31.       <property name="entityManagerFactory" ref="emfBean"></property>
32.     </bean>
33.
34.     <bean id="accountsDaoBean" class="‘com.smgc.AccountsDao">
35.       <property name="template" ref="jpaTemplateBean"></property>
36.     </bean>
37.      <bean id="jpaTxnManagerBean" class="org.springframework.orm.jpa.JpaTransactionManag
    er">
38.       <property name="entityManagerFactory" ref="emfBean"></property>
39.    </bean>
40.
41. </beans>
```

The **generateDdl** property will create the table automatically.

The **showSql** property will show the sql query on console.

---

6) Accountsclient.java

```
1.  package com.smgc;
2.
3.  import java.util.List;
4.  import org.springframework.context.ApplicationContext;
5.  import org.springframework.context.support.ClassPathXmlApplicationContext;
6.  import org.springframework.context.support.FileSystemXmlApplicationContext;
7.
8.  public class AccountsClient{
9.  public static void main(String[] args){
10.  ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml")
    ;
11.  AccountsDao accountsDao = context.getBean("accountsDaoBean",AccountsDao.class);
12.
13.  accountsDao.createAccount(15, "Jai Kumar", 41000);
14.  accountsDao.createAccount(20, "Rishi ", 35000);
15.  System.out.println("Accounts created");
16.
17.  //accountsDao.updateBalance(20, 50000);
18.  //System.out.println("Account balance updated");
19.
20.
21.  /*List<Account> accounts = accountsDao.getAllAccounts();
22.  for (int i = 0; i < accounts.size(); i++) {
23.    Account acc = accounts.get(i);
24.    System.out.println(acc.getAccountNumber() + " : " + acc.getOwner() + " (" + acc.getBalance()
    + ")");
25.  }*/
```

```
26.
27.   //accountsDao.deleteAccount(111);
28.   //System.out.println("Account deleted");
29.
30.  }
31. }
```

Output

```
Hibernate: insert into account100 (balance, owner, accountnumber) values (?, ?, ?)
Hibernate: insert into account100 (balance, owner, accountnumber) values (?, ?, ?)
Accounts created
```

☺ SpEL Examples

Spring Expression Language (SPEL) Tutorial

1. SPEL
2. SPEL API
3. Hello SPEL Example
4. Other Examples of SPEL

SpEL is an exression language supporting the features of querying and manipulating an object graph at runtime.

There are many expression languages available such as JSP EL, OGNL, MVEL and JBoss EL. SpEL provides some additional features such as method invocation and string templating functionality.

SpEL API

The SpEL API provides many interfaces and classes. They are as follows:

- Expression interface
- SpelExpression class
- ExpressionParser interface
- SpelExpressionParser class
- EvaluationContext interface
- StandardEvaluationContext class

Hello SPEL Example

```
1. import org.springframework.expression.Expression;
2. import org.springframework.expression.ExpressionParser;
3. import org.springframework.expression.spel.standard.SpelExpressionParser;
4.
5. public class Test {
6. public static void main(String[] args) {
7. ExpressionParser parser = new SpelExpressionParser();
8.
9. Expression exp = parser.parseExpression("'Hello SPEL'");
10. String message = (String) exp.getValue();
11. System.out.println(message);
12. //OR
13. //System.out.println(parser.parseExpression("'Hello SPEL'").getValue());
14. }
15. }
```

Other SPEL Example

Let's see a lot of useful examples of SPEL. Here, we are assuming all the examples have been written inside the main() method.

Using concat() method with String

```
1. ExpressionParser parser = new SpelExpressionParser();
2. Expression exp = parser.parseExpression("'Welcome SPEL'.concat('!')");
3. String message = (String) exp.getValue();
4. System.out.println(message);
```

Converting String into byte array

```
1. Expression exp = parser.parseExpression("'Hello World'.bytes");
2. byte[] bytes = (byte[]) exp.getValue();
3. for(int i=0;i<bytes.length;i++){
4.     System.out.print(bytes[i]+" ");
5. }
```

Getting length after converting string into bytes

```
1. Expression exp = parser.parseExpression("'Hello World'.bytes.length");
2. int length = (Integer) exp.getValue();
3. System.out.println(length);
```

Converting String contents into uppercase letter

```
1. Expression exp = parser.parseExpression("new String('hello world').toUpperCase()");
2. String message = exp.getValue(String.class);
3. System.out.println(message);
4. //OR
5. System.out.println(parser.parseExpression("'hello world'.toUpperCase()").getValue());
```

☺ Operators In SpEL

Operators in SPEL

1. Examples of Operators in SPEL

We can use many operators in SpEL such as arithmetic, relational, logical etc. There are given a lot of examples of using different operators in SpEL.

Examples of using operators in SPEL

```
1. import org.springframework.expression.ExpressionParser;
2. import org.springframework.expression.spel.standard.SpelExpressionParser;
3.
4.
5. public class Test {
6. public static void main(String[] args) {
7. ExpressionParser parser = new SpelExpressionParser();
8.
9. //arithmetic operator
10. System.out.println(parser.parseExpression("'Welcome SPEL'+'!'").getValue());
11. System.out.println(parser.parseExpression("10 * 10/2").getValue());
12. System.out.println(parser.parseExpression("'Today is: '+ new java.util.Date()").getValue());
13.
14. //logical operator
15. System.out.println(parser.parseExpression("true and true").getValue());
16.
17. //Relational operator
18. System.out.println(parser.parseExpression("'Piyush'.length()==5").getValue());
19. }
20. }
```

☺ Variables In SpEL

Variable in SPEL | StandardEvaluationContext

1. Using Variable in SPEL
2. StandardEvaluationContext class
3. Example of Using variable in SPEL

In SpEL, we can store a value in the variable and use the variable in the method and call the method. To work on variable, we need to use **StandardEvaluationContext** class.

Example of Using variable in SPEL

Calculation.java

```
1. public class Calculation {
2. private int number;
3. public int getNumber() {
4.    return number;
5. }
6. public void setNumber(int number) {
7.    this.number = number;
8. }
9. public int cube(){
10.    return number*number*number;
11. }
12. }
```

Test.java

```
1. import org.springframework.expression.ExpressionParser;
2. import org.springframework.expression.spel.standard.SpelExpressionParser;
3. import org.springframework.expression.spel.support.StandardEvaluationContext;
4.
5. public class Test {
6. public static void main(String[] args) {
7. Calculation calculation=new Calculation();
8. StandardEvaluationContext context=new StandardEvaluationContext(calculation);
9.
10. ExpressionParser parser = new SpelExpressionParser();
11. parser.parseExpression("number").setValue(context,"5");
12.
13. System.out.println(calculation.cube());
14. }
15. }
```

Chapter-2.3

Spring 3 MVC and Remoting with Spring

☺ Spring with RMI

Spring and RMI Integration

1. Spring and RMI Integration
2. Example of Spring and RMI Integration

Spring RMI lets you expose your services through the RMI infrastructure.

Spring provides an easy way to run RMI application by the help of org.springframework.remoting.rmi.**RmiProxyFactoryBean** and org.springframework.remoting.rmi.**RmiServiceExporter** classes.

RmiServiceExporter

It provides the exportation service for the rmi object. This service can be accessed via plain RMI or RmiProxyFactoryBean.

RmiProxyFactoryBean

It is the factory bean for Rmi Proxies. It exposes the proxied service that can be used as a bean reference.

Example of Spring and RMI Integration

Let's see the simple steps to integration spring application with RMI:

1. **Calculation.java**
2. **CalculationImpl.java**
3. **applicationContext.xml**
4. **client-beans.xml**
5. **Host.java**
6. **Client.java**

Required Jar files

To run this example, you need to load:

- **Spring Core jar files**
- **Spring Remoting jar files**
- **Spring AOP jar files**

---

1) Calculation.java

It is the simple interface containing one method cube.

```
1. package com.smgc;
2.
3. public interface Calculation {
4. int cube(int number);
5. }
```

---

2) CalculationImpl.java

This class provides the implementation of Calculation interface.

```
1. package com.smgc;
2.
3. public class CalculationImpl implements Calculation{
4.
5.     @Override
6.     public int cube(int number) {
7.         return number*number*number;
8.     }
9.
10. }
```

---

3) applicationContext.xml

In this xml file we are defining the bean for CalculationImpl class and **RmiServiceExporter** class. We need to provide values for the following properties of RmiServiceExporter class.

1. service
2. serviceInterface
3. serviceName
4. replaceExistingBinding
5. registryPort

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans xmlns="http://www.springframework.org/schema/beans"
3.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
4.      xsi:schemaLocation="http://www.springframework.org/schema/beans
5.  http://www.springframework.org/schema/beans/spring-beans.xsd">
6.
7.  <bean id="calculationBean" class="com.smgc.CalculationImpl"></bean>
8.  <bean class="org.springframework.remoting.rmi.RmiServiceExporter">
9.      <property name="service" ref="calculationBean"></property>
10.     <property name="serviceInterface" value="com.smgc.Calculation"></property>
11.     <property name="serviceName" value="CalculationService"></property>
12.     <property name="replaceExistingBinding" value="true"></property>
13.     <property name="registryPort" value="1099"></property>
14. </bean>
15. </beans>
```

---

4) client-beans.xml

In this xml file, we are defining bean for **RmiProxyFactoryBean**. You need to define two properties of this class.

1. serviceUrl
2. serviceInterface

```
1.  <?xml version="1.0" encoding="UTF-8"?>
2.  <beans xmlns="http://www.springframework.org/schema/beans"
3.      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.      xsi:schemaLocation="http://www.springframework.org/schema/beans
5.  http://www.springframework.org/schema/beans/spring-beans.xsd">
6.
7.  <bean id="calculationBean" class="org.springframework.remoting.rmi.RmiProxyFactoryBean">
8.  <property name="serviceUrl" value="rmi://localhost:1099/CalculationService"></property>
9.  <property name="serviceInterface" value="com.smgc.Calculation"></property>
10. </bean>
11. </beans>
```

5) Host.java

It is simply getting the instance of ApplicationContext. But you need to run this class first to run the example.

```
1.  package com.smgc;
2.  import org.springframework.context.ApplicationContext;
3.  import org.springframework.context.support.ClassPathXmlApplicationContext;
4.
5.  public class Host{
6.  public static void main(String[] args){
7.  ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
8.  System.out.println("Waiting for requests");
9.  }
10. }
```

---

6) Client.java

This class gets the instance of Calculation and calls the method.

```
1. package com.smgc;
2. import org.springframework.context.ApplicationContext;
3. import org.springframework.context.support.ClassPathXmlApplicationContext;
4.
5. public class Client {
6. public static void main(String[] args)  {
7. ApplicationContext context = new ClassPathXmlApplicationContext("client-beans.xml");
8. Calculation calculation = (Calculation)context.getBean("calculationBean");
9. System.out.println(calculation.cube(7));
10. }
11. }
```

How to run this example

First Compile and Run the Host.java

Then, Compile and Run the Client.java

☺ Http Invoker

Spring Remoting by HTTP Invoker Example

Spring provides its own implementation of remoting service known as **HttpInvoker**. It can be used for http request than RMI and works well across the firewall.

By the help of **HttpInvokerServiceExporter** and **HttpInvokerProxyFactoryBean** classes, we can implement the remoting service provided by Spring's Http Invokers.

Http Invoker and Other Remoting techniques

You can use many Remoting techniques, let's see which one can be best for you.

Http Invoker Vs RMI

RMI uses JRMP protocol whereas Http Invokes uses HTTP protocol. Since enterprise applications mostly use http protocol, it is the better to use HTTP Invoker. RMI also has some security issues than HTTP Invoker. HTTP Invoker works well across firewalls.

Http Invoker Vs Hessian and Burlap

HTTP Invoker is the part of Spring framework but Hessian and burlap are proprietary. All works well across firewall. Hessian and Burlap are portable to integrate with other languages such as .Net and PHP but HTTP Invoker cannot be.

Example of Spring HTTP Invoker

To create a simple spring's HTTP invoker application, you need to create following files.

1. **Calculation.java**
2. **CalculationImpl.java**
3. **web.xml**
4. **httpInvoker-servlet.xml**
5. **client-beans.xml**
6. **Client.java**

1) Calculation.java

It is the simple interface containing one method cube.

```
1. package com.smgc;
2. public interface Calculation {
3. int cube(int number);
4. }
```

---

2) CalculationImpl.java

This class provides the implementation of Calculation interface.

```
1. package com.smgc;
2. public class CalculationImpl implements Calculation{
3.     public int cube(int number) {
4.         return number*number*number;
5.     }
6. }
```

---

3) web.xml

In this xml file, we are defining DispatcherServlet as the front controller. If any request is followed by .http extension, it will be forwarded to DispatcherServlet.

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <web-app version="2.5"
3.     xmlns="http://java.sun.com/xml/ns/javaee"
4.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.     xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
6.     http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">
7.
8.     <servlet>
9.     <servlet-name>httpInvoker</servlet-name>
10.    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
11.    <load-on-startup>1</load-on-startup>
12. </servlet>
13.
14. <servlet-mapping>
15.    <servlet-name>httpInvoker</servlet-name>
16.    <url-pattern>*.http</url-pattern>
17. </servlet-mapping>
18.
19. </web-app>
```

---

4) httpInvoker-servlet.xml

It must be created inside the WEB-INF folder. Its name must be servletname-servlet.xml. It defines bean for **CalculationImpl** and **HttpInvokerServiceExporter**.

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans xmlns="http://www.springframework.org/schema/beans"
3.    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.    xsi:schemaLocation="http://www.springframework.org/schema/beans
5.    http://www.springframework.org/schema/beans/spring-beans.xsd">
6.
7. <bean id="calculationBean" class="com.smgc.CalculationImpl"></bean>
8. <bean name="/Calculation.http"
9. class="org.springframework.remoting.httpinvoker.HttpInvokerServiceExporter">
10.    <property name="service" ref="calculationBean"></property>
11.    <property name="serviceInterface" value="com.smgc.Calculation"></property>
12. </bean>
13.
14. </beans>
```

---

5) client-beans.xml

In this xml file, we are defining bean for **HttpInvokerProxyFactoryBean**. You need to define two properties of this class.

```
1. serviceUrl
2. serviceInterface
```

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans xmlns="http://www.springframework.org/schema/beans"
3.    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.    xsi:schemaLocation="http://www.springframework.org/schema/beans
5. http://www.springframework.org/schema/beans/spring-beans.xsd">
6.
7. <bean id="calculationBean"
8. class="org.springframework.remoting.httpinvoker.HttpInvokerProxyFactoryBean">
9.    <property name="serviceUrl"
10.       value="http://localhost:8888/httpinvoker/Calculation.http"></property>
11.    <property name="serviceInterface" value="com.smgc.Calculation"></property>
12. </bean>
13. </beans>
```

---

6) Client.java

This class gets the instance of Calculation and calls the method.

```
1. package com.smgc;
2. import org.springframework.context.ApplicationContext;
3. import org.springframework.context.support.ClassPathXmlApplicationContext;
```

```
4.
5.  public class Client {
6.   public static void main(String[] args){
7.    ApplicationContext context = new ClassPathXmlApplicationContext("client-beans.xml");
8.    Calculation calculation = (Calculation)context.getBean("calculationBean");
9.    System.out.println(calculation.cube(5));
10.  }
11. }
```

Output

```
1.  Output: 125
```

How to run this example

Start and deploy the project, here we are assuming that server is running on 8888 port number. If the port number is different, change the serviceURL in client-beans.xml.

Then, Compile and Run the Client.java file.

Web-based Client

In the example given above, we used console based client. We can also use web based client. You need to create 3 additional files. Here, we are using following files:

1. ClientInvoker.java
2. index.jsp
3. process.jsp

ClientInvoker.java

It defines only one method getCube() that returns cube of the given number

```
1.  package com.smgc;
2.  import org.springframework.context.ApplicationContext;
3.  import org.springframework.context.support.ClassPathXmlApplicationContext;
4.
5.  public class ClientInvoker {
6.      public static int getCube(int number){
7.          ApplicationContext context = new ClassPathXmlApplicationContext("client-beans.xml");
8.          Calculation calculation = (Calculation)context.getBean("calculationBean");
9.          return calculation.cube(number);
10.     }
11. }
```

index.jsp

It creates a form to get number.

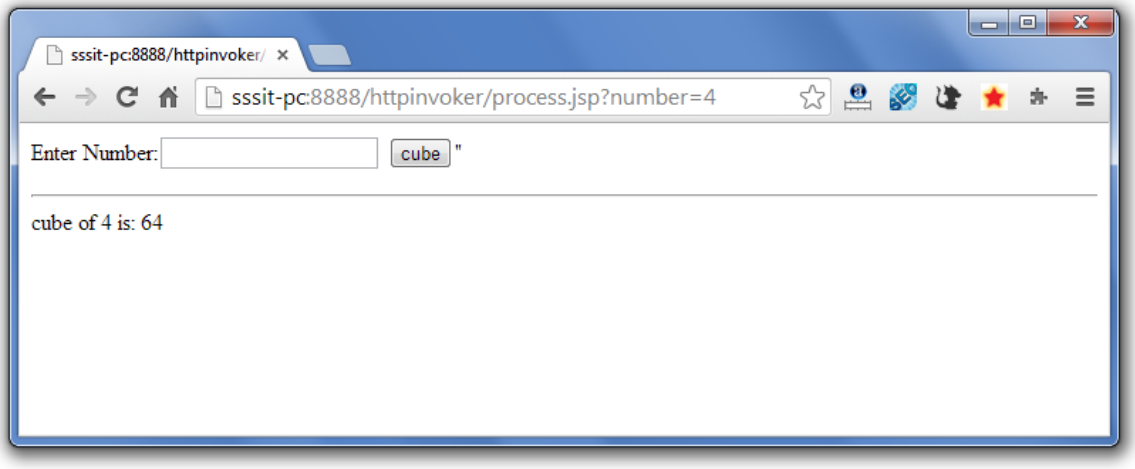
```
1. <form action="process.jsp">
2. Enter Number:<input type="text" name="number"/>
3. <input type="submit" value="cube" />
4. </form>
```

process.jsp

It creates a form to get number.

```
1. <jsp:include page="index.jsp"></jsp:include>
2. <hr/>
3. <%@page import="com.smgc.ClientInvoker"%>
4.
5. <%
6. int number=Integer.parseInt(request.getParameter("number"));
7. out.print("cube of "+number+" is: "+ClientInvoker.getCube(number));
8. %>
```

Output



☺ Hessian

Spring Remoting by Hessian Example

1. Spring Remoting by Hessian
2. Example of Spring Hessian

By the help of **HessianServiceExporter** and **HessianProxyFactoryBean** classes, we can implement the remoting service provided by hessian.

Advantage of Hessian

Hessian works well across firewall. Hessian is portable to integrate with other languages such as PHP and .Net.

Example of Remoting by Hessian

You need to create following files for creating a simple hessian application:

1. **Calculation.java**
2. **CalculationImpl.java**
3. **web.xml**
4. **hessian-servlet.xml**
5. **client-beans.xml**
6. **Client.java**

1) Calculation.java

It is the simple interface containing one method cube.

```
1. package com.smgc;
2. public interface Calculation {
3. int cube(int number);
4. }
```

---

2) CalculationImpl.java

This class provides the implementation of Calculation interface.

```
1. package com.smgc;
2. public class CalculationImpl implements Calculation{
3.     public int cube(int number) {
4.         return number*number*number;
5.     }
6. }
```

---

3) web.xml

In this xml file, we are defining DispatcherServlet as the front controller. If any request is followed by .http extension, it will be forwarded to DispatcherServlet.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="2.5"
    xmlns="http://java.sun.com/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
    http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">

    <servlet>
    <servlet-name>hessian</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>1</load-on-startup>
</servlet>

<servlet-mapping>
    <servlet-name>hessian</servlet-name>
    <url-pattern>*.http</url-pattern>
</servlet-mapping>

</web-app>
```

---

4) hessian-servlet.xml

It must be created inside the WEB-INF folder. Its name must be servletname-servlet.xml. It defines bean for **CalculationImpl** and **HessianServiceExporter**.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">

<bean id="calculationBean" class="com.smgc.CalculationImpl"></bean>
<bean name="/Calculation.http"
class="org.springframework.remoting.caucho.HessianServiceExporter">
    <property name="service" ref="calculationBean"></property>
    <property name="serviceInterface" value="com.smgc.Calculation"></property>
</bean>

</beans>
```

---

5) client-beans.xml

In this xml file, we are defining bean for **HessianProxyFactoryBean**. You need to define two properties of this class.

1. serviceUrl
2. serviceInterface

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

<bean id="calculationBean"
class="org.springframework.remoting.caucho.HessianProxyFactoryBean">
    <property name="serviceUrl"
        value="http://localhost:8888/hessian/Calculation.http"></property>
    <property name="serviceInterface" value="com.smgc.Calculation"></property>
</bean>
</beans>
```

In this example, our project name is hessian, i.e. used as the context root in the serviceURL.

---

6) Client.java

This class gets the instance of Calculation and calls the cube method.

```
package com.smgc;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Client {
 public static void main(String[] args){
  ApplicationContext context = new ClassPathXmlApplicationContext("client-beans.xml");
  Calculation calculation = (Calculation)context.getBean("calculationBean");
  System.out.println(calculation.cube(5));
 }
}
```

How to run this example

Start and deploy the project, here we are assuming that server is running on 8888 port number. If the port number is different, change the serviceURL in client-beans.xml.

Then, Compile and Run the Client.java file.

☺ Burlap

Spring Remoting by Burlap Example

1. Spring Remoting by Burlap
2. Example of Spring Burlap

Both, Hessian and Burlap are provided by Coucho. Burlap is the xml-based alternative of Hessian.

By the help of **BurlapServiceExporter** and **BurlapProxyFactoryBean** classes, we can implement the remoting service provided by burlap.

Example of Burlap is same as Hessian, you need to change Hessian to Burlap only.

Example of Remoting by Burlap

You need to create following files for creating a simple burlap application:

1. **Calculation.java**
2. **CalculationImpl.java**
3. **web.xml**
4. **burlap-servlet.xml**
5. **client-beans.xml**
6. **Client.java**

1) Calculation.java

It is the simple interface containing one method cube.

```
1. package com.smgc;
2. public interface Calculation {
3. int cube(int number);
4. }
```

---

2) CalculationImpl.java

This class provides the implementation of Calculation interface.

```
1. package com.smgc;
2. public class CalculationImpl implements Calculation{
3.     public int cube(int number) {
4.         return number*number*number;
5.     }
6. }
```

---

3) web.xml

In this xml file, we are defining DispatcherServlet as the front controller. If any request is followed by .http extension, it will be forwarded to DispatcherServlet.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="2.5"
    xmlns="http://java.sun.com/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
    http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">

    <servlet>
    <servlet-name>burlap</servlet-name>
    <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
    <load-on-startup>1</load-on-startup>
</servlet>

<servlet-mapping>
    <servlet-name>burlap</servlet-name>
    <url-pattern>*.http</url-pattern>
</servlet-mapping>

</web-app>
```

---

4) burlap-servlet.xml

It must be created inside the WEB-INF folder. Its name must be servletname-servlet.xml. It defines bean for **CalculationImpl** and **BurlapServiceExporter**.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">

<bean id="calculationBean" class="com.smgc.CalculationImpl"></bean>
<bean name="/Calculation.http"
class="org.springframework.remoting.caucho.BurlapServiceExporter">
    <property name="service" ref="calculationBean"></property>
    <property name="serviceInterface" value="com.smgc.Calculation"></property>
</bean>

</beans>
```

---

5) client-beans.xml

In this xml file, we are defining bean for **BurlapProxyFactoryBean**. You need to define two properties of this class.

1. serviceUrl
2. serviceInterface

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans xmlns="http://www.springframework.org/schema/beans"
3.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.     xsi:schemaLocation="http://www.springframework.org/schema/beans
5. http://www.springframework.org/schema/beans/spring-beans.xsd">
6.
7. <bean id="calculationBean"
8. class="org.springframework.remoting.caucho.BurlapProxyFactoryBean">
9.     <property name="serviceUrl"
10.         value="http://localhost:8888/burlap/Calculation.http"></property>
11.    <property name="serviceInterface" value="com.smgc.Calculation"></property>
12. </bean>
13. </beans>
```

In this example, our project name is burlap, i.e. used as the context root in the serviceURL.

---

6) Client.java

This class gets the instance of Calculation and calls the cube method.

```
1. package com.smgc;
2. import org.springframework.context.ApplicationContext;
3. import org.springframework.context.support.ClassPathXmlApplicationContext;
4.
5. public class Client {
6.  public static void main(String[] args){
7.   ApplicationContext context = new ClassPathXmlApplicationContext("client-beans.xml");
8.   Calculation calculation = (Calculation)context.getBean("calculationBean");
9.   System.out.println(calculation.cube(3));
10.  }
11. }
```

How to run this example

Start and deploy the project, here we are assuming that server is running on 8888 port number. If the port number is different, change the serviceURL in client-beans.xml.

Then, Compile and Run the Client.java file.

☺ Spring with JMS

Spring and JMS Integration

To integrate spring with JMS, you need to create two applications.

1. JMS Receiver Application
2. JMS Sender Application

To create JMS application using spring, we are using **Active MQ Server** of Apache to create the Queue.

Let's see the simple steps to integration spring application with JMS:

Required Jar Files

1) You need to add **spring core**, **spring misc**,**spring aop**, **spring j2ee** and **spring persistence core** jar files.

2) Add **activemqall5.9.jar** file located inside the activemq directory.

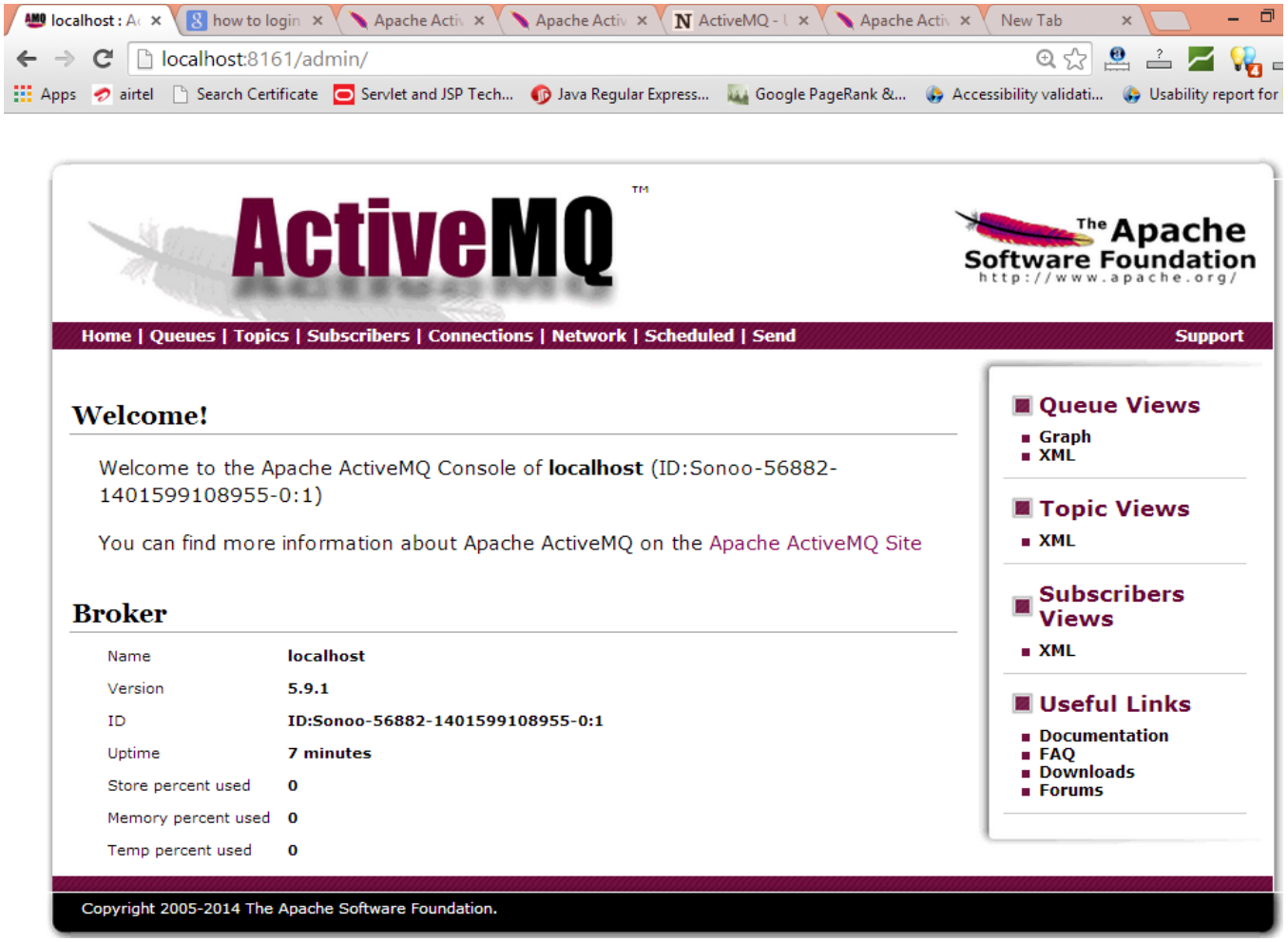
Create a queue in ActiveMQ Server

Download the Active MQ Server

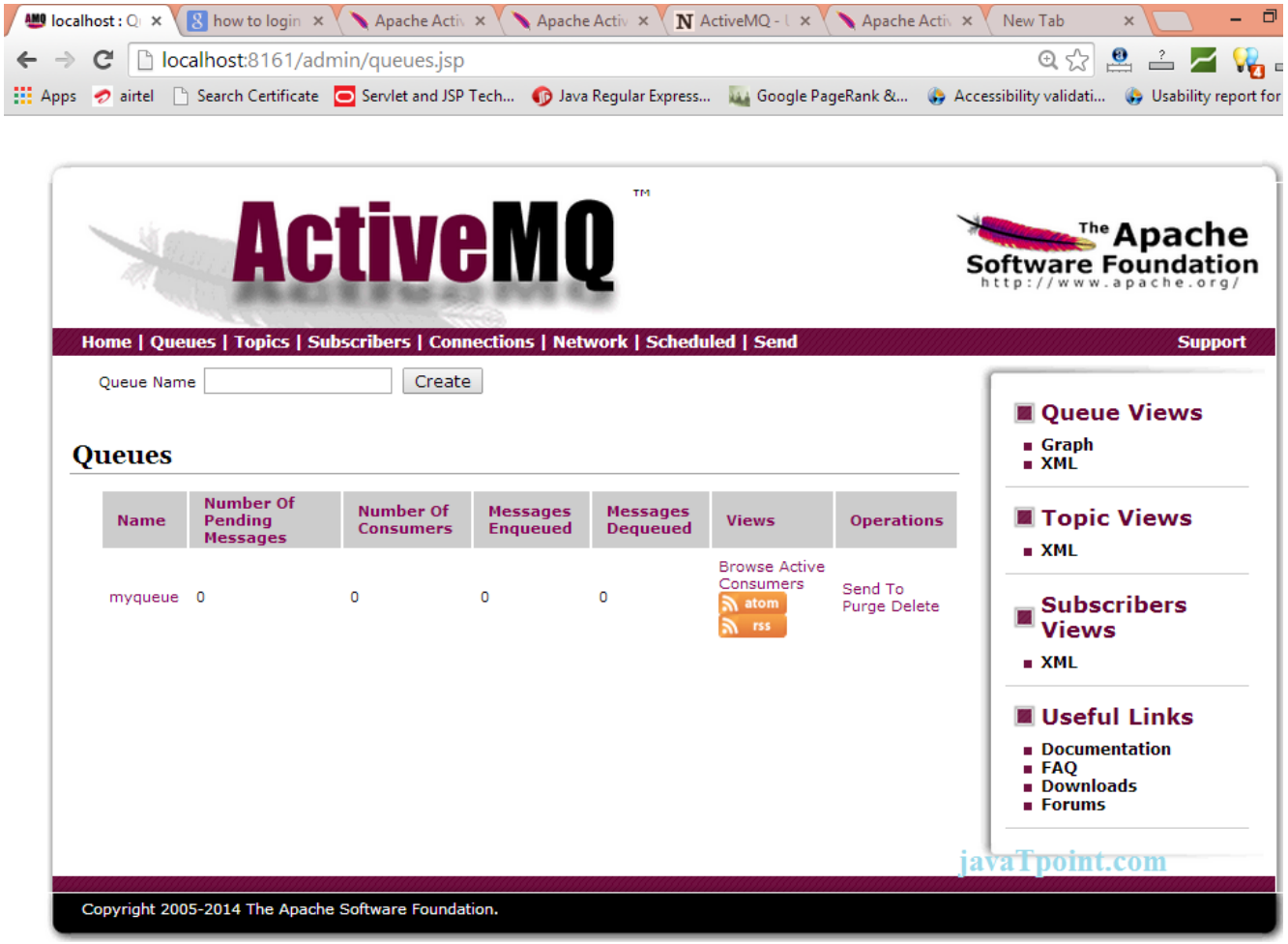
Double Click on the **activemq.bat** file located inside apache-activemq-5.9.1-bin\apache-activemq-5.9.1\bin\win64 or win32 directory.

Now activemq server console will open.

Access the admin console of activemq server by **http://localhost:8161/admin/** url.



Now, click on the **Queues link**, write **myqueue** in the textfield and click on the create button.



1) JMS Receiver Application

Let's see the simple steps to integration spring application with JMS:

1. **MyMessageListener.java**
2. **TestListener.java**
3. **applicationContext.xml**

1) MyMessageListener.java

```
1.  package com.smgc;
2.  import javax.jms.Message;
3.  import javax.jms.MessageListener;
4.  import javax.jms.TextMessage;
5.  public class MyMessageListener implements MessageListener{
6.      @Override
7.      public void onMessage(Message m) {
8.          TextMessage message=(TextMessage)m;
9.          try{
10.             System.out.println(message.getText());
11.         }catch (Exception e) {e.printStackTrace();  }
```

```
12.     }
13. }
```

---

2) TestListener.java

```
1.  package com.smgc;
2.  import org.springframework.context.support.GenericXmlApplicationContext;
3.  public class TestListener {
4.  public static void main(String[] args) {
5.      GenericXmlApplicationContext ctx=new GenericXmlApplicationContext();
6.      ctx.load("classpath:applicationContext.xml");
7.      ctx.refresh();
8.
9.      while(true){}
10. }
11. }
```

3) applicationContext.xml

```
1.  <?xml version="1.0" encoding="UTF-8"?>
2.  <beans
3.      xmlns="http://www.springframework.org/schema/beans"
4.      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.      xmlns:jms="http://www.springframework.org/schema/jms"
6.
7.      xmlns:p="http://www.springframework.org/schema/p"
8.      xsi:schemaLocation="http://www.springframework.org/schema/beans
9.      http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
10.     http://www.springframework.org/schema/jms
11.     http://www.springframework.org/schema/jms/spring-jms-3.0.xsd">
12.
13. <bean id="connectionFactory" class="org.apache.activemq.ActiveMQConnectionFactory"
14.  p:brokerURL="tcp://localhost:61616" />
15.
16. <bean id="listener" class="com.smgc.MyMessageListener"></bean>
17.
18. <jms:listener-container container-type="default" connection-factory="connectionFactory"
19.  acknowledge="auto">
20. <jms:listener destination="myqueue" ref="listener" method="onMessage"></jms:listener>
21. </jms:listener-container>
22.
23. </beans>
```

---

2) JMS Sender Application

Let's see the files to create the JMS Sender application:

1. **MyMessageSender.java**
2. **TestJmsSender.java**
3. **applicationContext.xml**

1) MyMessageListener.java

```
1. package com.smgc;
2. import javax.jms.*;
3. import org.springframework.beans.factory.annotation.Autowired;
4. import org.springframework.jms.core.JmsTemplate;
5. import org.springframework.jms.core.MessageCreator;
6. import org.springframework.stereotype.Component;
7.
8. @Component("messageSender")
9. public class MyMessageSender {
10. @Autowired
11. private JmsTemplate jmsTemplate;
12. public void sendMessage(final String message){
13.     jmsTemplate.send(new MessageCreator(){
14.
15.         @Override
16.         public Message createMessage(Session session) throws JMSException {
17.             return session.createTextMessage(message);
18.         }
19.     });
20. }
21. }
```

---

2) TestJmsSender.java

```
1. package com.smgc;
2. import org.springframework.context.support.GenericXmlApplicationContext;
3. public class TestJmsSender {
4. public static void main(String[] args) {
5.     GenericXmlApplicationContext ctx=new GenericXmlApplicationContext();
6.     ctx.load("classpath:applicationContext.xml");
7.     ctx.refresh();
8.
9.     MyMessageSender sender=ctx.getBean("messageSender",MyMessageSender.class);
10.    sender.sendMessage("hello jms3");
11.
12. }
13. }
```

3) applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
   xmlns="http://www.springframework.org/schema/beans"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xmlns:context="http://www.springframework.org/schema/context"
   xmlns:jms="http://www.springframework.org/schema/jms"

   xmlns:p="http://www.springframework.org/schema/p"
   xsi:schemaLocation="http://www.springframework.org/schema/beans
   http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
   http://www.springframework.org/schema/context
   http://www.springframework.org/schema/context/spring-context-3.0.xsd
   http://www.springframework.org/schema/jms
   http://www.springframework.org/schema/jms/spring-jms-3.0.xsd">

<bean id="connectionFactory" class="org.apache.activemq.ActiveMQConnectionFactory"
 p:brokerURL="tcp://localhost:61616" />

<bean id="jmsTemplate" class="org.springframework.jms.core.JmsTemplate">
<constructor-arg name="connectionFactory" ref="connectionFactory"></constructor-arg>
<property name="defaultDestinationName" value="myqueue"></property>
</bean>

<context:component-scan base-package="com.javatpoint"></context:component-scan>

</beans>
```

Chapter-3.1

OXM Framework, Spring JAVA Mail and Web Integration

☺ Spring with JAXB

Spring and JAXB Integration Example

1. Spring and JAXB Integration
2. Example of Spring and JAXB Integration

JAXB is an acronym for **Java Architecture for XML Binding**. It allows java developers to map Java class to XML representation. JAXB can be used to marshal java objects into XML and vice-versa.

It is an OXM (Object XML Mapping) or O/M framework provided by Sun.

Advantage of JAXB

No need to create or use a SAX or DOM parser and write callback methods.

Example of Spring and JAXB Integration (Marshalling Java Object into XML)

You need to create following files for marshalling java object into XML using Spring with JAXB:

1. **Employee.java**
2. **applicationContext.xml**
3. **Client.java**

Required Jar files

To run this example, you need to load:

- **Spring Core jar files**
- **Spring Web jar files**

Employee.java

If defines three properties id, name and salary. We have used following annotations in this class:

1. **@XmlRootElement** It specifies the root element for the xml file.
2. **@XmlAttribute** It specifies attribute for the property.
3. **@XmlElement** It specifies the element.

```
1. package com.smgc;
2. import javax.xml.bind.annotation.XmlAttribute;
3. import javax.xml.bind.annotation.XmlElement;
4. import javax.xml.bind.annotation.XmlRootElement;
5.
6. @XmlRootElement(name="employee")
7. public class Employee {
8. private int id;
9. private String name;
10. private float salary;
11.
12. @XmlAttribute(name="id")
13. public int getId() {
14.     return id;
15. }
16. public void setId(int id) {
17.     this.id = id;
18. }
19. @XmlElement(name="name")
20. public String getName() {
21.     return name;
22. }
23. public void setName(String name) {
24.     this.name = name;
25. }
26. @XmlElement(name="salary")
27. public float getSalary() {
28.     return salary;
29. }
30. public void setSalary(float salary) {
31.     this.salary = salary;
32. }
33. }
```

---

applicationContext.xml

It defines a bean jaxbMarshallerBean where Employee class is bound with the OXM framework.

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans xmlns="http://www.springframework.org/schema/beans"
3.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.     xmlns:oxm="http://www.springframework.org/schema/oxm"
```

```
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
        http://www.springframework.org/schema/oxm
        http://www.springframework.org/schema/oxm/spring-oxm-3.0.xsd">

        <oxm:jaxb2-marshaller id="jaxbMarshallerBean">
            <oxm:class-to-be-bound name="com.smgc.Employee"/>
        </oxm:jaxb2-marshaller>

</beans>
```

Client.java

It gets the instance of Marshaller from the applicationContext.xml file and calls the marshal method.

```
package com.smgc;
import java.io.FileWriter;
import java.io.IOException;
import javax.xml.transform.stream.StreamResult;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.oxm.Marshaller;

public class Client{
 public static void main(String[] args)throws IOException{
  ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
  Marshaller marshaller = (Marshaller)context.getBean("jaxbMarshallerBean");

  Employee employee=new Employee();
  employee.setId(101);
  employee.setName("Piyush Patel");
  employee.setSalary(100000);

  marshaller.marshal(employee, new StreamResult(new FileWriter("employee.xml")));

  System.out.println("XML Created Sucessfully");
 }
}
```

Output of the example

employee.xml

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<employee id="101">
<name>Piyush Patel</name>
<salary>100000.0</salary>
</employee>
```

☺ Spring with Xstream

Spring with Xstream Example

1. Spring and Xstream Integration
2. Example of Spring and Xstream Integration

Xstream is a library to serialize objects to xml and vice-versa without requirement of any mapping file. Notice that castor requires an mapping file.

XStreamMarshaller class provides facility to marshal objects into xml and vice-versa.

Example of Spring and Xstream Integration (Marshalling Java Object into XML)

You need to create following files for marshalling java object into XML using Spring with Xstream:

1. **Employee.java**
2. **applicationContext.xml**
3. **Client.java**

Required Jar files

To run this example, you need to load:

- **Spring Core jar files**
- **Spring Web jar files**
- **xstream-1.3.jar**

Employee.java

If defines three properties id, name and salary with setters and getters.

```
1. package com.smgc;
2. public class Employee {
3. private int id;
4. private String name;
5. private float salary;
6.
7. public int getId() {
8.     return id;
9. }
10. public void setId(int id) {
11.     this.id = id;
12. }
```

```
13. public String getName() {
14.     return name;
15. }
16. public void setName(String name) {
17.     this.name = name;
18. }
19. public float getSalary() {
20.     return salary;
21. }
22. public void setSalary(float salary) {
23.     this.salary = salary;
24. }
25. }
```

---

applicationContext.xml

It defines a bean xstreamMarshallerBean where Employee class is bound with the OXM framework.

```
1.  <?xml version="1.0" encoding="UTF-8"?>
2.  <beans xmlns="http://www.springframework.org/schema/beans"
3.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.     xsi:schemaLocation="http://www.springframework.org/schema/beans
5.  http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
6.
7.  <bean id="xstreamMarshallerBean" class="org.springframework.oxm.xstream.XStreamMarshaller">
8.     <property name="annotatedClasses" value="com.smgc.Employee"></property>
9.  </bean>
10. </beans>
```

Client.java

It gets the instance of Marshaller from the applicationContext.xml file and calls the marshal method.

```
1.  package com.smgc;
2.  import java.io.FileWriter;
3.  import java.io.IOException;
4.  import javax.xml.transform.stream.StreamResult;
5.  import org.springframework.context.ApplicationContext;
6.  import org.springframework.context.support.ClassPathXmlApplicationContext;
7.  import org.springframework.oxm.Marshaller;
8.
9.  public class Client{
10.  public static void main(String[] args)throws IOException{
11.   ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
12.   Marshaller marshaller = (Marshaller)context.getBean("xstreamMarshallerBean");
13.
14.   Employee employee=new Employee();
15.   employee.setId(101);
16.   employee.setName("Piyush Patel");
```

```
17.   employee.setSalary(100000);
18.
19.   marshaller.marshal(employee, new StreamResult(new FileWriter("employee.xml")));
20.
21.   System.out.println("XML Created Sucessfully");
22.  }
23. }
```

Output of the example

employee.xml

```
1.  <com.javatpoint.Employee>
2.  <id>101</id>
3.  <name>Piyush Patel</name>
4.  <salary>100000.0</salary>
5.  </com.javatpoint.Employee>
```

☺ Spring with Caster

Spring with Castor Example

1. Spring and Castor Integration
2. Example of Spring and Castor Integration

By the help of **CastorMarshaller** class, we can marshal the java objects into xml and vice-versa using castor. It is the implementation class for Marshaller and Unmarshaller interfaces. It doesn't require any further configuration bydefault.

Example of Spring and Castor Integration (Marshalling Java Object into XML)

You need to create following files for marshalling java object into XML using Spring with Castor:

1. **Employee.java**
2. **applicationContext.xml**
3. **mapping.xml**
4. **Client.java**

Required Jar files

To run this example, you need to load:

- **Spring Core jar files**
- **Spring Web jar files**
- **castor-1.3.jar**
- **castor-1.3-core.jar**

Employee.java

If defines three properties id, name and salary with setters and getters.

```
1. package com.smgc;
2. public class Employee {
3. private int id;
4. private String name;
5. private float salary;
6.
7. public int getId() {
8.     return id;
9. }
10. public void setId(int id) {
11.     this.id = id;
```

```
12. }
13. public String getName() {
14.     return name;
15. }
16. public void setName(String name) {
17.     this.name = name;
18. }
19. public float getSalary() {
20.     return salary;
21. }
22. public void setSalary(float salary) {
23.     this.salary = salary;
24. }
25. }
```

---

applicationContext.xml

It defines a bean castorMarshallerBean where Employee class is bound with the OXM framework.

```
1.  <?xml version="1.0" encoding="UTF-8"?>
2.  <beans xmlns="http://www.springframework.org/schema/beans"
3.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.     xsi:schemaLocation="http://www.springframework.org/schema/beans
5.  http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
6.  <bean id="castorMarshallerBean" class="org.springframework.oxm.castor.CastorMarshaller">
7.         <property name="targetClass" value="com.smgc.Employee"></property>
8.         <property name="mappingLocation" value="mapping.xml"></property>
9.     </bean>
10. </beans>
```

---

mapping.xml

```
1.  <?xml version="1.0"?>
2.  <!DOCTYPE mapping PUBLIC "-//EXOLAB/Castor Mapping DTD Version 1.0//EN"
3.                      "http://castor.org/mapping.dtd">
4.   <mapping>
5.     <class name="com.smgc.Employee" auto-complete="true" >
6.       <map-to xml="Employee" ns-uri="http://www.javatpoint.com" ns-prefix="dp"/>
7.       <field name="id" type="integer">
8.         <bind-xml name="id" node="attribute"></bind-xml>
9.       </field>
10.      <field name="name">
11.        <bind-xml name="name"></bind-xml>
12.      </field>
13.      <field name="salary">
14.        <bind-xml name="salary" type="float"></bind-xml>
15.      </field>
16.
17.    </class>
```

```
18.
19.  </mapping>
```

Client.java

It gets the instance of Marshaller from the applicationContext.xml file and calls the marshal method.

```
1.  package com.smgc;
2.  import java.io.FileWriter;
3.  import java.io.IOException;
4.  import javax.xml.transform.stream.StreamResult;
5.  import org.springframework.context.ApplicationContext;
6.  import org.springframework.context.support.ClassPathXmlApplicationContext;
7.  import org.springframework.oxm.Marshaller;
8.
9.  public class Client{
10.  public static void main(String[] args)throws IOException{
11.   ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
12.   Marshaller marshaller = (Marshaller)context.getBean("castorMarshallerBean");
13.
14.   Employee employee=new Employee();
15.   employee.setId(101);
16.   employee.setName("Piyush Patel");
17.   employee.setSalary(100000);
18.
19.   marshaller.marshal(employee, new StreamResult(new FileWriter("employee.xml")));
20.
21.   System.out.println("XML Created Sucessfully");
22.  }
23. }
```

Output of the example

employee.xml

```
1.  <?xml version="1.0" encoding="UTF-8"?>
2.  <dp:Employee xmlns:dp="http://www.javatpoint.com" id="101">
3.  <dp:name>Piyush Patel</dp:name>
4.  <dp:salary>100000.0</dp:salary>
5.  </dp:Employee>
```

☺ Spring with Struts2

Spring and Struts 2 Integration

1. Spring and Struts 2 Integration
2. Example of Spring and Struts 2 Integration

Spring framework provides an easy way to manage the dependency. It can be easily integrated with struts 2 framework.

The **ContextLoaderListener** class is used to communicate spring application with struts 2. It must be specified in the web.xml file.

You need to follow following steps:

1. Create struts2 application and add spring jar files.
2. In **web.xml** file, define ContextLoaderListener class.
3. In **struts.xml** file, define bean name for the action class.
4. In **applicationContext.xml** file, create the bean. Its class name should be action class name e.g. com.javatpoint.Login and id should match with the action class of struts.xml file (e.g. login).
5. In the **action class**, define extra property e.g. message.

Example of Spring and Struts 2 Integration

You need to create following files for simple spring and struts 2 application:

1. **index.jsp**
2. **web.xml**
3. **struts.xml**
4. **applicationContext.xml**
5. **Login.java**
6. **welcome.jsp**
7. **error.jsp**

1) index.jsp

This page gets the name from the user.

```
<%@ taglib uri="/struts-tags" prefix="s"%>

<s:form action="login">
<s:textfield name="userName" label="UserName"></s:textfield>
<s:submit></s:submit>
</s:form>
```

---

2) web.xml

It defines controller class for struts 2 and **ContextLoaderListener** listener class to make connection between struts2 and spring application.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="2.5"
    xmlns="http://java.sun.com/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
    http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>
  <filter>
    <filter-name>struts2</filter-name>
    <filter-class>
       org.apache.struts2.dispatcher.ng.filter.StrutsPrepareAndExecuteFilter
    </filter-class>
  </filter>

<listener>
<listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>

  <filter-mapping>
    <filter-name>struts2</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>

  </web-app>
```

---

3) struts.xml

It defines the package with action and result. Here, the action class name is login which will be searched in the applicationContext.xml file.

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE struts PUBLIC "-//Apache Software Foundation//DTD Struts Configuration 2.1//EN"
"http://struts.apache.org/dtds/struts-2.1.dtd">
<struts>
<package name="abc" extends="struts-default">
<action name="login" class="login">
<result name="success">welcome.jsp</result>
</action>

</package>

</struts>
```

4) applicationContext.xml

It defines a bean with id login. This beans corresponds to the mypack.Login class. It will be considered as the action class here.

It should be located inside the WEB-INF directory.

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans
3.    xmlns="http://www.springframework.org/schema/beans"
4.    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.    xmlns:p="http://www.springframework.org/schema/p"
6.    xsi:schemaLocation="http://www.springframework.org/schema/beans
7. http://www.springframework.org/schema/beans/spring-beans-2.5.xsd">
8.
9. <bean id="login" class="mypack.Login">
10. <property name="message" value="Welcome Spring"></property>
11. </bean>
12.
13. </beans>
```

5) Login.java

It defines two property userName and message with execute method where success is returned.

```
1. package mypack;
2. public class Login {
3. private String userName,message;
4.
5. public String getMessage() {
6.    return message;
7. }
8. public void setMessage(String message) {
9.    this.message = message;
10. }
11. public String getUserName() {
12.    return userName;
13. }
14. public void setUserName(String userName) {
15.    this.userName = userName;
16. }
17. public String execute(){
18.    return "success";
19. }
20. }
```

6) welcome.jsp

It prints values of userName and message properties.

```
1. <%@ taglib uri="/struts-tags" prefix="s"%>
2.
3. Welcome, <s:property value="userName"/><br/>
4. ${message}
```

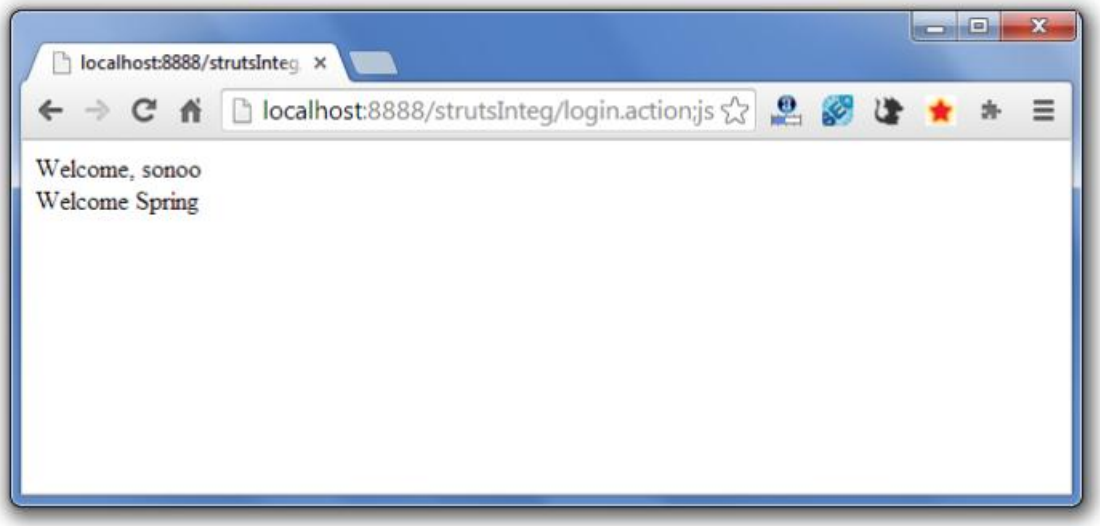
---

7) error.jsp

It is the error page. But it is not required in this example because we are not defining any logic in the execute method of action class.

```
1. Sorry!
```

Output



☺ Login and Logout Application

Login Example with Spring and Struts 2 Integration

1. Spring and Struts2 Integration
2. Example of Spring and Struts2 Integration

In the previous example, we have simply integrated the spring application with struts 2. Now let's develop a login application with struts 2 and spring frameworks.

It is the simple example of login application without database and session management. If you need to apply the database interactivity and session management with this example.

Example of Login application Spring and Struts2 Integration

You need to create following files for simple login application using spring and struts 2 application:

1. **index.jsp**
2. **web.xml**
3. **struts.xml**
4. **applicationContext.xml**
5. **Login.java**
6. **login_success.jsp**
7. **login_error.jsp**

1) index.jsp

This page gets the name and password from the user.

```
1. <%@ taglib uri="/struts-tags" prefix="s"%>
2.
3. <s:form action="login">
4. <s:textfield name="name" label="Username"></s:textfield>
5. <s:password name="password" label="Password"></s:password>
6. <s:submit value="login"></s:submit>
7. </s:form>
```

---

2) web.xml

It defines controller class for struts 2 and **ContextLoaderListener** listener class to make connection between struts2 and spring application.

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <web-app version="2.5"
3.     xmlns="http://java.sun.com/xml/ns/javaee"
4.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.     xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
6.     http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">
7.   <welcome-file-list>
```

```
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>
  <filter>
    <filter-name>struts2</filter-name>
    <filter-class>
       org.apache.struts2.dispatcher.ng.filter.StrutsPrepareAndExecuteFilter
    </filter-class>
  </filter>

<listener>
<listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>

  <filter-mapping>
    <filter-name>struts2</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>

  </web-app>
```

---

3) struts.xml

It defines the package with action and result. Here, the action class name is login which will be searched in the applicationContext.xml file.

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE struts PUBLIC "-//Apache Software Foundation//DTD Struts Configuration 2.1//EN"
"http://struts.apache.org/dtds/struts-2.1.dtd">
<struts>
<package name="abc" extends="struts-default">
<action name="login" class="login">
<result name="success">login_success.jsp</result>
<result name="error">login_error.jsp</result>
</action>

</package>

</struts>
```

---

4) applicationContext.xml

It defines a bean with id login. This beans corresponds to the com.javatpoint.Login class. It will be considered as the action class here.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
   xmlns="http://www.springframework.org/schema/beans"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xmlns:p="http://www.springframework.org/schema/p"
```

```
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

<bean id="login" class="com.smgc.Login">
<property name="successmessage" value="You are successfully logged in!"></property>
<property name="errormessage" value="Sorry, username or password error!"></property>
</bean>

</beans>
```

---

5) Login.java

It defines four properties name, password, successmessage and errormessage; with execute method where success is returned if password is javatpoint.

```
package com.smgc;

public class Login {
private String name,password,successmessage,errormessage;
//setters and getters

public String execute(){
    if(password.equals("javatpoint")){
        return "success";
    }
    else{
        return "error";
    }
}
}
```

---

6) login_success.jsp

It prints values of userName and message properties.

```
<%@ taglib uri="/struts-tags" prefix="s"%>
${successmessage}
<br/>
Welcome, <s:property value="name"/><br/>
```

---

7) login_error.jsp

It is the error page. But it is not required in this example because we are not defining any logic in the execute method of action class.

```
${errormessage}
<jsp:include page="index.jsp"></jsp:include>
```

Chapter-3.2

Basics of Hibernate And Hibernate with IDE

☺ Hibernate Introduction

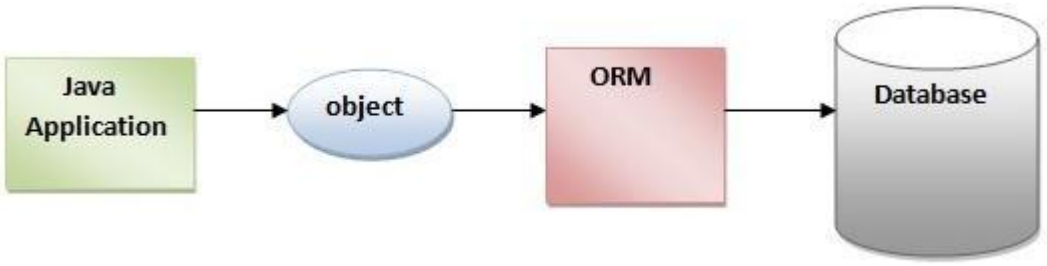
Hibernate Tutorial

This hibernate tutorial provides in-depth concepts of Hibernate Framework with simplified examples. It was started in 2001 by Gavin King as an alternative to EJB2 style entity bean. The stable release of Hibernate till July 16, 2014, is hibernate 4.3.6. It is helpful for beginners and experienced persons.

Hibernate Framework

Hibernate framework simplifies the development of java application to interact with the database. Hibernate is an open source, lightweight, ORM (Object Relational Mapping) tool.

An ORM tool simplifies the data creation, data manipulation and data access. It is a programming technique that maps the object to the data stored in the database.



The ORM tool internally uses the JDBC API to interact with the database.

Advantages of Hibernate Framework

There are many advantages of Hibernate Framework. They are as follows:

1) Opensource and Lightweight: Hibernate framework is opensource under the LGPL license and lightweight.

2) Fast performance: The performance of hibernate framework is fast because cache is internally used in hibernate framework. There are two types of cache in hibernate framework first level cache and second level cache. First level cache is enabled bydefault.

3) Database Independent query: HQL (Hibernate Query Language) is the object-oriented version of SQL. It generates the database independent queries. So you don't need to write database specific queries. Before Hibernate, If database is changed for the project, we need to change the SQL query as well that leads to the maintenance problem.

4) Automatic table creation: Hibernate framework provides the facility to create the tables of the database automatically. So there is no need to create tables in the database manually.

5) Simplifies complex join: To fetch data form multiple tables is easy in hibernate framework.

6) Provides query statistics and database status: Hibernate supports Query cache and provide statistics about query and database status.

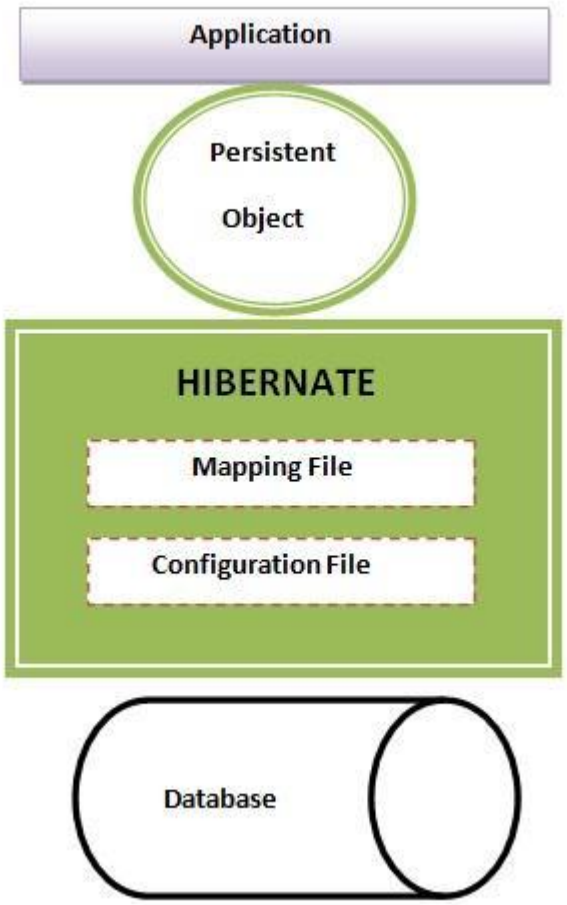
☺ Hibernate Architecture

Hibernate Architecture

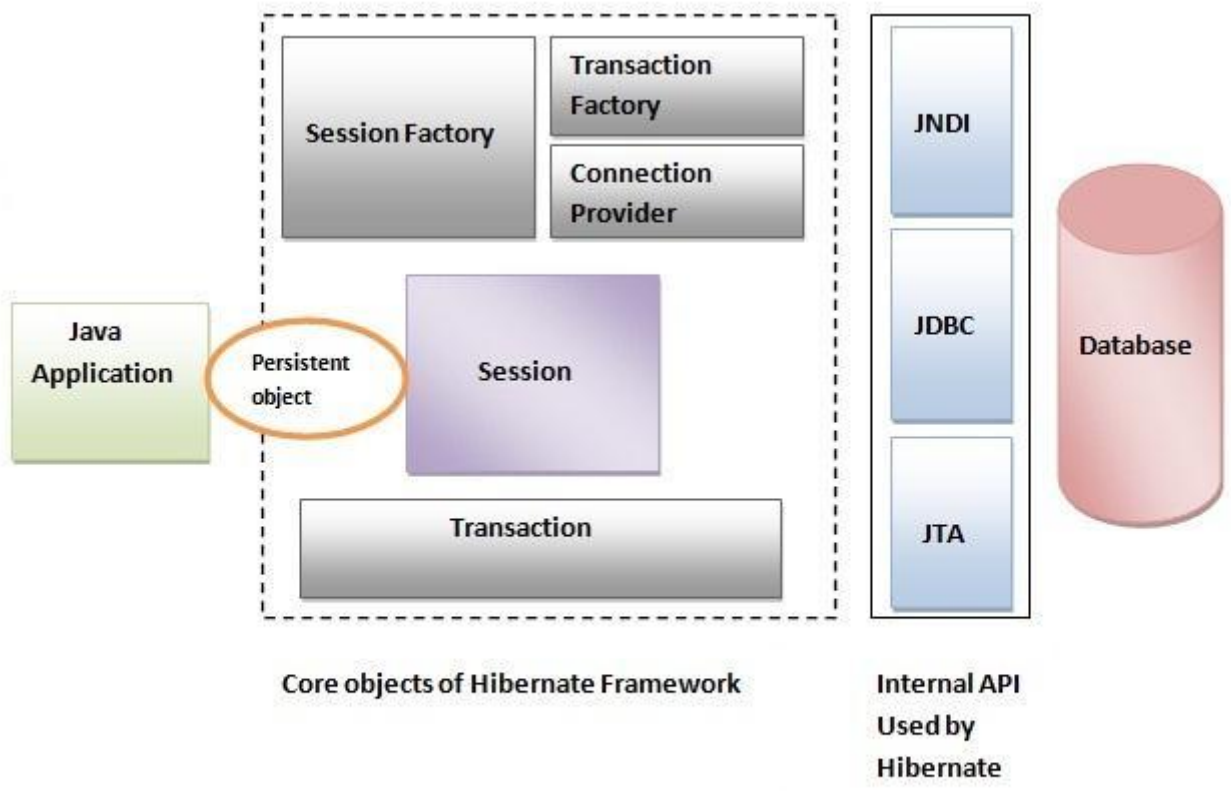
1. Hibernate Architecture
2. Elements of Hibernate Architecture
    1. SessionFactory
    2. Session
    3. Transaction
    4. ConnectionProvider
    5. TransactionFactory

The Hibernate architecture includes many objects persistent object, session factory, transaction factory, connection factory, session, transaction etc.

There are 4 layers in hibernate architecture java application layer, hibernate framework layer, backhand api layer and database layer.Let's see the diagram of hibernate architecture:



This is the high level architecture of Hibernate with mapping file and configuration file.



Hibernate framework uses many objects session factory, session, transaction etc. alongwith existing Java API such as JDBC (Java Database Connectivity), JTA (Java Transaction API) and JNDI (Java Naming Directory Interface).

Elements of Hibernate Architecture

For creating the first hibernate application, we must know the elements of Hibernate architecture. They are as follows:

SessionFactory

The SessionFactory is a factory of session and client of ConnectionProvider. It holds second level cache (optional) of data. The org.hibernate.SessionFactory interface provides factory method to get the object of Session.

Session

The session object provides an interface between the application and data stored in the database. It is a short-lived object and wraps the JDBC connection. It is factory of Transaction, Query and Criteria. It

holds a first-level cache (mandatory) of data. The org.hibernate.Session interface provides methods to insert, update and delete the object. It also provides factory methods for Transaction, Query and Criteria.

Transaction

The transaction object specifies the atomic unit of work. It is optional. The org.hibernate.Transaction interface provides methods for transaction management.

ConnectionProvider

It is a factory of JDBC connections. It abstracts the application from DriverManager or DataSource. It is optional.

TransactionFactory

It is a factory of Transaction. It is optional.

☺ Understanding First Hibernate application

Steps to create first Hibernate Application without IDE

1. Steps to create first Hibernate Application
    1. Create the Persistent class
    2. Create the mapping file for Persistent class
    3. Create the Configuration file
    4. Create the class that retrieves or stores the persistent object
    5. Load the jar file
    6. Run the first hibernate application without IDE

Here, we are going to create the first hibernate application without IDE. For creating the first hibernate application, we need to follow following steps:

1. Create the Persistent class
2. Create the mapping file for Persistent class
3. Create the Configuration file
4. Create the class that retrieves or stores the persistent object
5. Load the jar file
6. Run the first hibernate application without IDE

1) Create the Persistent class

A simple Persistent class should follow some rules:

- **A no-arg constructor:** It is recommended that you have a default constructor at least package visibility so that hibernate can create the instance of the Persistent class by newInstance() method.
- **Provide an identifier property (optional):** It is mapped to the primary key column of the database.
- **Declare getter and setter methods (optional):** The Hibernate recognizes the method by getter and setter method names by default.
- **Prefer non-final class:** Hibernate uses the concept of proxies, that depends on the persistent class. The application programmer will not be able to use proxies for lazy association fetching.

Let's create the simple Persistent class:

Employee.java

```
1.  package com.javatpoint.mypackage;
2.
3.  public class Employee {
4.  private int id;
5.  private String firstName,lastName;
6.
7.  public int getId() {
8.      return id;
9.  }
10. public void setId(int id) {
11.     this.id = id;
12. }
```

```
13. public String getFirstName() {
14.     return firstName;
15. }
16. public void setFirstName(String firstName) {
17.     this.firstName = firstName;
18. }
19. public String getLastName() {
20.     return lastName;
21. }
22. public void setLastName(String lastName) {
23.     this.lastName = lastName;
24. }
25.
26.
27. }
```

---

2) Create the mapping file for Persistent class

The mapping file name conventionally, should be class_name.hbm.xml. There are many elements of the mapping file.

- **hibernate-mapping** is the root element in the mapping file.
- **class** It is the sub-element of the hibernate-mapping element. It specifies the Persistent class.
- **id** It is the subelement of class. It specifies the primary key attribute in the class.
- **generator** It is the subelement of id. It is used to generate the primary key. There are many generator classes such as assigned (It is used if id is specified by the user), increment, hilo, sequence, native etc. We will learn all the generator classes later.
- **property** It is the subelement of class that specifies the property name of the Persistent class.

Let's see the mapping file for the Employee class:

employee.hbm.xml

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-mapping PUBLIC
3.   "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.   "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.   <hibernate-mapping>
7.    <class name="com.smgc.mypackage.Employee" table="emp1000">
8.     <id name="id">
9.      <generator class="assigned"></generator>
10.    </id>
11.    <property name="firstName"></property>
12.    <property name="lastName"></property>
13.   </class>
14.  </hibernate-mapping>
```

---

3) Create the Configuration file

The configuration file contains informations about the database and mapping file. Conventionally, its name should be hibernate.cfg.xml .

hibernate.cfg.xml

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">

<hibernate-configuration>

   <session-factory>
      <property name="hbm2ddl.auto">update</property>
      <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
      <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
      <property name="connection.username">system</property>
      <property name="connection.password">oracle</property>
      <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
   <mapping resource="employee.hbm.xml"/>
   </session-factory>

</hibernate-configuration>
```

---

4) Create the class that retrieves or stores the object

In this class, we are simply storing the employee object to the database.

```
package com.javatpoint.mypackage;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

public class StoreData {
public static void main(String[] args) {

   //creating configuration object
   Configuration cfg=new Configuration();
   cfg.configure("hibernate.cfg.xml");//populates the data of the configuration file

   //creating seession factory object
   SessionFactory factory=cfg.buildSessionFactory();

   //creating session object
   Session session=factory.openSession();
```

```
20.
21.     //creating transaction object
22.     Transaction t=session.beginTransaction();
23.
24.     Employee e1=new Employee();
25.     e1.setId(115);
26.     e1.setFirstName("Piyush");
27.     e1.setLastName("Patel");
28.
29.     session.persist(e1);//persisting the object
30.
31.     t.commit();//transaction is commited
32.     session.close();
33.
34.     System.out.println("successfully saved");
35.
36. }
37. }
```

---

5) Load the jar file

For successfully running the hibernate application, you should have the hibernate4.jar file.

Some other jar files or packages are required such as

- cglib
- log4j
- commons
- SLF4J
- dom4j
- xalan
- xerces

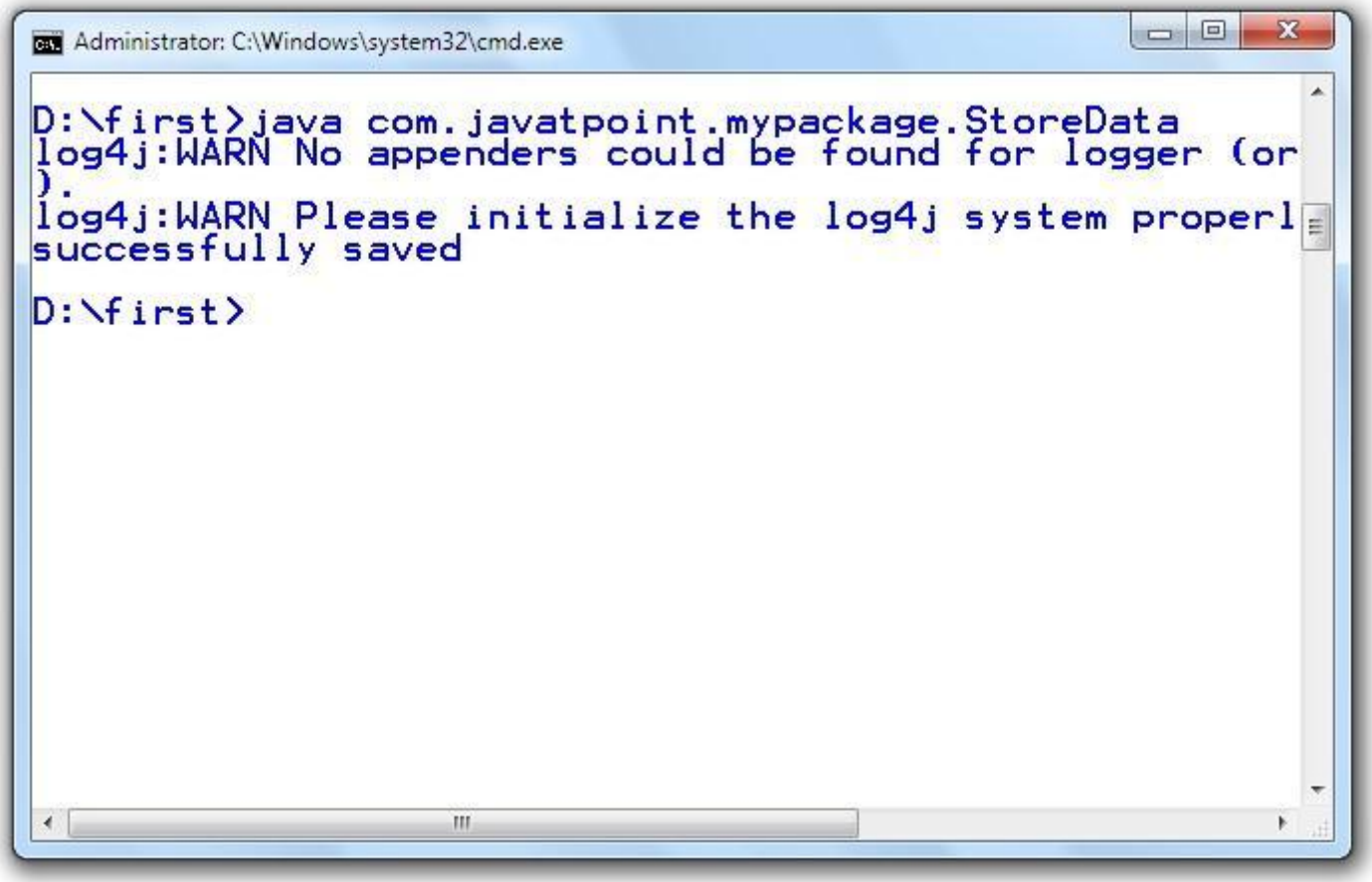
---

6) How to run the first hibernate application without IDE

We may run this hibernate application by IDE (e.g. Eclipse, Myeclipse, Netbeans etc.) or without IDE. We will learn about creating hibernate application in Eclipse IDE in next chapter.

To run the hibernate application without IDE:

- install the oracle10g for this example.
- load the jar files for hibernate. (One of the way to load the jar file is copy all the jar files under the JRE/lib/ext folder). It is better to put these jar files inside the public and private JRE both.
- Now Run the StoreData class by **java com.javatpoint.mypackage.StoreData**



Note: You need to connect with the internet to run this example.

☺ Hibernate In Eclipse

Example to create the Hibernate Application in Eclipse IDE

Here, we are going to create a simple example of hibernate application using eclipse IDE. For creating the first hibernate application in Eclipse IDE, we need to follow following steps:

1. Create the java project
2. Add jar files for hibernate
3. Create the Persistent class
4. Create the mapping file for Persistent class
5. Create the Configuration file
6. Create the class that retrieves or stores the persistent object
7. Run the application

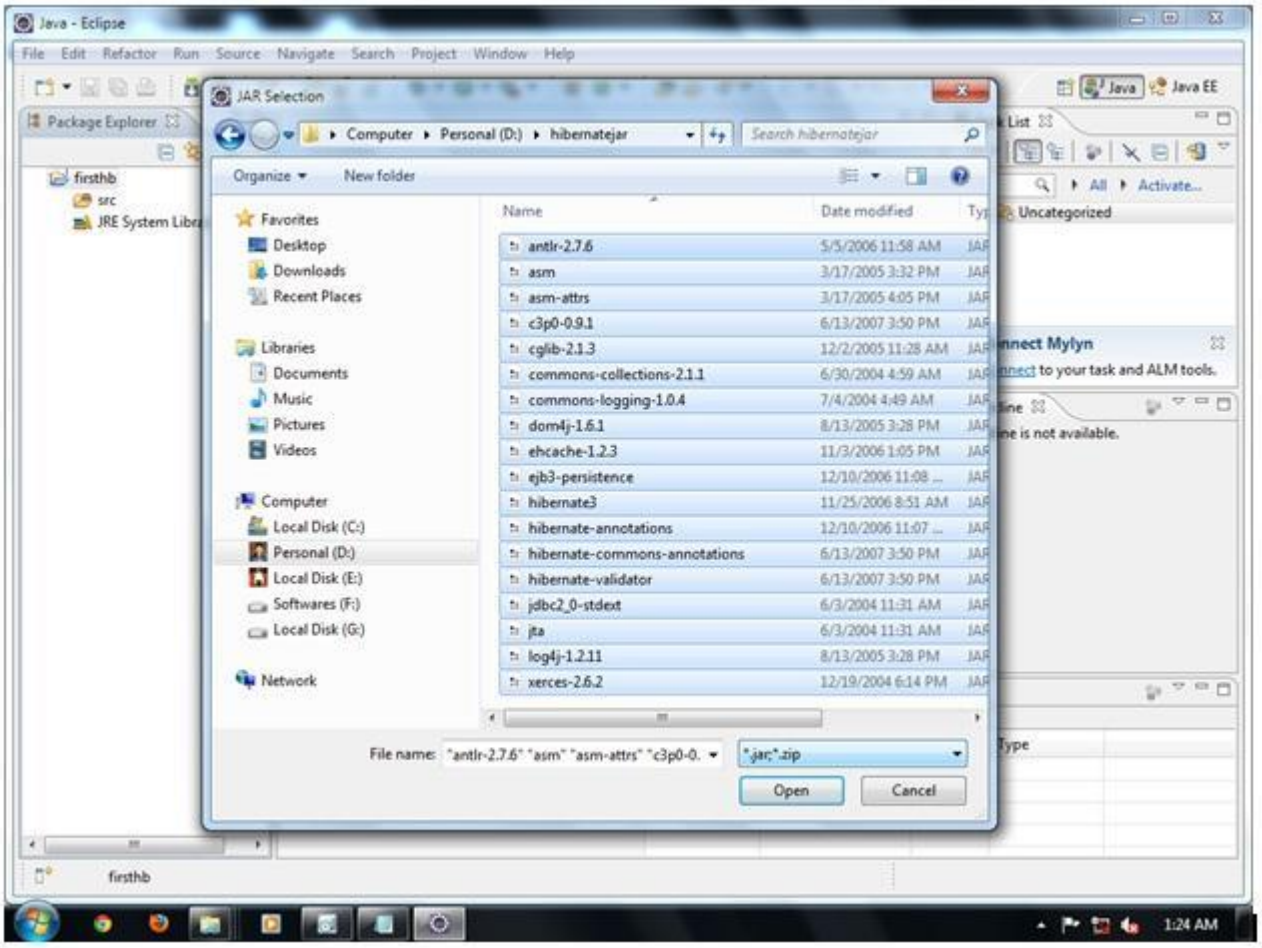
1) Create the java project

Create the java project by **File Menu** - **New** - **project** - **java project** . Now specify the project name e.g. firsthb then **next** - **finish** .

---

2) Add jar files for hibernate

To add the jar files **Right click on your project** - **Build path** - **Add external archives**. Now select all the jar files as shown in the image given below then click open.



In this example, we are connecting the application with oracle database. So you must add the ojdbc14.jar file.

---

3) Create the Persistent class

Here, we are creating the same persistent class which we have created in the previous topic. To create the persistent class, Right click on src - New - Class - specify the class with package name (e.g. com.javatpoint.mypackage) - finish.

Employee.java

```
1. package com.javatpoint.mypackage;
2.
3. public class Employee {
4. private int id;
5. private String firstName,lastName;
6.
7. public int getId() {
8.     return id;
9. }
```

```
10. public void setId(int id) {
11.     this.id = id;
12. }
13. public String getFirstName() {
14.     return firstName;
15. }
16. public void setFirstName(String firstName) {
17.     this.firstName = firstName;
18. }
19. public String getLastName() {
20.     return lastName;
21. }
22. public void setLastName(String lastName) {
23.     this.lastName = lastName;
24. }
25.
26.
27. }
```

---

4) Create the mapping file for Persistent class

Here, we are creating the same mapping file as created in the previous topic. To create the mapping file, Right click on src - new - file - specify the file name (e.g. employee.hbm.xml) - ok. It must be outside the package.

employee.hbm.xml

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-mapping PUBLIC
3.   "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.   "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.   <hibernate-mapping>
7.    <class name="com.smgc.mypackage.Employee" table="emp1000">
8.      <id name="id">
9.       <generator class="assigned"></generator>
10.     </id>
11.
12.     <property name="firstName"></property>
13.     <property name="lastName"></property>
14.
15.   </class>
16.
17.  </hibernate-mapping>
```

---

5) Create the Configuration file

The configuration file contains all the informations for the database such as connection_url, driver_class, username, password etc. The hbm2ddl.auto property is used to create the table in the database automatically. We will have in-depth learning about Dialect class in next topics. To create the configuration file, right click on src - new - file. Now specify the configuration file name e.g. hibernate.cfg.xml.

hibernate.cfg.xml

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">

<hibernate-configuration>

   <session-factory>
      <property name="hbm2ddl.auto">update</property>
      <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
      <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
      <property name="connection.username">system</property>
      <property name="connection.password">oracle</property>
      <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
   <mapping resource="employee.hbm.xml"/>
   </session-factory>

</hibernate-configuration>
```

---

6) Create the class that retrieves or stores the persistent object

In this class, we are simply storing the employee object to the database.

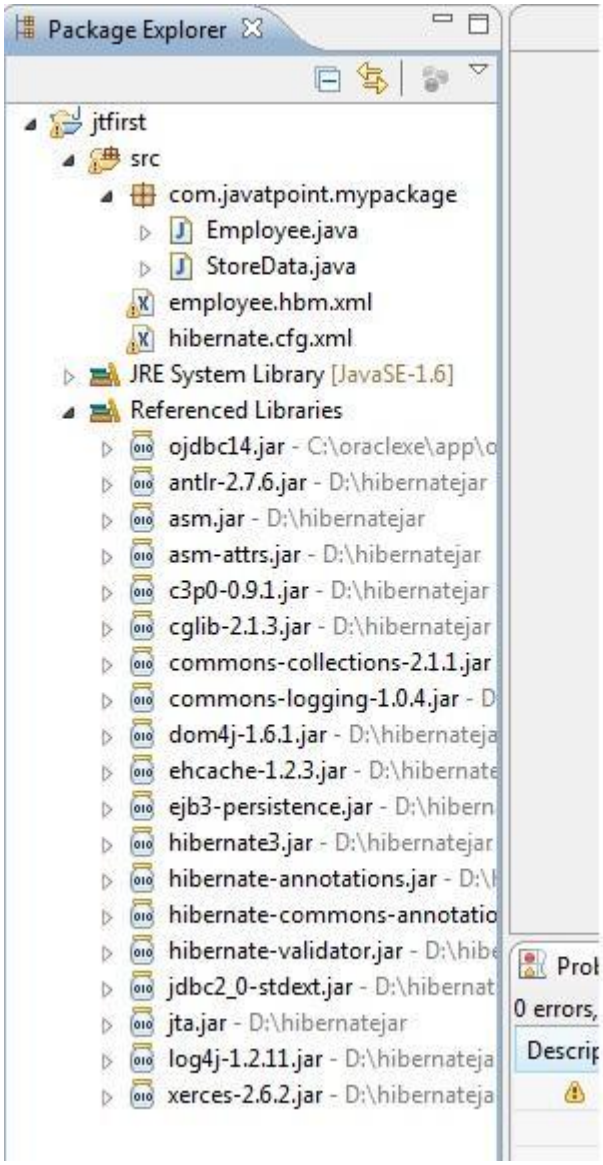
```
package com.javatpoint.mypackage;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

public class StoreData {
public static void main(String[] args) {

   //creating configuration object
   Configuration cfg=new Configuration();
   cfg.configure("hibernate.cfg.xml");//populates the data of the configuration file

   //creating seession factory object
   SessionFactory factory=cfg.buildSessionFactory();
```

```
17.
18.     //creating session object
19.     Session session=factory.openSession();
20.
21.     //creating transaction object
22.     Transaction t=session.beginTransaction();
23.
24.     Employee e1=new Employee();
25.     e1.setId(115);
26.     e1.setFirstName("Piyush");
27.     e1.setLastName("Patel");
28.
29.     session.persist(e1);//persisting the object
30.
31.     t.commit();//transaction is committed
32.     session.close();
33.
34.     System.out.println("successfully saved");
35.
36. }
37. }
```

---

7) Run the application

Before running the application, determine that directory structure is like this.



To run the hibernate application, right click on the StoreData class - Run As - Java Application.

Note: You need to connect with the internet to run this example.

☺ Hibernate In MyEclipse

Example to create the Hibernate Application in MyEclipse

Here, we are going to create a simple example of hibernate application using myeclipse IDE. For creating the first hibernate application in MyEclipse IDE, we need to follow following steps:

1. Create the java project
2. Add hibernate capabilities
3. Create the Persistent class
4. Create the mapping file for Persistent class
5. Add mapping of hbm file in configuration file
6. Create the class that retrieves or stores the persistent object
7. Add jar file for oracle
8. Run the application

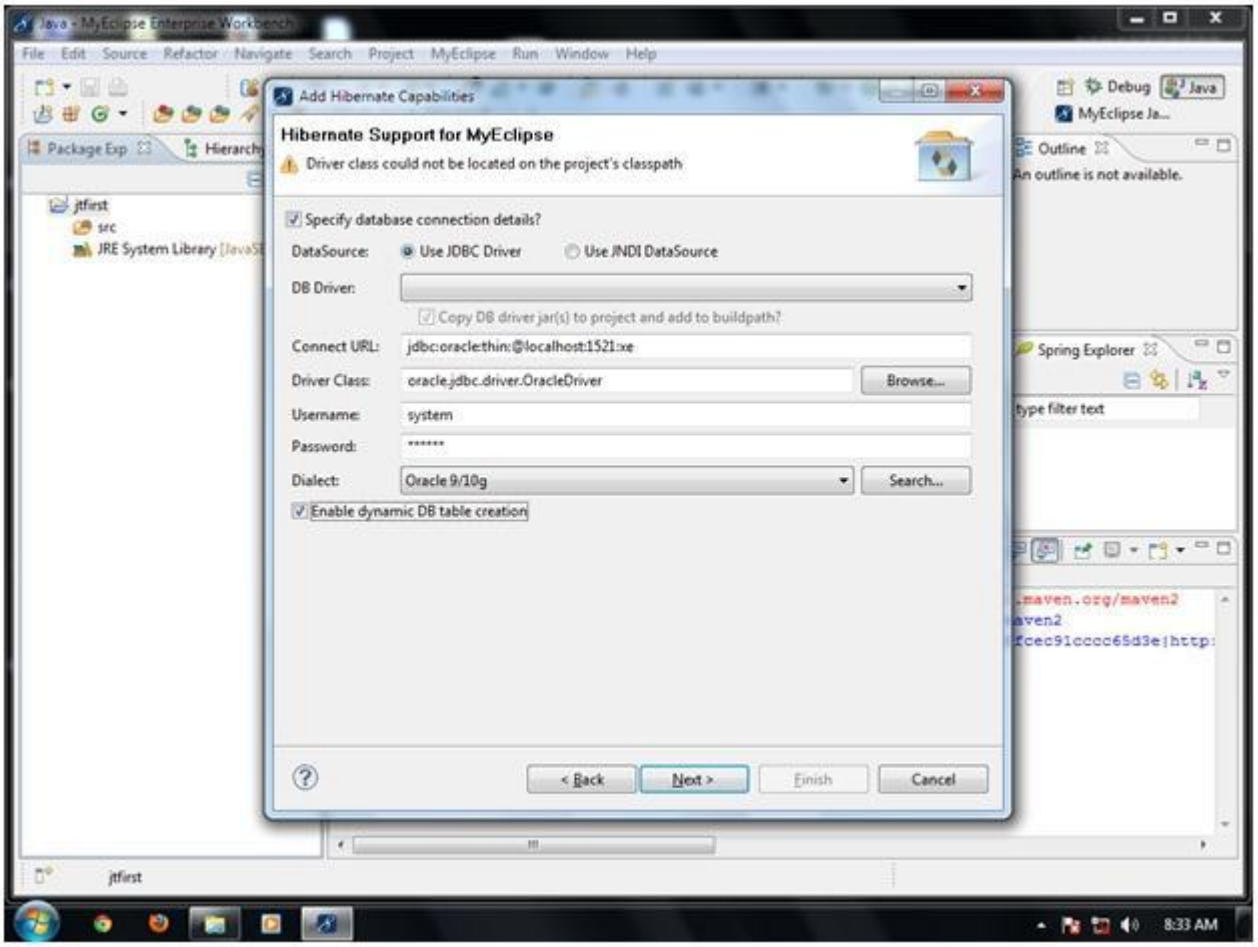
1) Create the java project

Create the java project by **File Menu** - **New** - **project** - **java project** . Now specify the project name e.g. firsthb then **next** - **finish** .

---

2) Add hibernate capabilities

To add the jar files **select your project** - **click on MyEclipse menu** - **Project Capabilities** - **add Hibernate capabilities**- **next**- **next**. Now specify the database connection details as displayed in the figure below.



Here, **Check the Enable dynamic table creation check box** to create automatic table - **next** - **Uncheck the create SessionFactory class** because we are going to write the code to get session object self for better understanding - **finish**.

Now configuration file will be created automatically.

---

3) Create the Persistent class

Here, we are creating the same persistent class which we have created in the previous topic. To create the persistent class, Right click on src - New - Class - specify the class with package name (e.g. com.javatpoint.mypackage) - finish.

Employee.java

```
1. package com.javatpoint.mypackage;
2.
3. public class Employee {
4. private int id;
5. private String firstName,lastName;
```

```
6.
7.  public int getId() {
8.      return id;
9.  }
10. public void setId(int id) {
11.     this.id = id;
12. }
13. public String getFirstName() {
14.     return firstName;
15. }
16. public void setFirstName(String firstName) {
17.     this.firstName = firstName;
18. }
19. public String getLastName() {
20.     return lastName;
21. }
22. public void setLastName(String lastName) {
23.     this.lastName = lastName;
24. }
25.
26.
27. }
```

---

4) Create the mapping file for Persistent class

Here, we are creating the same mapping file as created in the previous topic. To create the mapping file, Right click on src - new - file - specify the file name (e.g. employee.hbm.xml) - ok. It must be outside the package. Copy the dtd for this mapping file from this example after downloading it.

employee.hbm.xml

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-mapping PUBLIC
3.   "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.   "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.   <hibernate-mapping>
7.    <class name="com.smgc.mypackage.Employee" table="emp1000">
8.      <id name="id">
9.       <generator class="assigned"></generator>
10.     </id>
11.
12.     <property name="firstName"></property>
13.     <property name="lastName"></property>
14.
15.   </class>
16.
17.  </hibernate-mapping>
```

5) Add mapping of hbm file in configuration file

open the hibernate.cgf.xml file, and add an entry of mapping resource like this:

```
1.  <mapping resource="employee.hbm.xml"/>
```

Now the configuration file will look like this:

hibernate.cfg.xml

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-configuration PUBLIC
3.          "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.  <hibernate-configuration>
7.
8.     <session-factory>
9.        <property name="hbm2ddl.auto">update</property>
10.       <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
11.       <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
12.       <property name="connection.username">system</property>
13.       <property name="connection.password">oracle</property>
14.       <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
15.    <mapping resource="employee.hbm.xml"/>
16.    </session-factory>
17.
18. </hibernate-configuration>
```

6) Create the class that retrieves or stores the persistent object

In this class, we are simply storing the employee object to the database.

```
1.  package com.javatpoint.mypackage;
2.
3.  import org.hibernate.Session;
4.  import org.hibernate.SessionFactory;
5.  import org.hibernate.Transaction;
6.  import org.hibernate.cfg.Configuration;
7.
8.  public class StoreData {
9.  public static void main(String[] args) {
10.
11.    //creating configuration object
12.    Configuration cfg=new Configuration();
13.    cfg.configure("hibernate.cfg.xml");//populates the data of the configuration file
```

```
14.
15.     //creating seession factory object
16.     SessionFactory factory=cfg.buildSessionFactory();
17.
18.     //creating session object
19.     Session session=factory.openSession();
20.
21.     //creating transaction object
22.     Transaction t=session.beginTransaction();
23.
24.     Employee e1=new Employee();
25.     e1.setId(115);
26.     e1.setFirstName("Piyush");
27.     e1.setLastName("Patel");
28.
29.     session.persist(e1);//persisting the object
30.
31.     t.commit();//transaction is commited
32.     session.close();
33.
34.     System.out.println("successfully saved");
35.
36. }
37. }
```

---

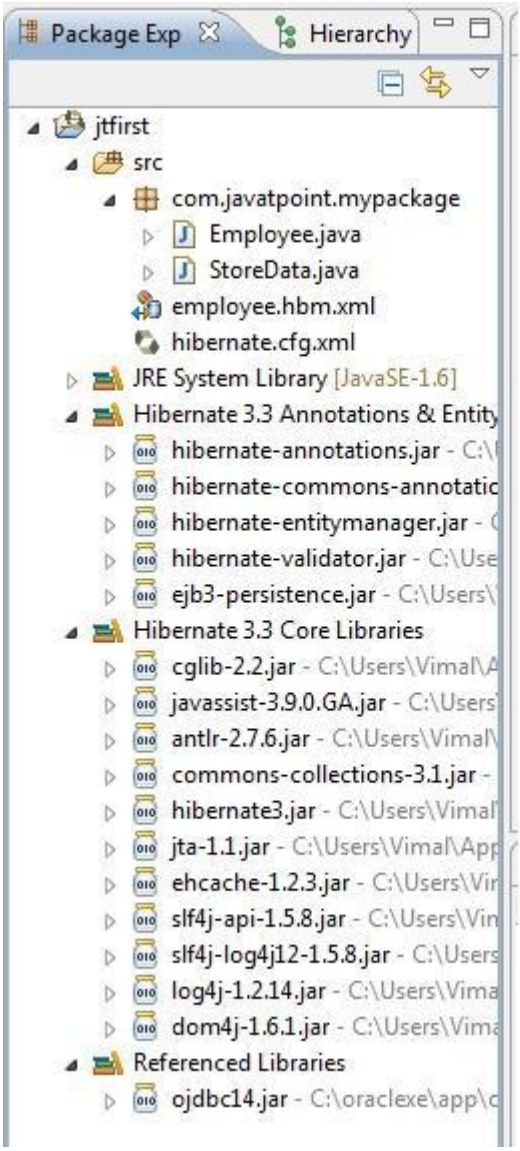
7) Add the jar file for oracle (ojdbc14.jar)

To add the ojdbc14.jar file, right click on your project - build path - add external archives - select the ojdbc14.jar file - open.

---

8) Run the application

Before running the application, determine that directory structure is like this.



To run the hibernate application, right click on the StoreData class - Run As - Java Application.

Chapter-3.3

Hibernate Application And Hibernate Logging

☺ Hibernate with annotation

Hibernate with Annotation

1. Hibernate with Annotation
2. Example to create the hibernate application with Annotation
    1. Add the jar file for annotation
    2. Create the Persistent class
    3. Add mapping of Persistent class in configuration file
    4. Create the class that retrieves or stores the persistent object

The hibernate application can be created with annotation. There are many annotations that can be used to create hibernate application such as @Entity, @Id, @Table etc.

Hibernate Annotations are based on the JPA 2 specification and supports all the features.

All the JPA annotations are defined in the javax.persistence.* package. Hibernate **EntityManager** implements the interfaces and life cycle defined by the JPA specification.

The core advantage of using hibernate annotation is that you don't need to create mapping (hbm) file. Here, hibernate annotations are used to provide the meta data.

Example to create the hibernate application with Annotation

There are 4 steps to create the hibernate application with annotation.

1. **Add the jar file for oracle (if your database is oracle) and annotation**
2. **Create the Persistent class**
3. **Add mapping of Persistent class in configuration file**
4. **Create the class that retrieves or stores the persistent object**

---

1) Add the jar file for oracle and annotation

For oracle you need to add **ojdbc14.jar** file. For using annotation, you need to add:

- **hibernate-commons-annotations.jar**
- **ejb3-persistence.jar**
- **hibernate-annotations.jar**

---

2) Create the Persistent class

Here, we are creating the same persistent class which we have created in the previous topic. But here, we are using annotation.

@Entity annotation marks this class as an entity.

@Table annotation specifies the table name where data of this entity is to be persisted. If you don't use @Table annotation, hibernate will use the class name as the table name bydefault.

@Id annotation marks the identifier for this entity.

@Column annotation specifies the details of the column for this property or field. If @Column annotation is not specified, property name will be used as the column name bydefault.

Employee.java

```
1. package com.smgc;
2. import javax.persistence.Entity;
3. import javax.persistence.Id;
4. import javax.persistence.Table;
5.
6. @Entity
7. @Table(name= "emp500")
8. public class Employee {
9. @Id
10. private int id;
11. private String firstName,lastName;
12.
13. public int getId() {
14.     return id;
15. }
16. public void setId(int id) {
17.     this.id = id;
18. }
19. public String getFirstName() {
20.     return firstName;
21. }
22. public void setFirstName(String firstName) {
23.     this.firstName = firstName;
24. }
25. public String getLastName() {
26.     return lastName;
27. }
28. public void setLastName(String lastName) {
29.     this.lastName = lastName;
30. }
31. }
```

3) Add mapping of Persistent class in configuration file

open the hibernate.cgf.xml file, and add an entry of mapping resource like this:

```
1.      <mapping   class="com.smgc.Employee"/>
```

Now the configuration file will look like this:

hibernate.cfg.xml

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-configuration PUBLIC
3.          "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.  <hibernate-configuration>
7.
8.  <session-factory>
9.   <property name="hbm2ddl.auto">create</property>
10.  <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
11.  <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
12.  <property name="connection.username">system</property>
13.  <property name="connection.password">oracle</property>
14.  <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
15.
16.  <mapping class="com.smgc.Employee"/>
17.  </session-factory>
18.
19. </hibernate-configuration>
```

---

4) Create the class that retrieves or stores the persistent object

In this class, we are simply storing the employee object to the database. Here, we are using the **AnnotationConfiguration** class to get the information of mapping from the persistent class.

```
1.  package com.javatpoint.mypackage;
2.
3.  package com.smgc;
4.
5.  import org.hibernate.*;
6.  import org.hibernate.cfg.*;
7.
8.  public class Test {
9.  public static void main(String[] args) {
10.    Session session=new AnnotationConfiguration()
11.        .configure().buildSessionFactory().openSession();
12.
13.    Transaction t=session.beginTransaction();
```

```
14.
15.     Employee e1=new Employee();
16.     e1.setId(1001);
17.     e1.setFirstName("Piyush");
18.     e1.setLastName("Patel");
19.
20.     Employee e2=new Employee();
21.     e2.setId(1002);
22.     e2.setFirstName("vimal");
23.     e2.setLastName("Patel");
24.
25.     session.persist(e1);
26.     session.persist(e2);
27.
28.     t.commit();
29.     session.close();
30.     System.out.println("successfully saved");
31. }
32. }
```

☺ Hibernate Web Application

Web Application with Hibernate

1. Web Application with Hibernate
2. Example to create web application using hibernate

Here, we are going to create a web application with hibernate. For creating the web application, we are using JSP for presentation logic, Bean class for representing data and DAO class for database codes.

As we create the simple application in hibernate, we don't need to perform any extra operations in hibernate for creating web application. In such case, we are getting the value from the user using the JSP file.

Example to create web application using hibernate

In this example, we are going to insert the record of the user in the database. It is simply a registration form.

---

index.jsp

This page gets input from the user and sends it to the register.jsp file using post method.

```
1. <form action="register.jsp" method="post">
2. Name:<input type="text" name="name"/><br><br/>
3. Password:<input type="password" name="password"/><br><br/>
4. Email ID:<input type="text" name="email"/><br><br/>
5. <input type="submit" value="register"/>"
6.
7. </form>
```

---

register.jsp

This file gets all request parameters and stores this information into an object of User class. Further, it calls the register method of UserDao class passing the User class object.

```
1. <%@page import=“com.smgc.mypack.UserDao"%>
2. <jsp:useBean id="obj" class=“com.smgc.mypack.User">
3. </jsp:useBean>
4. <jsp:setProperty property="*" name="obj"/>
5.
6. <%
7. int i=UserDao.register(obj);
```

```
8.  if(i>0)
9.  out.print("You are successfully registered");
10.
11. %>
```

User.java

It is the simple bean class representing the Persistent class in hibernate.

```
1.  package com.javatpoint.mypack;
2.
3.  public class User {
4.  private int id;
5.  private String name,password,email;
6.
7.  //getters and setters
8.
9.  }
```

---

user.hbm.xml

It maps the User class with the table of the database.

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-mapping PUBLIC
3.   "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.   "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.   <hibernate-mapping>
7.   <class name="com.smgc.mypack.User" table="u400">
8.   <id name="id">
9.   <generator class="increment"></generator>
10.  </id>
11.  <property name="name"></property>
12.  <property name="password"></property>
13.  <property name="email"></property>
14.  </class>
15.
16.  </hibernate-mapping>
```

UserDao.java

A Dao class, containing method to store the instance of User class.

```
1.  package com.javatpoint.mypack;
2.  import org.hibernate.Session;
3.  import org.hibernate.Transaction;
4.  import org.hibernate.cfg.Configuration;
5.
6.  public class UserDao {
7.
8.  public static int register(User u){
9.   int i=0;
10.  Session session=new Configuration().
11.   configure().buildSessionFactory().openSession();
12.
13.   Transaction t=session.beginTransaction();
14.   t.begin();
15.
16.   i=(Integer)session.save(u);
17.
18.   t.commit();
19.   session.close();
20.
21.   return i;
22.  }
23. }
```

---

hibernate.cfg.xml

It is a configuration file, containing informations about the database and mapping file.

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-configuration PUBLIC
3.          "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.  <hibernate-configuration>
6.  <session-factory>
7.   <property name="hbm2ddl.auto">create</property>
8.   <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
9.   <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
10.  <property name="connection.username">system</property>
11.  <property name="connection.password">oracle</property>
12.  <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
13.  <mapping resource="user.hbm.xml"/>
14.  </session-factory>
15. </hibernate-configuration>
```

☺ Hibernate Generator Classes

Generator classes in Hibernate

1. Hibernate Architecture
2. Hibernate Framework
3. Advantages of Hibernate Framework

The <generator> subelement of id used to generate the unique identifier for the objects of persistent class. There are many generator classes defined in the Hibernate Framework.

All the generator classes implements the **org.hibernate.id.IdentifierGenerator interface**. The application programmer may create one's own generator classes by implementing the IdentifierGenerator interface. Hibernate framework provides many built-in generator classes:

1. assigned
2. increment
3. sequence
4. hilo
5. native
6. identity
7. seqhilo
8. uuid
9. guid
10. select
11. foreign
12. sequence-identity

1) assigned

It is the default generator strategy if there is no <generator> element . In this case, application assigns the id. For example:

```
....
 <hibernate-mapping>
  <class ...>
    <id ...>
     <generator class="assigned"></generator>
    </id>

    .....

  </class>
</hibernate-mapping>
```

2) increment

It generates the unique id only if no other process is inserting data into this table. It generates **short**, **int** or **long** type identifier. The first generated identifier is 1 normally and incremented as 1. Syntax:

```
....
<hibernate-mapping>
 <class ...>
   <id ...>
    <generator class="increment"></generator>
   </id>

   .....

 </class>
</hibernate-mapping>
```

---

3) sequence

It uses the sequence of the database. if there is no sequence defined, it creates a sequence automatically e.g. in case of Oracle database, it creates a sequence named HIBERNATE_SEQUENCE. In case of Oracle, DB2, SAP DB, Postgre SQL or McKoi, it uses sequence but it uses generator in interbase. Syntax:

```
.....
<id ...>
 <generator class="sequence"></generator>
</id>
.....
```

For defining your own sequence, use the param subelement of generator.

```
.....
<id ...>
 <generator class="sequence">
    <param name="sequence">your_sequence_name</param>
 </generator>
</id>
.....
```

---

4) hilo

It uses high and low algorithm to generate the id of type short, int and long. Syntax:

```
1. .....
2.  <id ...>
3.   <generator class="hilo"></generator>
4.  </id>
5.  .....
```

---

5) native

It uses identity, sequence or hilo depending on the database vendor. Syntax:

```
1. .....
2.  <id ...>
3.   <generator class="native"></generator>
4.  </id>
5.  .....
```

---

6) identity

It is used in Sybase, My SQL, MS SQL Server, DB2 and HypersonicSQL to support the id column. The returned id is of type short, int or long.

---

7) seqhilo

It uses high and low algorithm on the specified sequence name. The returned id is of type short, int or long.

---

8) uuid

It uses 128-bit UUID algorithm to generate the id. The returned id is of type String, unique within a network (because IP is used). The UUID is represented in hexadecimal digits, 32 in length.

---

9) guid

It uses GUID generated by database of type string. It works on MS SQL Server and MySQL.

10) select

It uses the primary key returned by the database trigger.

11) foreign

It uses the id of another associated object, mostly used with <one-to-one> association.

12) sequence-identity

It uses a special sequence generation strategy. It is supported in Oracle 10g drivers only.

☺ Hibernate Dialects

SQL Dialects in Hibernate

For connecting any hibernate application with the database, you must specify the SQL dialects. There are many Dialects classes defined for RDBMS in the org.hibernate.dialect package. They are as follows:

RDBMS	Dialect
Oracle (any version)	org.hibernate.dialect.OracleDialect
Oracle9i	org.hibernate.dialect.Oracle9iDialect
Oracle10g	org.hibernate.dialect.Oracle10gDialect
MySQL	org.hibernate.dialect.MySQLDialect
MySQL with InnoDB	org.hibernate.dialect.MySQLInnoDBDialect
MySQL with MyISAM	org.hibernate.dialect.MySQLMyISAMDialect
DB2	org.hibernate.dialect.DB2Dialect
DB2 AS/400	org.hibernate.dialect.DB2400Dialect
DB2 OS390	org.hibernate.dialect.DB2390Dialect
Microsoft SQL Server	org.hibernate.dialect.SQLServerDialect
Sybase	org.hibernate.dialect.SybaseDialect
Sybase Anywhere	org.hibernate.dialect.SybaseAnywhereDialect
PostgreSQL	org.hibernate.dialect.PostgreSQLDialect
SAP DB	org.hibernate.dialect.SAPDBDialect
Informix	org.hibernate.dialect.InformixDialect
HypersonicSQL	org.hibernate.dialect.HSQLDialect
Ingres	org.hibernate.dialect.IngresDialect
Progress	org.hibernate.dialect.ProgressDialect
Mckoi SQL	org.hibernate.dialect.MckoiDialect
Interbase	org.hibernate.dialect.InterbaseDialect
Pointbase	org.hibernate.dialect.PointbaseDialect
FrontBase	org.hibernate.dialect.FrontbaseDialect
Firebird	org.hibernate.dialect.FirebirdDialect

☺ Hibernate with Log4j 1

Hibernate Logging by Log4j using xml file

1. Hibernate Logging
2. Example of Hibernate Logging

Logging enables the programmer to write the log details into a file permanently. Log4j and Logback frameworks can be used in hibernate framework to support logging.

There are two ways to perform logging using log4j:

1. By log4j.xml file (or)
2. By log4j.properties file

Steps to perform Hibernate Logging by Log4j using xml file

There are two ways to perform logging using log4j using xml file:

1. Load the log4j jar files with hibernate
2. Create the log4j.xml file inside the src folder (parallel with hibernate.cfg.xml file)

Example of Hibernate Logging by Log4j using xml file

You can enable logging in hibernate by following only two steps in any hibernate example. This is the first example of hibernate application with logging support using log4j.

Load the required jar files

You need to load the slf4j.jar and log4j.jar files with hibernate jar files.

Create log4j.xml file

Now you need to create log4j.xml file. In this example, all the log details will be written in the C:/javatpointlog.log file.

log4j.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE log4j:configuration SYSTEM "log4j.dtd">
<log4j:configuration xmlns:log4j="http://jakarta.apache.org/log4j/"
    debug="false">
<appender name="CONSOLE" class="org.apache.log4j.ConsoleAppender">
 <layout class="org.apache.log4j.PatternLayout">
  <param name="ConversionPattern" value="[%d{dd/MM/yy hh:mm:ss:sss z}] %5p %c{2}: %m%n" />
 </layout>
</appender>
   <appender name="ASYNC" class="org.apache.log4j.AsyncAppender">
      <appender-ref ref="CONSOLE" />
      <appender-ref ref="FILE" />
</appender>
<appender name="FILE" class="org.apache.log4j.RollingFileAppender">
   <param name="File" value="C:/javatpointlog.log" />
   <param name="MaxBackupIndex" value="100" />
 <layout class="org.apache.log4j.PatternLayout">
  <param name="ConversionPattern" value="[%d{dd/MM/yy hh:mm:ss:sss z}] %5p %c{2}: %m%n" />
</layout>
</appender>
   <category name="org.hibernate">
      <priority value="DEBUG" />
   </category>
   <category name="java.sql">
      <priority value="debug" />
   </category>
   <root>
      <priority value="INFO" />
      <appender-ref ref="FILE" />
   </root>
</log4j:configuration>
```

☺ Hibernate with Log4j 2

Hibernate Logging by Log4j using properties file

1. Hibernate Logging
2. Example of Hibernate Logging

As we know, Log4j and Logback frameworks are used to support logging in hibernate, there are two ways to perform logging using log4j:

1. By log4j.xml file (or)
2. By log4j.properties file

Here, we are going to enable logging using log4j through properties file.

Steps to perform Hibernate Logging by Log4j using properties file

There are two ways to perform logging using log4j using properties file:

1. Load the log4j jar files with hibernate
2. Create the log4j.properties file inside the src folder (parallel with hibernate.cfg.xml file)

Example of Hibernate Logging by Log4j using properties file

You can enable logging in hibernate by following only two steps in any hibernate example. This is the first example of hibernate application with logging support using log4j.

Load the required jar files

You need to load the slf4j.jar and log4j.jar files with hibernate jar files.

Create log4j.properties file

Now you need to create log4j.properties file. In this example, all the log details will be written in the C:\\javatpointhibernate.log file.

log4j.properties

```
# Direct log messages to a log file
log4j.appender.file=org.apache.log4j.RollingFileAppender
log4j.appender.file.File=C:\\javatpointhibernate.log
log4j.appender.file.MaxFileSize=1MB
log4j.appender.file.MaxBackupIndex=1
log4j.appender.file.layout=org.apache.log4j.PatternLayout
log4j.appender.file.layout.ConversionPattern=%d{ABSOLUTE} %5p %c{1}:%L - %m%n

# Direct log messages to stdout
log4j.appender.stdout=org.apache.log4j.ConsoleAppender
log4j.appender.stdout.Target=System.out
log4j.appender.stdout.layout=org.apache.log4j.PatternLayout
log4j.appender.stdout.layout.ConversionPattern=%d{ABSOLUTE} %5p %c{1}:%L - %m%n

# Root logger option
log4j.rootLogger=INFO, file, stdout

# Log everything. Good for troubleshooting
log4j.logger.org.hibernate=INFO

# Log all JDBC parameters
log4j.logger.org.hibernate.type=ALL
```

Chapter-4.1

Inheritance Mapping

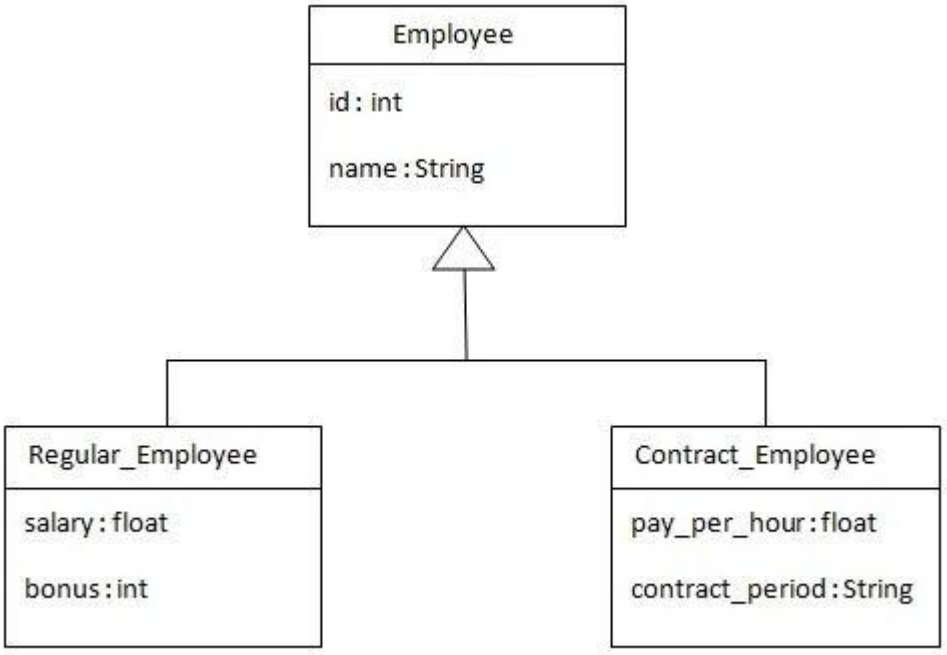
☺ Table Per Hierarchy

Hibernate Table Per Hierarchy using xml file

1. Table Per Hierarchy
2. Example of Table Per Hierarchy

By this inheritance strategy, we can map the whole hierarchy by single table only. Here, an extra column (also known as **discriminator column**) is created in the table to identify the class.

Let's understand the problem first. I want to map the whole hierarchy given below into one table of the database.



There are three classes in this hierarchy. Employee is the super class for Regular_Employee and Contract_Employee classes. Let's see the mapping file for this hierarchy.

```
1. <?xml version='1.0' encoding='UTF-8'?>
2. <!DOCTYPE hibernate-mapping PUBLIC
3.         "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.         "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6. <hibernate-mapping>
7. <class name="com.smgc.mypackage.Employee" table="emp121" discriminator-value="emp">
8. <id name="id">
```

```
9.  <generator class="increment"></generator>
10. </id>
11.
12. <discriminator column="type" type="string"></discriminator>
13. <property name="name"></property>
14.
15. <subclass name="com.smgc.mypackage.Regular_Employee" discriminator-value="reg_emp">
16. <property name="salary"></property>
17. <property name="bonus"></property>
18. </subclass>
19.
20. <subclass name="com.smgc.mypackage.Contract_Employee" discriminator-value="con_emp">
21. <property name="pay_per_hour"></property>
22. <property name="contract_duration"></property>
23. </subclass>
24.
25. </class>
26.
27. </hibernate-mapping>
```

In case of table per class hierarchy an discriminator column is added by the hibernate framework that specifies the type of the record. It is mainly used to distinguish the record. To specify this, **discriminator** subelement of class must be specified.

The **subclass** subelement of class, specifies the subclass. In this case, Regular_Employee and Contract_Employee are the subclasses of Employee class.

The table structure for this hierarchy is as shown below:

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
TYPE	VARCHAR2(255)	No	-	-
NAME	VARCHAR2(255)	Yes	-	-
SALARY	FLOAT	Yes	-	-
BONUS	NUMBER(10,0)	Yes	-	-
PAY_PER_HOUR	FLOAT	Yes	-	-
CONTRACT_DURATION	VARCHAR2(255)	Yes	-	-
				1 - 7

Example of Table per class hierarchy

In this example we are creating the three classes and provide mapping of these classes in the employee.hbm.xml file.

1) Create the Persistent classes

You need to create the persistent classes representing the inheritance. Let's create the three classes for the above hierarchy:

File: Employee.java

```
1. package com.javatpoint.mypackage;
2.
3. public class Employee {
4. private int id;
5. private String name;
6.
7. //getters and setters
8. }
```

File: Regular_Employee.java

```
1. package com.javatpoint.mypackage;
2.
3. public class Regular_Employee extends Employee{
4. private float salary;
5. private int bonus;
6.
7. //getters and setters
8. }
```

File: Contract_Employee.java

```
1. package com.javatpoint.mypackage;
2.
3. public class Contract_Employee extends Employee{
4.     private float pay_per_hour;
5.     private String contract_duration;
6.
7. //getters and setters
8. }
```

---

2) Create the mapping file for Persistent class

The mapping has been discussed above for the hierarchy.

File: employee.hbm.xml

```
1. <?xml version='1.0' encoding='UTF-8'?>
2. <!DOCTYPE hibernate-mapping PUBLIC
3.         "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.         "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
```

```
5.
6.  <hibernate-mapping>
7.  <class name="com.smgc.mypackage.Employee" table="emp121" discriminator-value="emp">
8.  <id name="id">
9.  <generator class="increment"></generator>
10. </id>
11.
12. <discriminator column="type" type="string"></discriminator>
13. <property name="name"></property>
14.
15. <subclass name="com.smgc.mypackage.Regular_Employee" discriminator-value="reg_emp">
16. <property name="salary"></property>
17. <property name="bonus"></property>
18. </subclass>
19.
20. <subclass name="com.smgc.mypackage.Contract_Employee" discriminator-value="con_emp">
21. <property name="pay_per_hour"></property>
22. <property name="contract_duration"></property>
23. </subclass>
24.
25. </class>
26.
27. </hibernate-mapping>
```

---

3) Add mapping of hbm file in configuration file

Open the hibernate.cgf.xml file, and add an entry of mapping resource like this:

```
1.  <mapping resource="employee.hbm.xml"/>
```

Now the configuration file will look like this:

File: hibernate.cfg.xml

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-configuration PUBLIC
3.          "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.  <hibernate-configuration>
7.
8.     <session-factory>
9.        <property name="hbm2ddl.auto">update</property>
10.       <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
11.       <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
12.       <property name="connection.username">system</property>
13.       <property name="connection.password">oracle</property>
14.       <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
15.    <mapping resource="employee.hbm.xml"/>
```

```
16.     </session-factory>
17.
18. </hibernate-configuration>
```

The hbm2ddl.auto property is defined for creating automatic table in the database.

---

4) Create the class that stores the persistent object

In this class, we are simply storing the employee objects in the database.

File: StoreData.java

```
1.  package com.javatpoint.mypackage;
2.  import org.hibernate.*;
3.  import org.hibernate.cfg.*;
4.
5.  public class StoreData {
6.  public static void main(String[] args) {
7.      Session session=new Configuration().configure("hibernate.cfg.xml")
8.                      .buildSessionFactory().openSession();
9.
10.     Transaction t=session.beginTransaction();
11.
12.     Employee e1=new Employee();
13.     e1.setName("Piyush");
14.
15.     Regular_Employee e2=new Regular_Employee();
16.     e2.setName("Vivek Kumar");
17.     e2.setSalary(50000);
18.     e2.setBonus(5);
19.
20.     Contract_Employee e3=new Contract_Employee();
21.     e3.setName("Arjun Kumar");
22.     e3.setPay_per_hour(1000);
23.     e3.setContract_duration("15 hours");
24.
25.     session.persist(e1);
26.     session.persist(e2);
27.     session.persist(e3);
28.
29.     t.commit();
30.     session.close();
31.     System.out.println("success");
32. }
33. }
```

Output:

EDIT	ID	TYPE	NAME	SALARY	BONUS	PAY_PER_HOUR	CONTRACT_DURATION
	1	emp	sonoo	-	-	-	-
	2	reg_emp	Vivek Kumar	50000	5	-	-
	3	con_emp	Arjun Kumar	-	-	1000	15 hours
							row(s) 1 - 3 of 3

☺ Table Per Hierarchy using Annotation

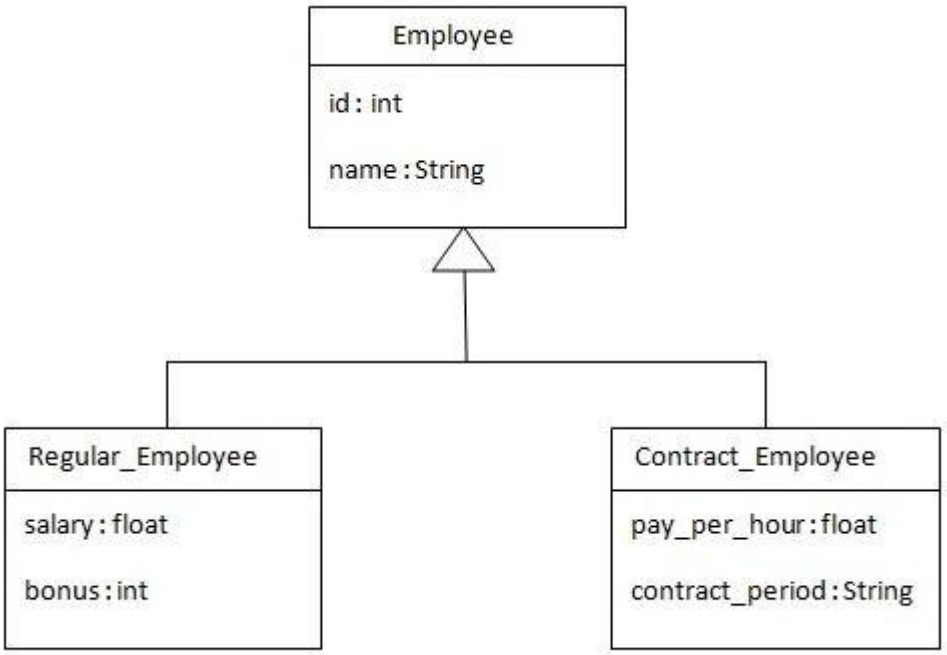
Hibernate Table Per Hierarchy using Annotation

1. Table Per Hierarchy
2. Example of Table Per Hierarchy

In the previous page, we have mapped the inheritance hierarchy with one table only using xml file. Here, we are going to perform this task using annotation. You need to use @Inheritance(strategy=InheritanceType.SINGLE_TABLE), @DiscriminatorColumn and @DiscriminatorValue annotations for mapping table per hierarchy strategy.

In case of table per hierarchy, only one table is required to map the inheritance hierarchy. Here, an extra column (also known as **discriminator column**) is created in the table to identify the class.

Let's see the inheritance hierarchy:



There are three classes in this hierarchy. Employee is the super class for Regular_Employee and Contract_Employee classes.

The table structure for this hierarchy is as shown below:

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
TYPE	VARCHAR2(255)	No	-	-
NAME	VARCHAR2(255)	Yes	-	-
SALARY	FLOAT	Yes	-	-
BONUS	NUMBER(10,0)	Yes	-	-
PAY_PER_HOUR	FLOAT	Yes	-	-
CONTRACT_DURATION	VARCHAR2(255)	Yes	-	-
				1 - 7

Example of Hibernate Table Per Hierarchy using Annotation

You need to follow following steps to create simple example:

- Create the persistent classes
- Create the configuration file
- Create the class to store the fetch the data

1) Create the Persistent classes

You need to create the persistent classes representing the inheritance. Let's create the three classes for the above hierarchy:

File: Employee.java

```
1. package com.javatpoint.mypackage;
2. import javax.persistence.*;
3.
4. @Entity
5. @Table(name = "employee101")
6. @Inheritance(strategy=InheritanceType.SINGLE_TABLE)
7. @DiscriminatorColumn(name="type",discriminatorType=DiscriminatorType.STRING)
8. @DiscriminatorValue(value="employee")
9.
10. public class Employee {
11. @Id
12. @GeneratedValue(strategy=GenerationType.AUTO)
13.
14. @Column(name = "id")
15. private int id;
16.
17. @Column(name = "name")
18. private String name;
19. //setters and getters
20. }
```

File: Regular_Employee.java

```
1. package com.javatpoint.mypackage;
2.
3. import javax.persistence.*;
4.
5. @Entity
6. @DiscriminatorValue("regularemployee")
7. public class Regular_Employee extends Employee{
8.
9. @Column(name="salary")
10. private float salary;
11.
12. @Column(name="bonus")
13. private int bonus;
14.
15. //setters and getters
16. }
```

File: Contract_Employee.java

```
1. package com.javatpoint.mypackage;
2.
3. import javax.persistence.Column;
4. import javax.persistence.DiscriminatorValue;
5. import javax.persistence.Entity;
6.
7. @Entity
8. @DiscriminatorValue("contractemployee")
9. public class Contract_Employee extends Employee{
10.
11.    @Column(name="pay_per_hour")
12.    private float pay_per_hour;
13.
14.    @Column(name="contract_duration")
15.    private String contract_duration;
16.
17.    //setters and getters
18. }
```

---

2) Add the persistent classes in configuration file

Open the hibernate.cgf.xml file, and add entries of entity classes like this:

```
1. <mapping class="com.smgc.mypackage.Employee"/>
2.  <mapping class="com.smgc.mypackage.Contract_Employee"/>
3.  <mapping class="com.smgc.mypackage.Regular_Employee"/>
4. </pre></div>
```

```
<table >
<tr><td>Now the configuration file will look like this:
</td></tr>
</table>
<span id="filename">File: hibernate.cfg.xml</span>
<div class="codeblock"><pre name="code" class="java" >
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">

<!-- Generated by MyEclipse Hibernate Tools.                   -->
<hibernate-configuration>

    <session-factory>
        <property name="hbm2ddl.auto">update</property>
        <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
        <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
        <property name="connection.username">system</property>
        <property name="connection.password">oracle</property>
        <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>

        <mapping class="com.smgc.mypackage.Employee"/>
        <mapping class="com.smgc.mypackage.Contract_Employee"/>
        <mapping class="com.smgc.mypackage.Regular_Employee"/>
    </session-factory>

</hibernate-configuration>
```

The hbm2ddl.auto property is defined for creating automatic table in the database.

---

3) Create the class that stores the persistent object

In this class, we are simply storing the employee objects in the database.

File: StoreTest.java

```
package com.javatpoint.mypackage;

import org.hibernate.*;
import org.hibernate.cfg.*;

public class StoreData {
public static void main(String[] args) {
    AnnotationConfiguration cfg=new AnnotationConfiguration();
    Session session=cfg.configure("hibernate.cfg.xml").buildSessionFactory().openSession();

    Transaction t=session.beginTransaction();

```

```
13.     Employee e1=new Employee();
14.     e1.setName("Piyush");
15.
16.     Regular_Employee e2=new Regular_Employee();
17.     e2.setName("Vivek Kumar");
18.     e2.setSalary(50000);
19.     e2.setBonus(5);
20.
21.     Contract_Employee e3=new Contract_Employee();
22.     e3.setName("Arjun Kumar");
23.     e3.setPay_per_hour(1000);
24.     e3.setContract_duration("15 hours");
25.
26.     session.persist(e1);
27.     session.persist(e2);
28.     session.persist(e3);
29.
30.     t.commit();
31.     session.close();
32.     System.out.println("success");
33. }
34. }
```

Output:

EDIT	ID	TYPE	NAME	SALARY	BONUS	PAY_PER_HOUR	CONTRACT_DURATION
	1	emp	sonoo	-	-	-	-
	2	reg_emp	Vivek Kumar	50000	5	-	-
	3	con_emp	Arjun Kumar	-	-	1000	15 hours
							row(s) 1 - 3 of 3

☺ Table Per Concrete

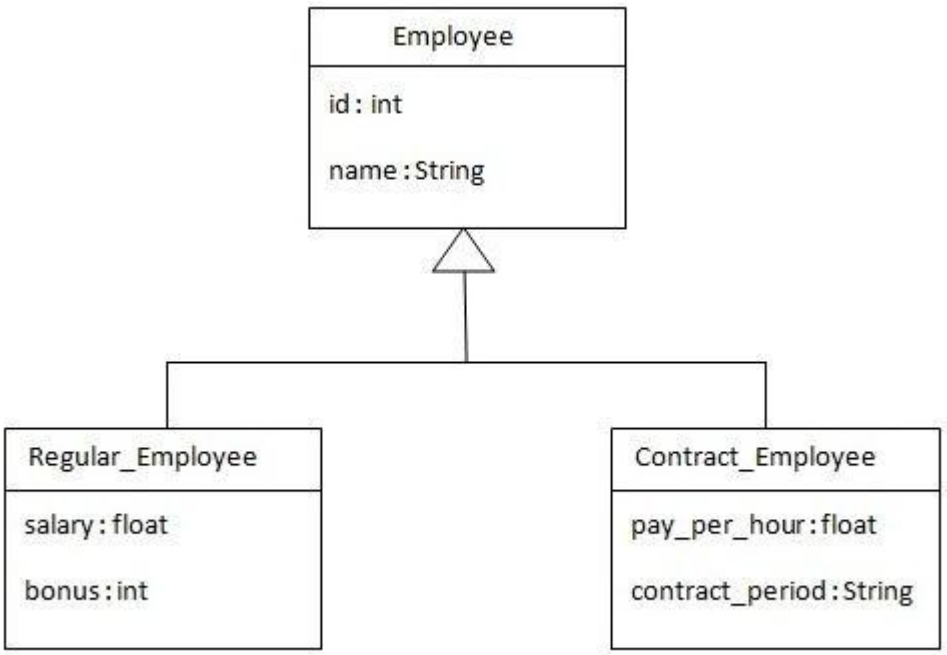
Table Per Concrete class using xml file

1. Table Per Concrete class
2. Example of Table Per Concrete class

In case of Table Per Concrete class, there will be three tables in the database having no relations to each other. There are two ways to map the table with table per concrete class strategy.

- By union-subclass element
- By Self creating the table for each class

Let's understand what hierarchy we are going to map.



Let's see how can we map this hierarchy by union-subclass element:

```
1. <?xml version='1.0' encoding='UTF-8'?>
2. <!DOCTYPE hibernate-mapping PUBLIC
3.         "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.         "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.  <hibernate-mapping>
7.  <class name="com.smgc.mypackage.Employee" table="emp122">
8.  <id name="id">
9.  <generator class="increment"></generator>
10. </id>
11.
12. <property name="name"></property>
```

```
13.
14.  <union-subclass name="com.smgc.mypackage.Regular_Employee" table="regemp122">
15.  <property name="salary"></property>
16.  <property name="bonus"></property>
17.  </union-subclass>
18.
19.  <union-
     subclass name="com.smgc.mypackage.Contract_Employee" table="contemp122">
20.  <property name="pay_per_hour"></property>
21.  <property name="contract_duration"></property>
22.  </union-subclass>
23.
24.  </class>
25.
26.  </hibernate-mapping>
```

27. In case of table per concrete class, there will be three tables in the database, each representing a particular class.

The **union-subclass** subelement of class, specifies the subclass. It adds the columns of parent table into this table. In other words, it is working as a union.

The table structure for each table will be as follows:

Table structure for Employee class

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
NAME	VARCHAR2(255)	Yes	-	-
				1 - 2

Table structure for Regular_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
NAME	VARCHAR2(255)	Yes	-	-
SALARY	FLOAT	Yes	-	-
BONUS	NUMBER(10,0)	Yes	-	-
				1 - 4

Table structure for Contract_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
NAME	VARCHAR2(255)	Yes	-	-
PAY_PER_HOUR	FLOAT	Yes	-	-
CONTRACT_DURATION	VARCHAR2(255)	Yes	-	-
				1 - 4

Example of Table per concrete class

In this example we are creating the three classes and provide mapping of these classes in the employee.hbm.xml file.

1) Create the Persistent classes

You need to create the persistent classes representing the inheritance. Let's create the three classes for the above hierarchy:

File: Employee.java

```
1. package com.javatpoint.mypackage;
2.
3. public class Employee {
4. private int id;
5. private String name;
6.
7. //getters and setters
8. }
```

File: Regular_Employee.java

```
1. package com.javatpoint.mypackage;
2.
3. public class Regular_Employee extends Employee{
4. private float salary;
5. private int bonus;
6.
7. //getters and setters
8. }
```

File: Contract_Employee.java

```
1. package com.javatpoint.mypackage;
2.
3. public class Contract_Employee extends Employee{
4.     private float pay_per_hour;
5.     private String contract_duration;
6.
7. //getters and setters
8. }
```

---

2) Create the mapping file for Persistent class

The mapping has been discussed above for the hierarchy.

File: employee.hbm.xml

```
1. <?xml version='1.0' encoding='UTF-8'?>
2. <!DOCTYPE hibernate-mapping PUBLIC
3.         "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.         "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.
7.  <hibernate-mapping>
8.  <class name="com.smgc.mypackage.Employee" table="emp122">
9.  <id name="id">
10. <generator class="increment"></generator>
11. </id>
12.
13. <property name="name"></property>
14.
15. <union-subclass name="com.smgc.mypackage.Regular_Employee" table="regemp122">
16. <property name="salary"></property>
17. <property name="bonus"></property>
18. </union-subclass>
19.
20. <union-
    subclass name="com.smgc.mypackage.Contract_Employee" table="contemp122">
21. <property name="pay_per_hour"></property>
22. <property name="contract_duration"></property>
23. </union-subclass>
24.
25. </class>
26.
27. </hibernate-mapping>
```

---

3) Add mapping of hbm file in configuration file

Open the hibernate.cgf.xml file, and add an entry of mapping resource like this:

```
1.  <mapping resource="employee.hbm.xml"/>
```

Now the configuration file will look like this:

File: hibernate.cfg.xml

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-configuration PUBLIC
3.          "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.  <hibernate-configuration>
7.
8.     <session-factory>
9.        <property name="hbm2ddl.auto">update</property>
10.       <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
11.       <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
12.       <property name="connection.username">system</property>
13.       <property name="connection.password">oracle</property>
14.       <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
15.    <mapping resource="employee.hbm.xml"/>
16.    </session-factory>
17.
18. </hibernate-configuration>
```

The hbm2ddl.auto property is defined for creating automatic table in the database.

---

4) Create the class that stores the persistent object

In this class, we are simply storing the employee objects in the database.

File: StoreData.java

```
1.  package com.javatpoint.mypackage;
2.
3.  import org.hibernate.*;
4.  import org.hibernate.cfg.*;
5.
6.  public class StoreData {
7.  public static void main(String[] args) {
8.     Session session=new Configuration().configure("hibernate.cfg.xml")
9.                    .buildSessionFactory().openSession();
10.
11.    Transaction t=session.beginTransaction();
```

```
12.
13.     Employee e1=new Employee();
14.     e1.setName("Piyush");
15.
16.     Regular_Employee e2=new Regular_Employee();
17.     e2.setName("Vivek Kumar");
18.     e2.setSalary(50000);
19.     e2.setBonus(5);
20.
21.     Contract_Employee e3=new Contract_Employee();
22.     e3.setName("Arjun Kumar");
23.     e3.setPay_per_hour(1000);
24.     e3.setContract_duration("15 hours");
25.
26.     session.persist(e1);
27.     session.persist(e2);
28.     session.persist(e3);
29.
30.     t.commit();
31.     session.close();
32.     System.out.println("success");
33. }
34. }
```

☺ Table Per Concrete using Annotation

Table Per Concrete class using Annotation

1. Table Per Concrete class using annotation

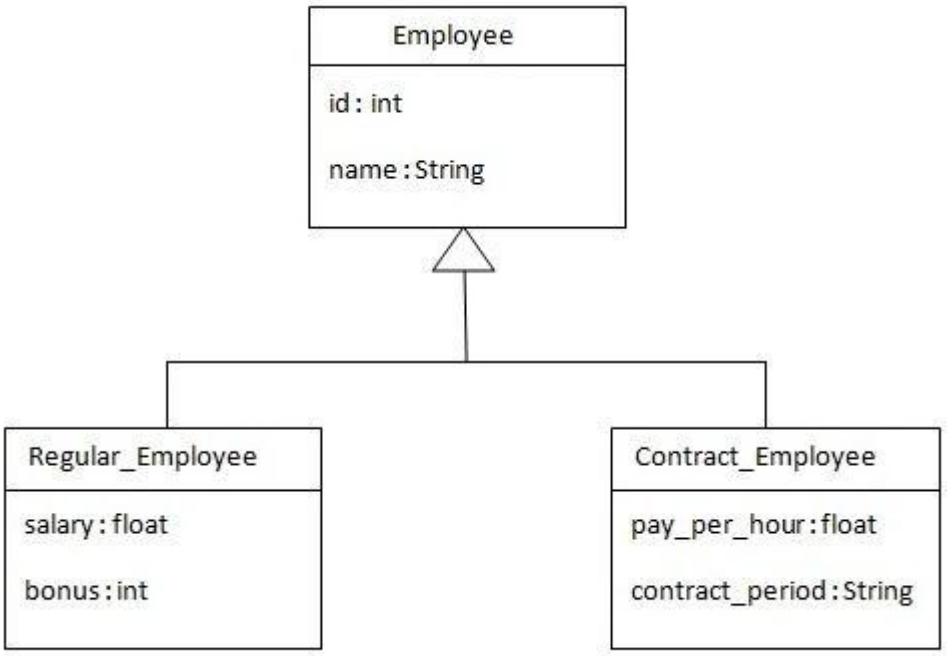
In case of Table Per Concrete class, tables are created per class. So there are no nullable values in the table. Disadvantage of this approach is that duplicate columns are created in the subclass tables.

Here, we need to use @Inheritance(strategy = InheritanceType.TABLE_PER_CLASS) annotation in the parent class and @AttributeOverrides annotation in the subclasses.

@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS) specifies that we are using table per concrete class strategy. It should be specified in the parent class only.

@AttributeOverrides defines that parent class attributes will be overriden in this class. In table structure, parent class table columns will be added in the subclass table.

The class hierarchy is given below:



The table structure for each table will be as follows:

Table structure for Employee class

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
NAME	VARCHAR2(255)	Yes	-	-
				1 - 2

Table structure for Regular_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
NAME	VARCHAR2(255)	Yes	-	-
SALARY	FLOAT	Yes	-	-
BONUS	NUMBER(10,0)	Yes	-	-
				1 - 4

Table structure for Contract_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
NAME	VARCHAR2(255)	Yes	-	-
PAY_PER_HOUR	FLOAT	Yes	-	-
CONTRACT_DURATION	VARCHAR2(255)	Yes	-	-
				1 - 4

Example of Table per concrete class

In this example we are creating the three classes and provide mapping of these classes in the employee.hbm.xml file.

1) Create the Persistent classes

You need to create the persistent classes representing the inheritance. Let's create the three classes for the above hierarchy:

File: Employee.java

```
1. package com.javatpoint.mypackage;
2. import javax.persistence.*;
```

```
3.
4.  @Entity
5.  @Table(name = "employee102")
6.  @Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
7.
8.  public class Employee {
9.  @Id
10. @GeneratedValue(strategy=GenerationType.AUTO)
11.
12. @Column(name = "id")
13. private int id;
14.
15. @Column(name = "name")
16. private String name;
17.
18. //setters and getters
19. }
```

File: Regular_Employee.java

```
1.  package com.javatpoint.mypackage;
2.  import javax.persistence.*;
3.
4.  @Entity
5.  @Table(name="regularemployee102")
6.  @AttributeOverrides({
7.      @AttributeOverride(name="id", column=@Column(name="id")),
8.      @AttributeOverride(name="name", column=@Column(name="name"))
9.  })
10. public class Regular_Employee extends Employee{
11.
12. @Column(name="salary")
13. private float salary;
14.
15. @Column(name="bonus")
16. private int bonus;
17.
18. //setters and getters
19. }
```

File: Contract_Employee.java

```
1.  package com.javatpoint.mypackage;
2.  import javax.persistence.*;
3.  @Entity
4.  @Table(name="contractemployee102")
5.  @AttributeOverrides({
6.      @AttributeOverride(name="id", column=@Column(name="id")),
7.      @AttributeOverride(name="name", column=@Column(name="name"))
8.  })
```

```
9.  public class Contract_Employee extends Employee{
10.
11.    @Column(name="pay_per_hour")
12.    private float pay_per_hour;
13.
14.    @Column(name="contract_duration")
15.    private String contract_duration;
16.
17.    public float getPay_per_hour() {
18.        return pay_per_hour;
19.    }
20.    public void setPay_per_hour(float payPerHour) {
21.        pay_per_hour = payPerHour;
22.    }
23.    public String getContract_duration() {
24.        return contract_duration;
25.    }
26.    public void setContract_duration(String contractDuration) {
27.        contract_duration = contractDuration;
28.    }
29. }
```

---

2) Add mapping of hbm file in configuration file

File: hibernate.cfg.xml

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-configuration PUBLIC
3.            "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.            "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.  <!-- Generated by MyEclipse Hibernate Tools.                 -->
6.  <hibernate-configuration>
7.      <session-factory>
8.          <property name="hbm2ddl.auto">update</property>
9.          <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
10.         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
11.         <property name="connection.username">system</property>
12.         <property name="connection.password">oracle</property>
13. <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
14.         <mapping class="com.smgc.mypackage.Employee"/>
15.         <mapping class="com.smgc.mypackage.Contract_Employee"/>
16.         <mapping class="com.smgc.mypackage.Regular_Employee"/>
17.     </session-factory>
18. </hibernate-configuration>
```

The hbm2ddl.auto property is defined for creating automatic table in the database.

---

3) Create the class that stores the persistent object

In this class, we are simply storing the employee objects in the database.

File: StoreData.java

```
1. package com.javatpoint.mypackage;
2.
3. import org.hibernate.*;
4. import org.hibernate.cfg.*;
5.
6. public class StoreData {
7. public static void main(String[] args) {
8. AnnotationConfiguration cfg=new AnnotationConfiguration();
9. Session session=cfg.configure("hibernate.cfg.xml").buildSessionFactory().openSession();
10.
11.    Transaction t=session.beginTransaction();
12.
13.    Employee e1=new Employee();
14.    e1.setName("Piyush");
15.
16.    Regular_Employee e2=new Regular_Employee();
17.    e2.setName("Vivek Kumar");
18.    e2.setSalary(50000);
19.    e2.setBonus(5);
20.
21.    Contract_Employee e3=new Contract_Employee();
22.    e3.setName("Arjun Kumar");
23.    e3.setPay_per_hour(1000);
24.    e3.setContract_duration("15 hours");
25.
26.    session.persist(e1);
27.    session.persist(e2);
28.    session.persist(e3);
29.
30.    t.commit();
31.    session.close();
32.    System.out.println("success");
33. }
34. }
```

☺ Table Per Subclass

Table Per Subclass Example using xml file

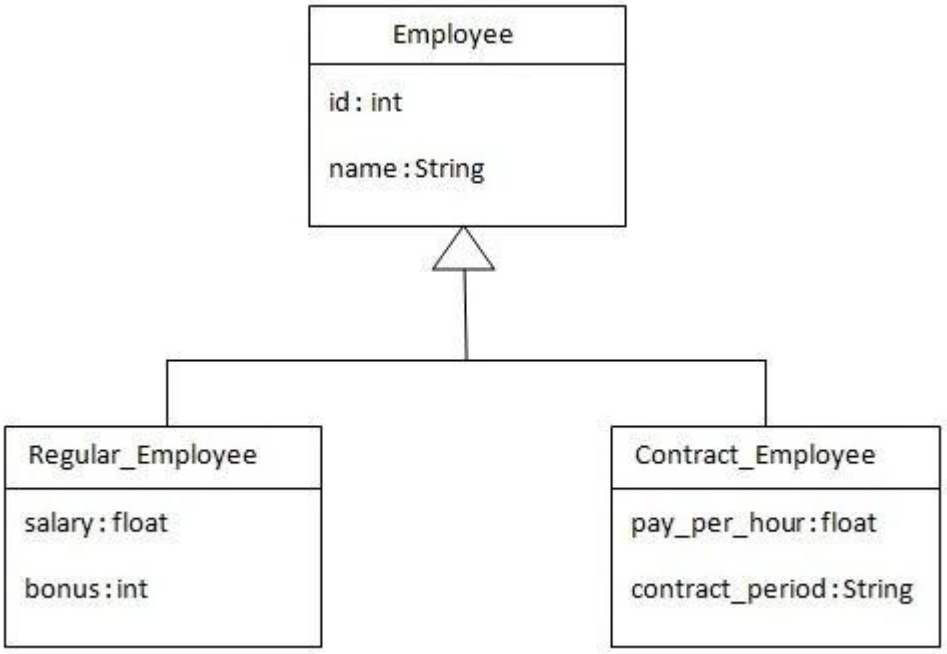
1. Table Per Subclass class
2. Example of Table Per Subclass

In case of Table Per Subclass, subclass mapped tables are related to parent class mapped table by primary key and foreign key relationship.

The <**joined-subclass**> element of class is used to map the child class with parent using the primary key and foreign key relation.

In this example, we are going to use hb2ddl.auto property to generate the table automatically. So we don't need to be worried about creating tables in the database.

Let's see the hierarchy of classes that we are going to map.



Let's see how can we map this hierarchy by joined-subclass element:

```
1. <?xml version='1.0' encoding='UTF-8'?>
2. <!DOCTYPE hibernate-mapping PUBLIC
3.
4.  "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
5.  "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
6.
7.
8.  <hibernate-mapping>
9.  <class name="com.smgc.mypackage.Employee" table="emp123">
```

```
<id name="id">
<generator class="increment"></generator>
</id>

<property name="name"></property>

<joined-subclass name="com.smgc.mypackage.Regular_Employee" table="regemp123">
<key column="eid"></key>
<property name="salary"></property>
<property name="bonus"></property>
</joined-subclass>

<joined-subclass name="com.smgc.mypackage.Contract_Employee" table="contemp123">
<key column="eid"></key>
<property name="pay_per_hour"></property>
<property name="contract_duration"></property>
</joined-subclass>

</class>
</hibernate-mapping>
```

In case of table per subclass class, there will be three tables in the database, each representing a particular class.

The **joined-subclass** subelement of class, specifies the subclass. The **key** subelement of joined-subclass is used to generate the foreign key in the subclass mapped table. This foreign key will be associated with the primary key of parent class mapped table.

The table structure for each table will be as follows:

Table structure for Employee class

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
NAME	VARCHAR2(255)	Yes	-	-
				1 - 2

Table structure for Regular_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
EID	NUMBER(10,0)	No	-	1
SALARY	FLOAT	Yes	-	-
BONUS	NUMBER(10,0)	Yes	-	-
				1 - 3

Table structure for Contract_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
EID	NUMBER(10,0)	No	-	1
PAY_PER_HOUR	FLOAT	Yes	-	-
CONTRACT_DURATION	VARCHAR2(255)	Yes	-	-
				1 - 3

Example of Table per subclass class

In this example we are creating the three classes and provide mapping of these classes in the employee.hbm.xml file.

1) Create the Persistent classes

You need to create the persistent classes representing the inheritance. Let's create the three classes for the above hierarchy:

File: Employee.java

```
1. package com.javatpoint.mypackage;
2.
3. public class Employee {
4. private int id;
5. private String name;
6.
7. //getters and setters
8. }
```

File: Regular_Employee.java

```
1. package com.javatpoint.mypackage;
2.
3. public class Regular_Employee extends Employee{
4. private float salary;
5. private int bonus;
6.
7. //getters and setters
8. }
```

File: Contract_Employee.java

```
1. package com.javatpoint.mypackage;
2.
3. public class Contract_Employee extends Employee{
4.     private float pay_per_hour;
```

```
5.      private String contract_duration;
6.
7.  //getters and setters
8.  }
```

---

2) Create the mapping file for Persistent class

The mapping has been discussed above for the hierarchy.

File: employee.hbm.xml

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-mapping PUBLIC
3.
4.    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
5.    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
6.
7.
8.    <hibernate-mapping>
9.    <class name="com.smgc.mypackage.Employee" table="emp123">
10.   <id name="id">
11.   <generator class="increment"></generator>
12.   </id>
13.
14.   <property name="name"></property>
15.
16.   <joined-subclass name="com.smgc.mypackage.Regular_Employee" table="regemp123">
17.   <key column="eid"></key>
18.   <property name="salary"></property>
19.   <property name="bonus"></property>
20.   </joined-subclass>
21.
22.   <joined-
      subclass name="com.smgc.mypackage.Contract_Employee" table="contemp123">
23.   <key column="eid"></key>
24.   <property name="pay_per_hour"></property>
25.   <property name="contract_duration"></property>
26.   </joined-subclass>
27.
28.   </class>
29.   </hibernate-mapping>
```

---

3) create configuration file

Open the hibernate.cgf.xml file, and add an entry of mapping resource like this:

```
1.  <mapping resource="employee.hbm.xml"/>
```

Now the configuration file will look like this:

File: hibernate.cfg.xml

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-configuration PUBLIC
3.          "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.  <hibernate-configuration>
7.
8.     <session-factory>
9.        <property name="hbm2ddl.auto">update</property>
10.       <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
11.       <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
12.       <property name="connection.username">system</property>
13.       <property name="connection.password">oracle</property>
14.       <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
15.    <mapping resource="employee.hbm.xml"/>
16.    </session-factory>
17.
18. </hibernate-configuration>
```

The hbm2ddl.auto property is defined for creating automatic table in the database.

---

4) Create the class that stores the persistent object

In this class, we are simply storing the employee objects in the database.

File: StoreData.java

```
1.  package com.javatpoint.mypackage;
2.
3.  import org.hibernate.*;
4.  import org.hibernate.cfg.*;
5.
6.  public class StoreData {
7.  public static void main(String[] args) {
8.     Session session=new Configuration().configure("hibernate.cfg.xml")
9.                   .buildSessionFactory().openSession();
10.
11.    Transaction t=session.beginTransaction();
```

```
12.
13.     Employee e1=new Employee();
14.     e1.setName("Piyush");
15.
16.     Regular_Employee e2=new Regular_Employee();
17.     e2.setName("Vivek Kumar");
18.     e2.setSalary(50000);
19.     e2.setBonus(5);
20.
21.     Contract_Employee e3=new Contract_Employee();
22.     e3.setName("Arjun Kumar");
23.     e3.setPay_per_hour(1000);
24.     e3.setContract_duration("15 hours");
25.
26.     session.persist(e1);
27.     session.persist(e2);
28.     session.persist(e3);
29.
30.     t.commit();
31.     session.close();
32.     System.out.println("success");
33. }
34. }
```

☺ Table Per Subclass using Annotation

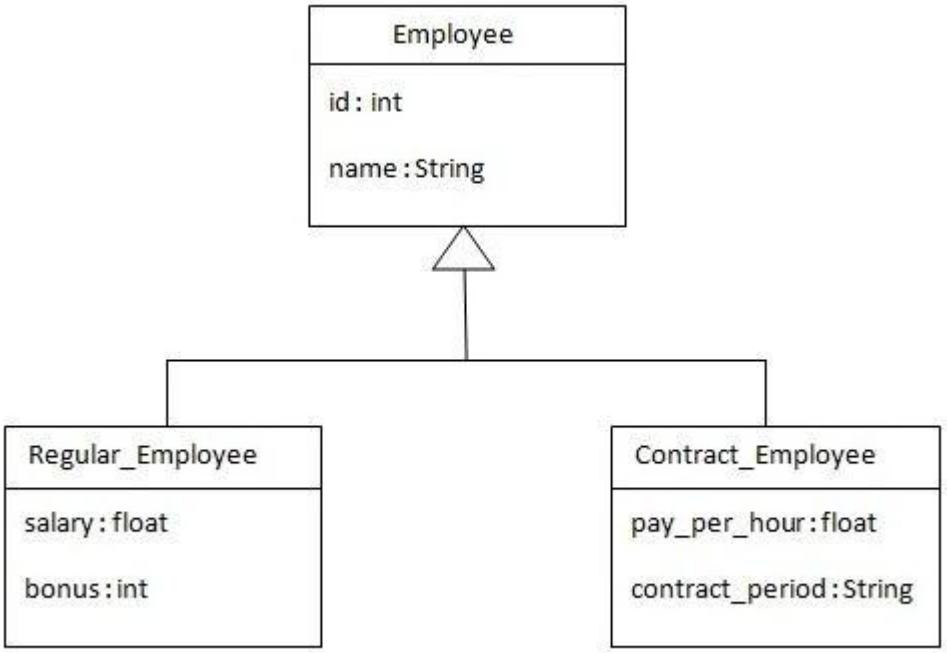
Table Per Subclass using Annotation

1. Table Per Subclass class
2. Example of Table Per Subclass

As we have specified earlier, in case of table per subclass strategy, tables are created as per persistent classes but they are reated using primary and foreign key. So there will not be duplicate columns in the relation.

We need to specify **@Inheritance(strategy=InheritanceType.JOINED)** in the parent class and **@PrimaryKeyJoinColumn** annotation in the subclasses.

Let's see the hierarchy of classes that we are going to map.



The table structure for each table will be as follows:

Table structure for Employee class

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
NAME	VARCHAR2(255)	Yes	-	-
				1 - 2

Table structure for Regular_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
EID	NUMBER(10,0)	No	-	1
SALARY	FLOAT	Yes	-	-
BONUS	NUMBER(10,0)	Yes	-	-
				1 - 3

Table structure for Contract_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
EID	NUMBER(10,0)	No	-	1
PAY_PER_HOUR	FLOAT	Yes	-	-
CONTRACT_DURATION	VARCHAR2(255)	Yes	-	-
				1 - 3

Example of Table per subclass class using Annotation

In this example we are creating the three classes and provide mapping of these classes in the employee.hbm.xml file.

1) Create the Persistent classes

You need to create the persistent classes representing the inheritance. Let's create the three classes for the above hierarchy:

File: Employee.java

```
1. package com.javatpoint.mypackage;
2. import javax.persistence.*;
3.
4. @Entity
5. @Table(name = "employee103")
6. @Inheritance(strategy=InheritanceType.JOINED)
7. public class Employee {
8. @Id
9. @GeneratedValue(strategy=GenerationType.AUTO)
10. @Column(name = "id")
11. private int id;
12. @Column(name = "name")
13. private String name;
14.
15. //setters and getters
16. }
```

File: Regular_Employee.java

```
1. package com.javatpoint.mypackage;
2.
3. import javax.persistence.*;
4.
5. @Entity
6. @Table(name="regularemployee103")
7. @PrimaryKeyJoinColumn(name="ID")
8. public class Regular_Employee extends Employee{
9.
10. @Column(name="salary")
11. private float salary;
12.
13. @Column(name="bonus")
14. private int bonus;
15.
16. //setters and getters
17. }
```

File: Contract_Employee.java

```
1. package com.javatpoint.mypackage;
2.
3. import javax.persistence.*;
4.
5. @Entity
6. @Table(name="contractemployee103")
7. @PrimaryKeyJoinColumn(name="ID")
8. public class Contract_Employee extends Employee{
9.
10.    @Column(name="pay_per_hour")
11.    private float pay_per_hour;
12.
13.    @Column(name="contract_duration")
14.    private String contract_duration;
15.
16.    //setters and getters
17. }
```

---

2) create configuration file

Open the hibernate.cgf.xml file, and add an entry of mapping resource like this:

```
1. <mapping class="com.smgc.mypackage.Employee"/>
2. <mapping class="com.smgc.mypackage.Contract_Employee"/>
3.       <mapping   class="com.smgc.mypackage.Regular_Employee"/>
```

Now the configuration file will look like this:

File: hibernate.cfg.xml

```
1. <?xml version='1.0' encoding='UTF-8'?>
2. <!DOCTYPE hibernate-configuration PUBLIC
3.           "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.           "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6. <!-- Generated by MyEclipse Hibernate Tools.                   -->
7. <hibernate-configuration>
8.
9.     <session-factory>
10.         <property name="hbm2ddl.auto">update</property>
11.         <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
12.         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
13.         <property name="connection.username">system</property>
14.         <property name="connection.password">oracle</property>
15. <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
16.
17.         <mapping class="com.smgc.mypackage.Employee"/>
18.         <mapping class="com.smgc.mypackage.Contract_Employee"/>
19.         <mapping class="com.smgc.mypackage.Regular_Employee"/>
20.     </session-factory>
21.
22. </hibernate-configuration>
```

The hbm2ddl.auto property is defined for creating automatic table in the database.

---

3) Create the class that stores the persistent object

In this class, we are simply storing the employee objects in the database.

File: StoreData.java

```
1. package com.javatpoint.mypackage;
2. import org.hibernate.*;
3. import org.hibernate.cfg.*;
4.
5. public class StoreData {
6. public static void main(String[] args) {
7.     AnnotationConfiguration cfg=new AnnotationConfiguration();
8.     Session session=cfg.configure("hibernate.cfg.xml").buildSessionFactory().openSession();
9.
10.    Transaction t=session.beginTransaction();
11.
12.    Employee e1=new Employee();
13.    e1.setName("Piyush");
14.
15.    Regular_Employee e2=new Regular_Employee();
16.    e2.setName("Vivek Kumar");
17.    e2.setSalary(50000);
18.    e2.setBonus(5);
19.
20.    Contract_Employee e3=new Contract_Employee();
21.    e3.setName("Arjun Kumar");
22.    e3.setPay_per_hour(1000);
23.    e3.setContract_duration("15 hours");
24.
25.    session.persist(e1);
26.    session.persist(e2);
27.    session.persist(e3);
28.
29.    t.commit();
30.    session.close();
31.    System.out.println("success");
32. }
33. }
```

Chapter-4.2

Collection Mapping

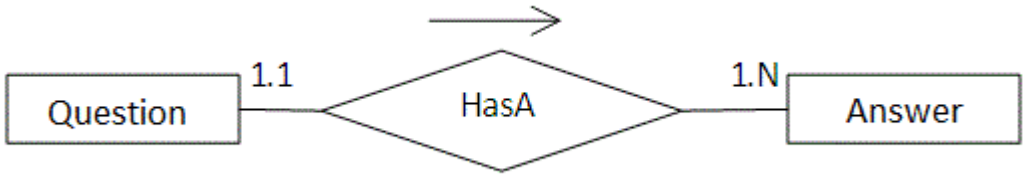
☺ Mapping List

Mapping List in Collection Mapping (using xml file)

1. Mapping List in Collection Mapping
2. Example of mapping list in collection mapping

If our persistent class has List object, we can map the List easily either by <list> element of class in mapping file or by annotation.

Here, we are using the scenario of Forum where one question has multiple answers.



Let's see how we can implement the list in the mapping file:

```
<class name="com.smgc.Question" table="q100">
    ...
      <list name="answers" table="ans100">
      <key column="qid"></key>
      <index column="type"></index>
      <element column="answer" type="string"></element>
      </list>

    ...
</class>
```

List and Map are index based collection, so an extra column will be created in the table for indexing.

Example of mapping list in collection mapping

In this example, we are going to see full example of collection mapping by list. This is the example of List that stores string value not entity reference that is why are going to use **element** instead of **one-to-many** within the list element.

1) create the Persistent class

This persistent class defines properties of the class including List.

```
1. package com.smgc;
2.
3. import java.util.List;
4.
5. public class Question {
6. private int id;
7. private String qname;
8. private List<String> answers;
9.
10. //getters and setters
11.
12. }
```

---

2) create the Mapping file for the persistent class

Here, we have created the question.hbm.xml file for defining the list.

```
1. <?xml version='1.0' encoding='UTF-8'?>
2. <!DOCTYPE hibernate-mapping PUBLIC
3.         "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.         "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6. <hibernate-mapping>
7.  <class name="com.smgc.Question" table="q100">
8.    <id name="id">
9.      <generator class="increment"></generator>
10.   </id>
11.   <property name="qname"></property>
12.
13.   <list name="answers" table="ans100">
14.     <key column="qid"></key>
15.     <index column="type"></index>
16.     <element column="answer" type="string"></element>
17.   </list>
18.
19. </class>
20.
21. </hibernate-mapping>
```

---

3) create the configuration file

This file contains information about the database and mapping file.

```
1. <?xml version='1.0' encoding='UTF-8'?>
2. <!DOCTYPE hibernate-configuration PUBLIC
3.         "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.         "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6. <!-- Generated by MyEclipse Hibernate Tools.                 -->
7. <hibernate-configuration>
8.
9.     <session-factory>
10.        <property name="hbm2ddl.auto">update</property>
11.        <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
12.        <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
13.        <property name="connection.username">system</property>
14.        <property name="connection.password">oracle</property>
15.        <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
16.    <mapping resource="question.hbm.xml"/>
17.    </session-factory>
18.
19. </hibernate-configuration>
```

---

4) Create the class to store the data

In this class we are storing the data of the question class.

```
1.  package com.smgc;
2.
3.  import java.util.ArrayList;
4.
5.  import org.hibernate.*;
6.  import org.hibernate.cfg.*;
7.
8.  public class StoreData {
9.   public static void main(String[] args) {
10.    Session session=new Configuration().configure("hibernate.cfg.xml")
11.                    .buildSessionFactory().openSession();
12.    Transaction t=session.beginTransaction();
13.
14.    ArrayList<String> list1=new ArrayList<String>();
15.    list1.add("java is a programming language");
16.    list1.add("java is a platform");
17.
18.    ArrayList<String> list2=new ArrayList<String>();
19.    list2.add("Servlet is an Interface");
20.    list2.add("Servlet is an API");
21.
```

```
22.     Question question1=new Question();
23.     question1.setQname("What is Java?");
24.     question1.setAnswers(list1);
25.
26.     Question question2=new Question();
27.     question2.setQname("What is Servlet?");
28.     question2.setAnswers(list2);
29.
30.     session.persist(question1);
31.     session.persist(question2);
32.
33.     t.commit();
34.     session.close();
35.     System.out.println("success");
36.  }
37. }
```

How to fetch the data of List

Here, we have used HQL to fetch all the records of Question class including answers. In such case, it fetches the data from two tables that are functional dependent.

```
1.  package com.smgc;
2.
3.  import java.util.*;
4.
5.  import org.hibernate.*;
6.  import org.hibernate.cfg.*;
7.
8.  public class FetchData {
9.  public static void main(String[] args) {
10.
11.     Session session=new Configuration().configure("hibernate.cfg.xml")
12.                     .buildSessionFactory().openSession();
13.
14.     Query query=session.createQuery("from Question");
15.     List<Question> list=query.list();
16.
17.     Iterator<Question> itr=list.iterator();
18.     while(itr.hasNext()){
19.        Question q=itr.next();
20.        System.out.println("Question Name: "+q.getQname());
21.
22.        //printing answers
23.        List<String> list2=q.getAnswers();
24.        Iterator<String> itr2=list2.iterator();
25.        while(itr2.hasNext()){
26.           System.out.println(itr2.next());
27.        }
```

```
28.
29.     }
30.     session.close();
31.     System.out.println("success");
32.
33. }
34. }
```

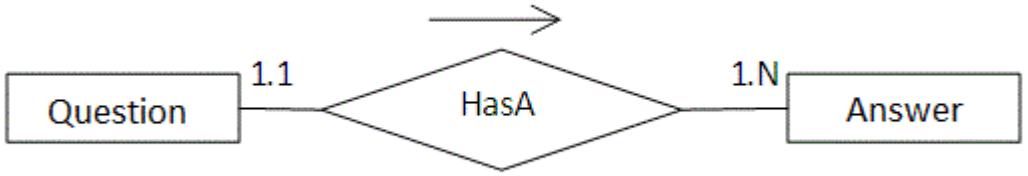
☺ One-to-many by List using XML

One to Many mapping in Hibernate by List Example (using xml file)

1. List One to many
2. Example of mapping list in collection mapping by one to many association

If the persistent class has list object that contains the entity reference, we need to use one-to-many association to map the list element.

Here, we are using the scenario of Forum where one question has multiple answers.



In such case, there can be many answers for a question and each answer may have its own informations that is why we have used **list** in the persistent class (containing the reference of Answer class) to represent a collection of answers.

Let's see the persistent class that has list objects (containing Answer class objects).

```
1. package com.smgc;
2.
3. import java.util.List;
4.
5. public class Question {
6. private int id;
7. private String qname;
8. private List<Answer> answers;
9. //getters and setters
10. }
```

The Answer class has its own informations such as id, answername, postedBy etc.

```
1. package com.smgc;
2. public class Answer {
3. private int id;
4. private String answername;
5. private String postedBy;
6. //getters and setters
7. }
8. }
```

The Question class has list object that have entity reference (i.e. Answer class object). In such case, we need to use **one-to-many** of list to map this object. Let's see how we can map it.

```
<list name="answers" cascade="all">
        <key column="qid"></key>
        <index column="type"></index>
        <one-to-many class="com.smgc.Answer"/>
</list>
```

Full example of One to Many mapping in Hibernate by List

In this example, we are going to see full example of mapping list that contains entity reference.

1) create the Persistent class

This persistent class defines properties of the class including List.

Question.java

```
package com.smgc;

import java.util.List;

public class Question {
private int id;
private String qname;
private List<Answer> answers;

//getters and setters

}
```

Answer.java

```
package com.smgc;
public class Answer {
private int id;
private String answername;
private String postedBy;
//getters and setters
}
}
```

---

2) create the Mapping file for the persistent class

Here, we have created the question.hbm.xml file for defining the list.

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-mapping PUBLIC
3.          "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.          <hibernate-mapping>
7.          <class name="com.smgc.Question" table="q501">
8.          <id name="id">
9.          <generator class="increment"></generator>
10.         </id>
11.         <property name="qname"></property>
12.
13.         <list name="answers" cascade="all">
14.         <key column="qid"></key>
15.         <index column="type"></index>
16.         <one-to-many class="com.smgc.Answer"/>
17.         </list>
18.
19.         </class>
20.
21.         <class name="com.smgc.Answer" table="ans501">
22.         <id name="id">
23.         <generator class="increment"></generator>
24.         </id>
25.         <property name="answername"></property>
26.         <property name="postedBy"></property>
27.         </class>
28.
29.         </hibernate-mapping>
```

---

3) create the configuration file

This file contains information about the database and mapping file.

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-configuration PUBLIC
3.          "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.  <!-- Generated by MyEclipse Hibernate Tools.                   -->
7.  <hibernate-configuration>
8.
9.      <session-factory>
10.       <property name="hbm2ddl.auto">update</property>
11.       <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
```

```
12.       <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
13.       <property name="connection.username">system</property>
14.       <property name="connection.password">oracle</property>
15.       <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
16.    <mapping resource="question.hbm.xml"/>
17.    </session-factory>
18.
19. </hibernate-configuration>
```

---

4) Create the class to store the data

In this class we are storing the data of the question class.

```
1.  package com.smgc;
2.  import java.util.ArrayList;
3.  import org.hibernate.*;
4.  import org.hibernate.cfg.*;
5.
6.  public class StoreData {
7.  public static void main(String[] args) {
8.     Session session=new Configuration().configure("hibernate.cfg.xml")
9.                         .buildSessionFactory().openSession();
10.    Transaction t=session.beginTransaction();
11.
12.    Answer ans1=new Answer();
13.    ans1.setAnswername("java is a programming language");
14.    ans1.setPostedBy("Ravi Malik");
15.
16.    Answer ans2=new Answer();
17.    ans2.setAnswername("java is a platform");
18.    ans2.setPostedBy("Sudhir Kumar");
19.
20.    Answer ans3=new Answer();
21.    ans3.setAnswername("Servlet is an Interface");
22.    ans3.setPostedBy("Jai Kumar");
23.
24.    Answer ans4=new Answer();
25.    ans4.setAnswername("Servlet is an API");
26.    ans4.setPostedBy("Arun");
27.
28.    ArrayList<Answer> list1=new ArrayList<Answer>();
29.    list1.add(ans1);
30.    list1.add(ans2);
31.
32.    ArrayList<Answer> list2=new ArrayList<Answer>();
33.    list2.add(ans3);
34.    list2.add(ans4);
35.
36.    Question question1=new Question();
```

```
37.     question1.setQname("What is Java?");
38.     question1.setAnswers(list1);
39.
40.     Question question2=new Question();
41.     question2.setQname("What is Servlet?");
42.     question2.setAnswers(list2);
43.
44.     session.persist(question1);
45.     session.persist(question2);
46.
47.     t.commit();
48.     session.close();
49.     System.out.println("success");
50. }
51. }
```

OUTPUT

ID	QNAME
1	What is Java?
2	What is Servlet?

ID	ANSWERNAME	POSTEDBY	QID	TYPE
1	java is a programming language	Ravi Malik	1	0
2	java is a platform	Sudhir Kumar	1	1
3	Servlet is an Interface	Jai Kumar	2	0
4	Servlet is an API	Arun	2	1

How to fetch the data of List

Here, we have used HQL to fetch all the records of Question class including answers. In such case, it fetches the data from two tables that are functional dependent. Here, we are direct printing the object of answer class, but we have overridden the **toString()** method in the Answer class returning answername and poster name. So it prints the answer name and postername rather than reference id.

FetchData.java

```
1.  package com.smgc;
2.  import java.util.*;
3.  import org.hibernate.*;
4.  import org.hibernate.cfg.*;
5.
6.  public class FetchData {
7.  public static void main(String[] args) {
8.
9.      Session session=new Configuration().configure("hibernate.cfg.xml")
10.                 .buildSessionFactory().openSession();
11.
12.     Query query=session.createQuery("from Question");
13.     List<Question> list=query.list();
14.
15.     Iterator<Question> itr=list.iterator();
16.     while(itr.hasNext()){
17.       Question q=itr.next();
18.       System.out.println("Question Name: "+q.getQname());
19.
20.       //printing answers
21.       List<Answer> list2=q.getAnswers();
22.       Iterator<Answer> itr2=list2.iterator();
23.       while(itr2.hasNext()){
24.         System.out.println(itr2.next());
25.       }
26.
27.     }
28.     session.close();
29.     System.out.println("success");
30. }
31. }
```

OUTPUT

```
Question Name: What is Java?
java is a programming language by: Ravi Malik
java is a platform by: Sudhir Kumar
Question Name: What is Servlet?
Servlet is an Interface by: Jai Kumar
Servlet is an API by: Arun
success
```

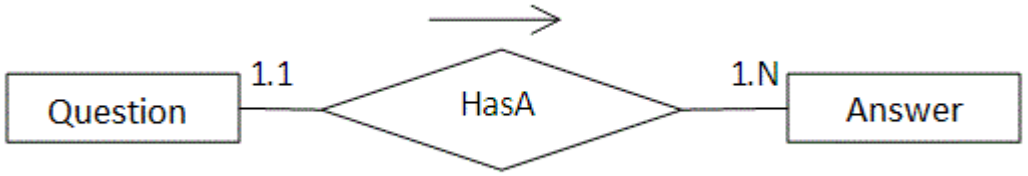
☺ Mpping Bag

Mapping Bag in Collection Mapping (using xml file)

1. Mapping Bag in Collection Mapping
2. Example of mapping Bag in Collection Mapping

If our persistent class has List object, we can map the List by list or bag element in the mapping file. The bag is just like List but it doesn't require index element.

Here, we are using the scenario of Forum where one question has multiple answers.



Let's see how we can implement the bag in the mapping file:

```
<class name="com.smgc.Question" table="q100">
    ...
      <bag name="answers" table="ans100">
      <key column="qid"></key>
      <element column="answer" type="string"></element>
      </bag>

    ...
</class>
```

Example of mapping bag in collection mapping

In this example, we are going to see full example of collection mapping by bag. This is the example of bag if it stores value not entity reference that is why are going to use **element** instead of **one-to-many**. If you have seen the example of mapping list, it is same in all cases instead mapping file where we are using bag instead of list.

1) create the Persistent class

This persistent class defines properties of the class including List.

```
1. package com.smgc;
2.
3. import java.util.List;
4.
5. public class Question {
6. private int id;
7. private String qname;
8. private List<String> answers;
9.
10. //getters and setters
11.
12. }
```

---

2) create the Mapping file for the persistent class

Here, we have created the question.hbm.xml file for defining the list.

```
1. <?xml version='1.0' encoding='UTF-8'?>
2. <!DOCTYPE hibernate-mapping PUBLIC
3.         "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.         "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6. <hibernate-mapping>
7.  <class name="com.smgc.Question" table="q101">
8.    <id name="id">
9.      <generator class="increment"></generator>
10.   </id>
11.   <property name="qname"></property>
12.
13.   <bag name="answers" table="ans101">
14.     <key column="qid"></key>
15.     <element column="answer" type="string"></element>
16.   </bag>
17.
18.  </class>
19.
20. </hibernate-mapping>
```

---

3) create the configuration file

This file contains information about the database and mapping file.

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">

<!-- Generated by MyEclipse Hibernate Tools.                   -->
<hibernate-configuration>

    <session-factory>
       <property name="hbm2ddl.auto">update</property>
       <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
       <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
       <property name="connection.username">system</property>
       <property name="connection.password">oracle</property>
       <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
    <mapping resource="question.hbm.xml"/>
    </session-factory>

</hibernate-configuration>
```

---

4) Create the class to store the data

In this class we are storing the data of the question class.

```
package com.smgc;

import java.util.ArrayList;

import org.hibernate.*;
import org.hibernate.cfg.*;

public class StoreData {
 public static void main(String[] args) {
    Session session=new Configuration().configure("hibernate.cfg.xml")
                    .buildSessionFactory().openSession();
    Transaction t=session.beginTransaction();

    ArrayList<String> list1=new ArrayList<String>();
    list1.add("java is a programming language");
    list1.add("java is a platform");

    ArrayList<String> list2=new ArrayList<String>();
    list2.add("Servlet is an Interface");
    list2.add("Servlet is an API");

```

```
22.     Question question1=new Question();
23.     question1.setQname("What is Java?");
24.     question1.setAnswers(list1);
25.
26.     Question question2=new Question();
27.     question2.setQname("What is Servlet?");
28.     question2.setAnswers(list2);
29.
30.     session.persist(question1);
31.     session.persist(question2);
32.
33.     t.commit();
34.     session.close();
35.     System.out.println("success");
36.  }
37. }
```

How to fetch the data

Here, we have used HQL to fetch all the records of Question class including answers. In such case, it fetches the data from two tables that are functional dependent.

```
1.  package com.smgc;
2.
3.  import java.util.*;
4.
5.  import org.hibernate.*;
6.  import org.hibernate.cfg.*;
7.
8.  public class FetchData {
9.  public static void main(String[] args) {
10.
11.     Session session=new Configuration().configure("hibernate.cfg.xml")
12.                     .buildSessionFactory().openSession();
13.
14.     Query query=session.createQuery("from Question");
15.     List<Question> list=query.list();
16.
17.     Iterator<Question> itr=list.iterator();
18.     while(itr.hasNext()){
19.        Question q=itr.next();
20.        System.out.println("Question Name: "+q.getQname());
21.
22.        //printing answers
23.        List<String> list2=q.getAnswers();
24.        Iterator<String> itr2=list2.iterator();
25.        while(itr2.hasNext()){
26.           System.out.println(itr2.next());
27.        }
```

```
28.
29.     }
30.     session.close();
31.     System.out.println("success");
32.
33. }
34. }
```

☺ One-to-many by Bag

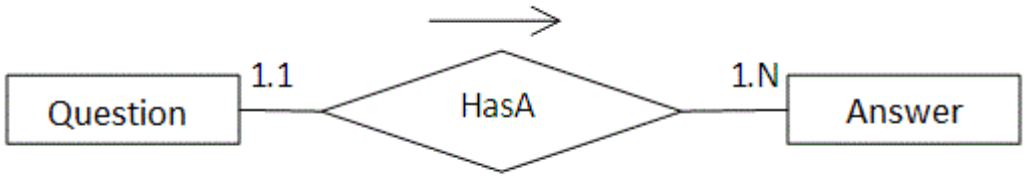
One to Many mapping in Hibernate by List Example (using xml file)

1. Bag One to many
2. Example of mapping bag in collection mapping by one to many association

If the persistent class has list object that contains the entity reference, we need to use **one-to-many** association to map the list element. We can map this list object by either **list** or **bag**.

Notice that bag is not index-based whereas list is index-based.

Here, we are using the scenario of Forum where one question has multiple answers.



Let's see the persistent class that has list objects. In this case, there can be many answers for a question and each answer may have its own informations that is why we have used list element (containing Answer objects) to represent a collection of answers.

```
1. package com.smgc;
2.
3. import java.util.List;
4.
5. public class Question {
6. private int id;
7. private String qname;
8. private List<Answer> answers;
9. //getters and setters
10.
11. }
```

The Answer class has its own informations such as id, answername, postedBy etc.

```
1. package com.smgc;
2.
3. public class Answer {
4. private int id;
5. private String answername;
6. private String postedBy;
7. //getters and setters
8. }
9. }
```

The Question class has list object that have entity reference (i.e. Answer class object). In such case, we need to use **one-to-many** of bag to map this object. Let's see how we can map it.

```
<bag name="answers" cascade="all">
        <key column="qid"></key>
        <one-to-many class="com.smgc.Answer"/>
</list>
```

Example of mapping bag in collection mapping by one to many association

In this example, we are going to see full example of mapping list that contains entity reference.

1) create the Persistent class

This persistent class defines properties of the class including List.

Question.java

```
package com.smgc;

import java.util.List;

public class Question {
private int id;
private String qname;
private List<Answer> answers;

//getters and setters

}
```

Answer.java

```
package com.smgc;

public class Answer {
private int id;
private String answername;
private String postedBy;
//getters and setters

}
}
```

---

2) create the Mapping file for the persistent class

Here, we have created the question.hbm.xml file for defining the list.

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-mapping PUBLIC
3.          "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.          <hibernate-mapping>
7.          <class name=“com.smgc.Question" table="q501">
8.          <id name="id">
9.          <generator class="increment"></generator>
10.         </id>
11.         <property name="qname"></property>
12.
13.         <bag name="answers" cascade="all">
14.         <index column="type"></index>
15.         <one-to-many class=“com.smgc.Answer"/>
16.         </bag>
17.
18.         </class>
19.
20.         <class name=“com.smgc.Answer" table="ans501">
21.         <id name="id">
22.         <generator class="increment"></generator>
23.         </id>
24.         <property name="answername"></property>
25.         <property name="postedBy"></property>
26.         </class>
27.
28.         </hibernate-mapping>
```

---

3) create the configuration file

This file contains information about the database and mapping file.

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-configuration PUBLIC
3.          "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.  <!-- Generated by MyEclipse Hibernate Tools.                   -->
7.  <hibernate-configuration>
8.
9.      <session-factory>
10.       <property name="hbm2ddl.auto">update</property>
11.       <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
12.       <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
```

```
13.       <property name="connection.username">system</property>
14.       <property name="connection.password">oracle</property>
15.       <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
16.    <mapping resource="question.hbm.xml"/>
17.    </session-factory>
18.
19. </hibernate-configuration>
```

---

4) Create the class to store the data

In this class we are storing the data of the question class.

```
1.  package com.smgc;
2.
3.  import java.util.ArrayList;
4.
5.  import org.hibernate.*;
6.  import org.hibernate.cfg.*;
7.
8.  public class StoreData {
9.  public static void main(String[] args) {
10.    Session session=new Configuration().configure("hibernate.cfg.xml")
11.                        .buildSessionFactory().openSession();
12.    Transaction t=session.beginTransaction();
13.
14.    Answer ans1=new Answer();
15.    ans1.setAnswername("java is a programming language");
16.    ans1.setPostedBy("Ravi Malik");
17.
18.    Answer ans2=new Answer();
19.    ans2.setAnswername("java is a platform");
20.    ans2.setPostedBy("Sudhir Kumar");
21.
22.    Answer ans3=new Answer();
23.    ans3.setAnswername("Servlet is an Interface");
24.    ans3.setPostedBy("Jai Kumar");
25.
26.    Answer ans4=new Answer();
27.    ans4.setAnswername("Servlet is an API");
28.    ans4.setPostedBy("Arun");
29.
30.    ArrayList<Answer> list1=new ArrayList<Answer>();
31.    list1.add(ans1);
32.    list1.add(ans2);
33.
34.    ArrayList<Answer> list2=new ArrayList<Answer>();
35.    list2.add(ans3);
36.    list2.add(ans4);
37.
```

```
38.    Question question1=new Question();
39.    question1.setQname("What is Java?");
40.    question1.setAnswers(list1);
41.
42.    Question question2=new Question();
43.    question2.setQname("What is Servlet?");
44.    question2.setAnswers(list2);
45.
46.    session.persist(question1);
47.    session.persist(question2);
48.
49.    t.commit();
50.    session.close();
51.    System.out.println("success");
52.
53. }
54. }
```

How to fetch the data of List

Here, we have used HQL to fetch all the records of Question class including answers. In such case, it fetches the data from two tables that are functional dependent. Here, we are direct printing the object of answer class, but we have overridden the **toString()** method in the Answer class returning answername and poster name. So it prints the answer name and postername rather than reference id.

FetchData.java

```
1.  package com.smgc;
2.
3.  import java.util.*;
4.
5.  import org.hibernate.*;
6.  import org.hibernate.cfg.*;
7.
8.  public class FetchData {
9.  public static void main(String[] args) {
10.
11.    Session session=new Configuration().configure("hibernate.cfg.xml")
12.                    .buildSessionFactory().openSession();
13.
14.    Query query=session.createQuery("from Question");
15.    List<Question> list=query.list();
16.
17.    Iterator<Question> itr=list.iterator();
18.    while(itr.hasNext()){
19.        Question q=itr.next();
20.        System.out.println("Question Name: "+q.getQname());
```

```
21.
22.         //printing answers
23.         List<Answer> list2=q.getAnswers();
24.         Iterator<Answer> itr2=list2.iterator();
25.         while(itr2.hasNext()){
26.             System.out.println(itr2.next());
27.         }
28.
29.     }
30.     session.close();
31.     System.out.println("success");
32.
33. }
34. }
```

OUTPUT

```
Question Name: What is Java?
java is a programming language by: Ravi Malik
java is a platform by: Sudhir Kumar
Question Name: What is Servlet?
Servlet is an Interface by: Jai Kumar
Servlet is an API by: Arun
success
```

☺ Mapping Set

Mapping Set in Collection Mapping

1. Mapping Set in Collection Mapping
2. Example of mapping Set in Collection Mapping

If our persistent class has Set object, we can map the Set by set element in the mapping file. The set element doesn't require index element. The one difference between List and Set is that, it stores only unique values.

Let's see how we can implement the set in the mapping file:

```
<class name="com.smgc.Question" table="q102">
    ...
      <set name="answers" table="ans102">
      <key column="qid"></key>
      <element column="answer" type="string"></element>
      </set>

    ...
</class>
```

Example of mapping set in collection mapping

In this example, we are going to see full example of collection mapping by set. This is the example of set that stores value not entity reference that is why are going to use element instead of one-to-many.

1) create the Persistent class

This persistent class defines properties of the class including Set.

```
package com.smgc;

import java.util.Set;

public class Question {
private int id;
private String qname;
private Set<String> answers;

//getters and setters

}
```

---

2) create the Mapping file for the persistent class

Here, we have created the question.hbm.xml file for defining the list.

```
1. <?xml version='1.0' encoding='UTF-8'?>
2. <!DOCTYPE hibernate-mapping PUBLIC
3.         "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.         "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6. <hibernate-mapping>
7.  <class name="com.smgc.Question" table="q102">
8.    <id name="id">
9.      <generator class="increment"></generator>
10.   </id>
11.   <property name="qname"></property>
12.
13.   <set name="answers" table="ans102">
14.    <key column="qid"></key>
15.    <element column="answer" type="string"></element>
16.   </set>
17.
18. </class>
19.
20. </hibernate-mapping>
```

---

3) create the configuration file

This file contains information about the database and mapping file.

```
1. <?xml version='1.0' encoding='UTF-8'?>
2. <!DOCTYPE hibernate-configuration PUBLIC
3.         "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.         "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6. <!-- Generated by MyEclipse Hibernate Tools.                 -->
7. <hibernate-configuration>
8.
9.     <session-factory>
10.       <property name="hbm2ddl.auto">update</property>
11.       <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
12.       <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
13.       <property name="connection.username">system</property>
14.       <property name="connection.password">oracle</property>
15.       <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
16.    <mapping resource="question.hbm.xml"/>
17.    </session-factory>
18. </hibernate-configuration>
```

---

4) Create the class to store the data

In this class we are storing the data of the question class.

```
1.  package com.smgc;
2.
3.  import java.util.ArrayList;
4.
5.  import org.hibernate.*;
6.  import org.hibernate.cfg.*;
7.
8.  public class StoreData {
9.   public static void main(String[] args) {
10.    Session session=new Configuration().configure("hibernate.cfg.xml")
11.                  .buildSessionFactory().openSession();
12.    Transaction t=session.beginTransaction();
13.
14.
15.    HashSet<String> set1=new HashSet<String>();
16.    set1.add("java is a programming language");
17.    set1.add("java is a platform");
18.
19.    HashSet<String> set2=new HashSet<String>();
20.    set2.add("Servlet is an Interface");
21.    set2.add("Servlet is an API");
22.
23.    Question question1=new Question();
24.    question1.setQname("What is Java?");
25.    question1.setAnswers(set1);
26.
27.    Question question2=new Question();
28.    question2.setQname("What is Servlet?");
29.    question2.setAnswers(set2);
30.
31.    session.persist(question1);
32.    session.persist(question2);
33.
34.    t.commit();
35.    session.close();
36.    System.out.println("success");
37.  }
38. }
```

How to fetch the data of Set

Here, we have used HQL to fetch all the records of Question class including answers. In such case, it fetches the data from two tables that are functional dependent.

```
1.  package com.smgc;
2.
3.  import java.util.*;
4.
5.  import org.hibernate.*;
6.  import org.hibernate.cfg.*;
7.
8.  public class FetchData {
9.  public static void main(String[] args) {
10.
11.    Session session=new Configuration().configure("hibernate.cfg.xml")
12.                    .buildSessionFactory().openSession();
13.
14.    Query query=session.createQuery("from Question");
15.    List<Question> list=query.list();
16.
17.    Iterator<Question> itr=list.iterator();
18.    while(itr.hasNext()){
19.       Question q=itr.next();
20.       System.out.println("Question Name: "+q.getQname());
21.
22.       //printing answers
23.       Set<String> set=q.getAnswers();
24.       Iterator<String> itr2=set.iterator();
25.       while(itr2.hasNext()){
26.          System.out.println(itr2.next());
27.       }
28.
29.    }
30.    session.close();
31.    System.out.println("success");
32.
33. }
34. }
```

☺ One-to-many by Set

Set One to many

1. Set One to many
2. Example of mapping set in collection mapping by one to many association

If the persistent class has set object that contains the entity reference, we need to use one-to-many association to map the set element. We can map this list object by either **set**.

It is non-index based and will not allow duplicate elements.

Let's see the persistent class that has set objects. In this case, there can be many answers for a question and each answer may have its own informations that is why we have used set element to represent a collection of answers.

```
1. package com.smgc;
2.
3. import java.util.List;
4.
5. public class Question {
6. private int id;
7. private String qname;
8. private Set<Answer> answers;
9. //getters and setters
10.
11. }
```

The Answer class has its own informations such as id, answername, postedBy etc.

```
1. package com.smgc;
2.
3. public class Answer {
4. private int id;
5. private String answername;
6. private String postedBy;
7. //getters and setters
8.
9. }
10. }
```

The Question class has set object that have entity reference (i.e. Answer class object). In such case, we need to use **one-to-many** of set to map this object. Let's see how we can map it.

```
1. <set name="answers" cascade="all">
2.         <key column="qid"></key>
3.         <one-to-many class="com.smgc.Answer"/>
4. </set>
```

Example of mapping set in collection mapping by one to many association

To understand this example, you may see the bag one-to-many relation example. We have changed only bag to set in the hbm file and ArrayList to HashSet in the Store class.

☺ Mapping Map

Mapping Map in collection mapping using xml file

1. Mapping Map in collection mapping
2. Example of Mapping Map in collection mapping

Hibernate allows you to map Map elements with the RDBMS. As we know, list and map are index-based collections. In case of map, index column works as the key and element column works as the value.

Example of Mapping Map in collection mapping using xml file

You need to create following pages for mapping map elements.

- Question.java
- question.hbm.xml
- hibernate.cfg.xml
- StoreTest.java
- FetchTest.java

Question.java

```
1. package com.smgc;
2.
3. import java.util.Map;
4.
5. public class Question {
6. private int id;
7. private String name,username;
8. private Map<String,String> answers;
9.
10. public Question() {}
11. public Question(String name, String username, Map<String, String> answers) {
12.     super();
13.     this.name = name;
14.     this.username = username;
15.     this.answers = answers;
16. }
17. public int getId() {
18.     return id;
19. }
20. public void setId(int id) {
21.     this.id = id;
22. }
23. public String getName() {
24.     return name;
25. }
26. public void setName(String name) {
```

```
27.     this.name = name;
28. }
29. public String getUsername() {
30.     return username;
31. }
32. public void setUsername(String username) {
33.     this.username = username;
34. }
35. public Map<String, String> getAnswers() {
36.     return answers;
37. }
38. public void setAnswers(Map<String, String> answers) {
39.     this.answers = answers;
40. }
41. }
```

---

question.hbm.xml

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-mapping PUBLIC
3.          "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.  <hibernate-mapping>
7.
8.  <class name="com.smgc.Question" table="question736">
9.  <id name="id">
10. <generator class="native"></generator>
11. </id>
12. <property name="name"></property>
13. <property name="username"></property>
14.
15. <map name="answers" table="answer736" cascade="all">
16. <key column="questionid"></key>
17. <index column="answer" type="string"></index>
18. <element column="username" type="string"></element>
19. </map>
20. </class>
21.
22. </hibernate-mapping>
```

hibernate.cfg.xml

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-configuration PUBLIC
3.          "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.  <!-- Generated by MyEclipse Hibernate Tools.                   -->
7.  <hibernate-configuration>
```

```
8.
9.      <session-factory>
10.       <property name="hbm2ddl.auto">update</property>
11.       <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
12.       <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
13.       <property name="connection.username">system</property>
14.       <property name="connection.password">oracle</property>
15.       <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
16.
17.    <mapping resource="question.hbm.xml"/>
18.    </session-factory>
19.
20. </hibernate-configuration>
```

StoreTest.java

```
1.  package com.smgc;
2.
3.  import java.util.HashMap;
4.  import org.hibernate.*;
5.  import org.hibernate.cfg.*;
6.  public class StoreTest {
7.  public static void main(String[] args) {
8.  Session session=new Configuration().configure().buildSessionFactory().openSession();
9.  Transaction tx=session.beginTransaction();
10.
11. HashMap<String,String> map1=new HashMap<String,String>();
12. map1.put("java is a programming language","John Milton");
13. map1.put("java is a platform","Ashok Kumar");
14.
15. HashMap<String,String> map2=new HashMap<String,String>();
16. map2.put("servlet technology is a server side programming","John Milton");
17. map2.put("Servlet is an Interface","Ashok Kumar");
18. map2.put("Servlet is a package","Rahul Kumar");
19.
20. Question question1=new Question("What is java?","Alok",map1);
21. Question question2=new Question("What is servlet?","Jai Dixit",map2);
22.
23. session.persist(question1);
24. session.persist(question2);
25.
26. tx.commit();
27. session.close();
28. System.out.println("successfully stored");
29. }
30. }
```

FetchTest.java

```
1.  package com.smgc;
2.  import java.util.*;
3.  import org.hibernate.*;
4.  import org.hibernate.cfg.*;
5.  public class FetchTest {
6.  public static void main(String[] args) {
7.   Session session=new Configuration().configure().buildSessionFactory().openSession();
8.
9.   Query query=session.createQuery("from Question ");
10.  List<Question> list=query.list();
11.
12.  Iterator<Question> iterator=list.iterator();
13.  while(iterator.hasNext()){
14.   Question question=iterator.next();
15.   System.out.println("question id:"+question.getId());
16.   System.out.println("question name:"+question.getName());
17.   System.out.println("question posted by:"+question.getUsername());
18.   System.out.println("answers.....");
19.   Map<String,String> map=question.getAnswers();
20.   Set<Map.Entry<String,String>> set=map.entrySet();
21.
22.   Iterator<Map.Entry<String,String>> iteratoranswer=set.iterator();
23.   while(iteratoranswer.hasNext()){
24.    Map.Entry<String,String> entry=(Map.Entry<String,String>)iteratoranswer.next();
25.    System.out.println("answer name:"+entry.getKey());
26.    System.out.println("answer posted by:"+entry.getValue());
27.   }
28.  }
29. session.close();
30. }
31. }
```

☺ Many-to-many by Map

Many to Many Mapping in hibernate by map Example (using xml file)

1. Many to Many Mapping
2. Example of Many to Many Mapping

We can map many to many relation either using set, bag, map etc. Here, we are going to use map for many-to-many mapping. In such case, three tables will be created.

Example of Many to Many Mapping

You need to create following pages for mapping map elements.

- Question.java
- User.java
- question.hbm.xml
- user.hbm.xml
- hibernate.cfg.xml
- StoreTest.java
- FetchTest.java

Question.java

```
1. package com.smgc;
2.
3. import java.util.Map;
4.
5. public class Question {
6. private int id;
7. private String name;
8. private Map<String,User> answers;
9.
10. public Question() {}
11. public Question(String name, Map<String, User> answers) {
12.     super();
13.     this.name = name;
14.     this.answers = answers;
15. }
16. public int getId() {
17.     return id;
18. }
19. public void setId(int id) {
20.     this.id = id;
21. }
22. public String getName() {
23.     return name;
24. }
25. public void setName(String name) {
```

```
26.     this.name = name;
27. }
28. public Map<String, User> getAnswers() {
29.     return answers;
30. }
31. public void setAnswers(Map<String, User> answers) {
32.     this.answers = answers;
33. }
34.
35.
36. }
```

User.java

```
1.  package com.smgc;
2.
3.  public class User {
4.  private int id;
5.  private String username,email,country;
6.
7.  public User() {}
8.  public User(String username, String email, String country) {
9.      super();
10.     this.username = username;
11.     this.email = email;
12.     this.country = country;
13. }
14. public int getId() {
15.     return id;
16. }
17.
18. public void setId(int id) {
19.     this.id = id;
20. }
21.
22. public String getUsername() {
23.     return username;
24. }
25.
26. public void setUsername(String username) {
27.     this.username = username;
28. }
29.
30. public String getEmail() {
31.     return email;
32. }
33.
34. public void setEmail(String email) {
35.     this.email = email;
36. }
```

```
37.
38. public String getCountry() {
39.     return country;
40. }
41.
42. public void setCountry(String country) {
43.     this.country = country;
44. }
45. }
```

---

question.hbm.xml

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-mapping PUBLIC
3.          "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.  <hibernate-mapping>
7.  <class name="com.smgc.Question" table="question738">
8.  <id name="id">
9.  <generator class="native"></generator>
10. </id>
11. <property name="name"></property>
12.
13. <map name="answers" table="answer738" cascade="all">
14. <key column="questionid"></key>
15. <index column="answer" type="string"></index>
16. <many-to-many class="com.smgc.User" column="userid"></many-to-many>
17. </map>
18. </class>
19.
20. </hibernate-mapping>
```

---

user.hbm.xml

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-mapping PUBLIC
3.          "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.  <hibernate-mapping>
7.  <class name="com.smgc.User" table="user738">
8.  <id name="id">
9.  <generator class="native"></generator>
10. </id>
11. <property name="username"></property>
12. <property name="email"></property>
13. <property name="country"></property>
14. </class>
```

```
15.
16. </hibernate-mapping>
```

---

hibernate.cfg.xml

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-configuration PUBLIC
3.          "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.  <!-- Generated by MyEclipse Hibernate Tools.                 -->
7.  <hibernate-configuration>
8.
9.    <session-factory>
10.   <property name="hbm2ddl.auto">update</property>
11.   <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
12.   <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
13.   <property name="connection.username">system</property>
14.   <property name="connection.password">oracle</property>
15.   <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
16.
17.    <mapping resource="question.hbm.xml"/>
18.    <mapping resource="user.hbm.xml"/>
19.    </session-factory>
20.
21. </hibernate-configuration>
```

StoreTest.java

```
1.  package com.smgc;
2.
3.  import java.util.HashMap;
4.  import org.hibernate.*;
5.  import org.hibernate.cfg.*;
6.  public class StoreTest {
7.  public static void main(String[] args) {
8.  Session session=new Configuration().configure().buildSessionFactory().openSession();
9.  Transaction tx=session.beginTransaction();
10.
11. HashMap<String,User> map1=new HashMap<String,User>();
12. map1.put("java is a programming language",
13. new User("John Milton","john@gmail.com","usa"));
14.
15. map1.put("java is a platform",
16. new User("Ashok Kumar","ashok@gmail.com","india"));
17.
18. HashMap<String,User> map2=new HashMap<String,User>();
19. map2.put("servlet technology is a server side programming",
20.
21. new User("John Milton","john@gmail.com","usa"));
```

```
22. map2.put("Servlet is an Interface",
23. new User("Ashok Kumar","ashok@gmail.com","india"));
24.
25. Question question1=new Question("What is java?",map1);
26. Question question2=new Question("What is servlet?",map2);
27.
28. session.persist(question1);
29. session.persist(question2);
30.
31. tx.commit();
32. session.close();
33. System.out.println("successfully stored");
34. }
35. }
```

FetchTest.java

```
1.  package com.smgc;
2.  import java.util.*;
3.  import org.hibernate.*;
4.  import org.hibernate.cfg.*;
5.  public class FetchTest {
6.  public static void main(String[] args) {
7.   Session session=new Configuration().configure().buildSessionFactory().openSession();
8.   Query query=session.createQuery("from Question ");
9.   List<Question> list=query.list();
10.
11.  Iterator<Question> iterator=list.iterator();
12.  while(iterator.hasNext()){
13.   Question question=iterator.next();
14.   System.out.println("question id:"+question.getId());
15.   System.out.println("question name:"+question.getName());
16.   System.out.println("answers.....");
17.   Map<String,User> map=question.getAnswers();
18.   Set<Map.Entry<String,User>> set=map.entrySet();
19.
20.   Iterator<Map.Entry<String,User>> iteratoranswer=set.iterator();
21.   while(iteratoranswer.hasNext()){
22.    Map.Entry<String,User> entry=(Map.Entry<String,User>)iteratoranswer.next();
23.    System.out.println("answer name:"+entry.getKey());
24.    System.out.println("answer posted by.....");
25.    User user=entry.getValue();
26.    System.out.println("username:"+user.getUsername());
27.    System.out.println("user emailid:"+user.getEmail());
28.    System.out.println("user country:"+user.getCountry());
29.   }
30.  }
31. session.close();
32. }
33. }
```

Chapter-5.1

Component Mapping, Association Mapping, Transaction Management, HQL and HCQL

☺ One-to-one using Primary Key

One to One Mapping in Hibernate by many-to-one example

1. Example of One to one mapping in hibernate
    1. Persistent classes for one to one mapping
    2. Mapping files for the persistent classes
    3. Configuration file
    4. User classes to store and fetch the data

We can perform one to one mapping in hibernate by two ways:

- By many-to-one element
- By one-to-one element

Here, we are going to perform one to one mapping by many-to-one element. In such case, a foreign key is created in the primary table.

In this example, one employee can have one address and one address belongs to one employee only. Here, we are using bidirectional association. Let's look at the persistent classes.

1) Persistent classes for one to one mapping

There are two persistent classes Employee.java and Address.java. Employee class contains Address class reference and vice versa.

Employee.java

```
1. package com.smgc;
2.
3. public class Employee {
4. private int employeeId;
5. private String name,email;
6. private Address address;
7. //setters and getters
8. }
```

Address.java

```
1. package com.smgc;
2.
3. public class Address {
4. private int addressId;
5. private String addressLine1,city,state,country;
```

```
6. private int pincode;
7. private Employee employee;
8. //setters and getters
9. }
```

---

2) Mapping files for the persistent classes

The two mapping files are employee.hbm.xml and address.hbm.xml.

employee.hbm.xml

In this mapping file we are using **many-to-one** element with unique="true" attribute to make the one to one mapping.

```
1. <?xml version='1.0' encoding='UTF-8'?>
2. <!DOCTYPE hibernate-mapping PUBLIC
3.         "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.         "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.         <hibernate-mapping>
7.         <class name="com.smgc.Employee" table="emp211">
8.         <id name="employeeId">
9.         <generator class="increment"></generator>
10.        </id>
11.        <property name="name"></property>
12.        <property name="email"></property>
13.
14.        <many-to-one name="address" unique="true" cascade="all"></many-to-one>
15.        </class>
16.
17.        </hibernate-mapping>
```

address.hbm.xml

This is the simple mapping file for the Address class.

```
1. <?xml version='1.0' encoding='UTF-8'?>
2. <!DOCTYPE hibernate-mapping PUBLIC
3.         "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.         "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.         <hibernate-mapping>
7.         <class name="com.smgc.Address" table="address211">
8.         <id name="addressId">
9.         <generator class="increment"></generator>
10.        </id>
11.        <property name="addressLine1"></property>
12.        <property name="city"></property>
13.        <property name="state"></property>
```

```
14.         <property name="country"></property>
15.
16.         </class>
17.
18.         </hibernate-mapping>
```

---

3) Configuration file

This file contains information about the database and mapping file.

hibernate.cfg.xml

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-configuration PUBLIC
3.          "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.  <!-- Generated by MyEclipse Hibernate Tools.                 -->
7.  <hibernate-configuration>
8.
9.      <session-factory>
10.         <property name="hbm2ddl.auto">update</property>
11.         <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
12.         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
13.         <property name="connection.username">system</property>
14.         <property name="connection.password">oracle</property>
15.         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
16.     <mapping resource="employee.hbm.xml"/>
17.     <mapping resource="address.hbm.xml"/>
18.     </session-factory>
19.
20. </hibernate-configuration>
```

---

4) User classes to store and fetch the data

Store.java

```
1.  package com.smgc;
2.  import org.hibernate.cfg.*;
3.  import org.hibernate.*;
4.
5.  public class Store {
6.  public static void main(String[] args) {
7.      Configuration cfg=new Configuration();
8.      cfg.configure("hibernate.cfg.xml");
9.      SessionFactory sf=cfg.buildSessionFactory();
10.     Session session=sf.openSession();
```

```
11.     Transaction tx=session.beginTransaction();
12.
13.     Employee e1=new Employee();
14.     e1.setName("Ravi Malik");
15.     e1.setEmail("ravi@gmail.com");
16.
17.     Address address1=new Address();
18.     address1.setAddressLine1("G-21,Lohia nagar");
19.     address1.setCity("Ghaziabad");
20.     address1.setState("UP");
21.     address1.setCountry("India");
22.     address1.setPincode(201301);
23.
24.
25.     e1.setAddress(address1);
26.     address1.setEmployee(e1);
27.
28.     session.persist(e1);
29.     tx.commit();
30.
31.     session.close();
32.     System.out.println("success");
33. }
34. }
```

Fetch.java

```
1.  package com.smgc;
2.  import java.util.Iterator;
3.  import java.util.List;
4.  import org.hibernate.Query;
5.  import org.hibernate.Session;
6.  import org.hibernate.SessionFactory;
7.  import org.hibernate.cfg.Configuration;
8.
9.  public class Fetch {
10. public static void main(String[] args) {
11.     Configuration cfg=new Configuration();
12.     cfg.configure("hibernate.cfg.xml");
13.     SessionFactory sf=cfg.buildSessionFactory();
14.     Session session=sf.openSession();
15.
16.     Query query=session.createQuery("from Employee e");
17.     List<Employee> list=query.list();
18.
19.     Iterator<Employee> itr=list.iterator();
20.     while(itr.hasNext()){
21.      Employee emp=itr.next();
22.      System.out.println(emp.getEmployeeId()+" "+emp.getName()+" "+emp.getEmail());
23.      Address address=emp.getAddress();
24.      System.out.println(address.getAddressLine1()+" "+address.getCity()+" "+
```

```
25.         address.getState()+" "+address.getCountry());
26.     }
27.
28.     session.close();
29.     System.out.println("success");
30. }
31. }
```

☺ One-to-one using Foreign Key

One to One Mapping in Hibernate by one-to-one example

1. Example of One to one mapping in hibernate
    1. Persistent classes for one to one mapping
    2. Mapping files for the persistent classes
    3. Configuration file
    4. User classes to store and fetch the data

As We have discussed in the previous example, there are two ways to perform one to one mapping in hibernate:

- By many-to-one element
- By one-to-one element

Here, we are going to perform one to one mapping by one-to-one element. In such case, no foreign key is created in the primary table.

In this example, one employee can have one address and one address belongs to one employee only. Here, we are using bidirectional association. Let's look at the persistent classes.

1) Persistent classes for one to one mapping

There are two persistent classes Employee.java and Address.java. Employee class contains Address class reference and vice versa.

Employee.java

```
1. package com.smgc;
2.
3. public class Employee {
4. private int employeeId;
5. private String name,email;
6. private Address address;
7. //setters and getters
8. }
```

Address.java

```
1. package com.smgc;
2. public class Address {
3. private int addressId;
4. private String addressLine1,city,state,country;
5. private int pincode;
6. private Employee employee;
7. //setters and getters
8. }
```

2) Mapping files for the persistent classes

The two mapping files are employee.hbm.xml and address.hbm.xml.

employee.hbm.xml

In this mapping file we are using **one-to-one** element in both the mapping files to make the one to one mapping.

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-mapping PUBLIC
3.          "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.           <hibernate-mapping>
7.          <class name="com.smgc.Employee" table="emp212">
8.          <id name="employeeId">
9.          <generator class="increment"></generator>
10.         </id>
11.         <property name="name"></property>
12.         <property name="email"></property>
13.
14.         <one-to-one name="address" cascade="all"></one-to-one>
15.         </class>
16.         </hibernate-mapping>
```

address.hbm.xml

This is the simple mapping file for the Address class. But the important thing is generator class. Here, we are using **foreign** generator class that depends on the Employee class primary key.

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-mapping PUBLIC
3.          "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.           <hibernate-mapping>
6.          <class name="com.smgc.Address" table="address212">
7.          <id name="addressId">
8.          <generator class="foreign">
9.          <param name="property">employee</param>
10.         </generator>
11.         </id>
12.         <property name="addressLine1"></property>
13.         <property name="city"></property>
14.         <property name="state"></property>
15.         <property name="country"></property>
16.         <one-to-one name="employee"></one-to-one>
17.         </class>
18.         </hibernate-mapping>
```

3) Configuration file

This file contains information about the database and mapping file.

hibernate.cfg.xml

```
1. <?xml version='1.0' encoding='UTF-8'?>
2. <!DOCTYPE hibernate-configuration PUBLIC
3.         "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.         "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6. <!-- Generated by MyEclipse Hibernate Tools.                 -->
7. <hibernate-configuration>
8.
9.     <session-factory>
10.        <property name="hbm2ddl.auto">update</property>
11.        <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
12.        <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
13.        <property name="connection.username">system</property>
14.        <property name="connection.password">oracle</property>
15.        <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
16.    <mapping resource="employee.hbm.xml"/>
17.    <mapping resource="address.hbm.xml"/>
18.    </session-factory>
19.
20. </hibernate-configuration>
```

---

4) User classes to store and fetch the data

Store.java

```
1.  package com.smgc;
2.  import org.hibernate.cfg.*;
3.  import org.hibernate.*;
4.
5.  public class Store {
6.  public static void main(String[] args) {
7.      Configuration cfg=new Configuration();
8.      cfg.configure("hibernate.cfg.xml");
9.      SessionFactory sf=cfg.buildSessionFactory();
10.     Session session=sf.openSession();
11.     Transaction tx=session.beginTransaction();
12.
13.     Employee e1=new Employee();
14.     e1.setName("Ravi Malik");
15.     e1.setEmail("ravi@gmail.com");
16.
17.     Address address1=new Address();
18.     address1.setAddressLine1("G-21,Lohia nagar");
```

```
19.     address1.setCity("Ghaziabad");
20.     address1.setState("UP");
21.     address1.setCountry("India");
22.     address1.setPincode(201301);
23.
24.
25.     e1.setAddress(address1);
26.     address1.setEmployee(e1);
27.
28.     session.persist(e1);
29.     tx.commit();
30.
31.     session.close();
32.     System.out.println("success");
33. }
34. }
```

Fetch.java

```
1.  package com.smgc;
2.  import java.util.Iterator;
3.  import java.util.List;
4.  import org.hibernate.Query;
5.  import org.hibernate.Session;
6.  import org.hibernate.SessionFactory;
7.  import org.hibernate.cfg.Configuration;
8.
9.  public class Fetch {
10. public static void main(String[] args) {
11.     Configuration cfg=new Configuration();
12.     cfg.configure("hibernate.cfg.xml");
13.     SessionFactory sf=cfg.buildSessionFactory();
14.     Session session=sf.openSession();
15.
16.     Query query=session.createQuery("from Employee e");
17.     List<Employee> list=query.list();
18.
19.     Iterator<Employee> itr=list.iterator();
20.     while(itr.hasNext()){
21.      Employee emp=itr.next();
22.      System.out.println(emp.getEmployeeId()+" "+emp.getName()+" "+emp.getEmail());
23.      Address address=emp.getAddress();
24.      System.out.println(address.getAddressLine1()+" "+address.getCity()+" "+
25.         address.getState()+" "+address.getCountry());
26.     }
27.     session.close();
28.     System.out.println("success");
29. }
30. }
```

Chapter-5.2

Named Query, Hibernate Caching and Integration

☺ First Level Cache

First Level Cache

Session object holds the first level cache data. It is enabled by default. The first level cache data will not be available to entire application. An application can use many session object.

Second Level Cache

SessionFactory object holds the second level cache data. The data stored in the second level cache will be available to entire application. But we need to enable it explicitely.

Second Level Cache implementations are provided by different vendors such as:

- EH (Easy Hibernate) Cache
- Swarm Cache
- OS Cache
- JBoss Cache

☺ Second Level Cache

Hibernate Second Level Cache

Hibernate second level cache uses *a common cache for all the session object of a session factory*. It is useful if you have multiple session objects from a session factory.

SessionFactory holds the second level cache data. It is global for all the session objects and not enabled by default.

Different vendors have provided the implementation of Second Level Cache.

1. EH Cache
2. OS Cache
3. Swarm Cache
4. JBoss Cache

Each implementation provides different cache usage functionality. There are four ways to use second level cache.

1. **read-only:** caching will work for read only operation.
2. **nonstrict-read-write:** caching will work for read and write but one at a time.
3. **read-write:** caching will work for read and write, can be used simultaneously.
4. **transactional:** caching will work for transaction.

The cache-usage property can be applied to class or collection level in hbm.xml file. The example to define cache usage is given below:

```
1. <cache usage="read-only" />
```

Let's see the second level cache implementation and cache usage.

Implementation	read-only	nonstrict-read-write	read-write	transactional
EH Cache	Yes	Yes	Yes	No
OS Cache	Yes	Yes	Yes	No
Swarm Cache	Yes	Yes	No	No
JBoss Cache	No	No	No	Yes

---

3 extra steps for second level cache example using EH cache

1) Add 2 configuration setting in hibernate.cfg.xml file

```
1. <property name="cache.provider_class">org.hibernate.cache.EhCacheProvider</property>
2. <property name="hibernate.cache.use_second_level_cache">true</property>
```

2) Add cache usage setting in hbm file

```
<cache usage="read-only" />
```

3) Create ehcache.xml file

```
<?xml version="1.0"?>
<ehcache>

<defaultCache
maxElementsInMemory="100"
eternal="true"/>

</ehcache>
```

Hibernate Second Level Cache Example

To understand the second level cache through example, we need to create following pages:

1. Employee.java
2. employee.hbm.xml
3. hibernate.cfg.xml
4. ehcache.xml
5. FetchTest.java

Here, we are assuming, there is emp1012 table in the oracle database containing some records.

File: Employee.java

```
package com.smgc;

public class Employee {
private int id;
private String name;
private float salary;

public Employee() {}
public Employee(String name, float salary) {
    super();
    this.name = name;
    this.salary = salary;
}
//setters and getters
}
```

File: employee.hbm.xml

```
1. <?xml version='1.0' encoding='UTF-8'?>
2. <!DOCTYPE hibernate-mapping PUBLIC
3.         "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.         "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.         <hibernate-mapping>
7.         <class name="com.smgc.Employee" table="emp1012">
8.         <cache usage="read-only" />
9.         <id name="id">
10.        <generator class="native"></generator>
11.        </id>
12.        <property name="name"></property>
13.        <property name="salary"></property>
14.        </class>
15.
16.        </hibernate-mapping>
```

Here, we are using **read-only** cache usage for the class. The cache usage can also be used in collection.

File: hibernate.cfg.xml

```
1. <?xml version='1.0' encoding='UTF-8'?>
2. <!DOCTYPE hibernate-configuration PUBLIC
3.         "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.         "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6. <!-- Generated by MyEclipse Hibernate Tools.                   -->
7. <hibernate-configuration>
8.
9.     <session-factory>
10.        <property name="show_sql">true</property>
11.        <property name="hbm2ddl.auto">update</property>
12.        <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
13.        <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
14.        <property name="connection.username">system</property>
15.        <property name="connection.password">oracle</property>
16.        <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
17.
18.        <property name="cache.provider_class">org.hibernate.cache.EhCacheProvider</property>

19.         <property name="hibernate.cache.use_second_level_cache">true</property>
20.
21.    <mapping resource="employee.hbm.xml"/>
22.    </session-factory>
23.
24. </hibernate-configuration>
```

To implement second level cache, we need to define **cache.provider_class** property in the configuration file.

File: ehcache.xml

```
1. <?xml version="1.0"?>
2. <ehcache>
3. <defaultCache
4. maxElementsInMemory="100"
5. eternal="false"
6. timeToIdleSeconds="120"
7. timeToLiveSeconds="200" />
8.
9. <cache name=“com.smgc.Employee"
10. maxElementsInMemory="100"
11. eternal="false"
12. timeToIdleSeconds="5"
13. timeToLiveSeconds="200" />
14. </ehcache>
```

You need to create ehcache.xml file to define the cache property.

defaultCache will be used for all the persistent classes. We can also define persistent class explicitely by using the cache element.

eternal If we specify eternal="true", we don't need to define timeToIdleSeconds and timeToLiveSeconds attributes because it will be handled by hibernate internally. Specifying eternal="false" gives control to the programmer, but we need to define timeToIdleSeconds and timeToLiveSeconds attributes.

timeToIdleSeconds It defines that how many seconds object can be idle in the second level cache.

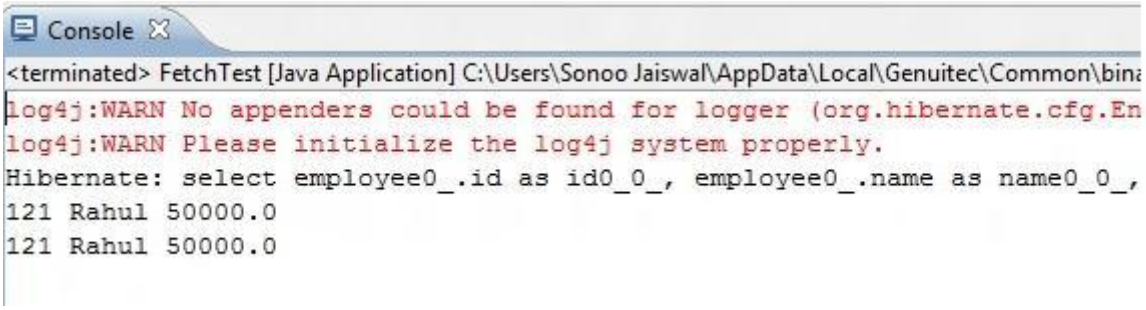
timeToLiveSeconds It defines that how many seconds object can be stored in the second level cache whether it is idle or not.

File: FetchTest.java

```
1. package com.smgc;
2.
3. import org.hibernate.Session;
4. import org.hibernate.SessionFactory;
5. import org.hibernate.cfg.Configuration;
6.
7. public class FetchTest {
8. public static void main(String[] args) {
9. Configuration cfg=new Configuration().configure("hibernate.cfg.xml");
10. SessionFactory factory=cfg.buildSessionFactory();
11.
```

```
12.  Session session1=factory.openSession();
13.  Employee emp1=(Employee)session1.load(Employee.class,121);
14.  System.out.println(emp1.getId()+" "+emp1.getName()+" "+emp1.getSalary());
15.  session1.close();
16.
17.  Session session2=factory.openSession();
18.  Employee emp2=(Employee)session2.load(Employee.class,121);
19.  System.out.println(emp2.getId()+" "+emp2.getName()+" "+emp2.getSalary());
20.  session2.close();
21.
22. }
23. }
```

Output:



As we can see here, hibernate does not fire query twice. If you don't use second level cache, hibernate will fire query twice because both query uses different session objects.

☺ Hibernate and Struts

Hibernate and Struts 2 Integration

1. Hibernate and Struts2 Integration
2. Example of Hibernate and struts2 integration

We can integrate any **struts application with hibernate**. There is no requirement of extra efforts.

In this example, we going to use struts 2 framework with hibernate. You need to have jar files for struts 2 and hibernate.

Example of Hibernate and struts2 integration

In this example, we are creating the registration form using struts2 and storing this data into the database using Hibernate. Let's see the files that we should create to integrate the struts2 application with hibernate.

- **index.jsp** file to get input from the user.
- **User.java** A action class for handling the request. It uses the dao class to store the data.
- **RegisterDao.java** A java class that uses DAO design pattern to store the data using hibernate.
- **user.hbm.xml** A mapping file that contains information about the persistent class. In this case, action class works as the persistent class.
- **hibernate.cfg.xml** A configuration file that contains informations about the database and mapping file.
- **struts.xml** file contains information about the action class and result page to be invoked.
- **welcome.jsp** A jsp file that displays the welcome information with username.
- **web.xml** A web.xml file that contains information about the Controller of Struts framework.

index.jsp

In this page, we have created a form using the struts tags. The action name for this form is register.

```
1. <%@ taglib uri="/struts-tags" prefix="S" %>
2.
3. <S:form action="register">
4. <S:textfield name="name" label="Name"></S:textfield>
5. <S:submit value="register"></S:submit>
6.
7. </S:form>
```

User.java

It is a simple POJO class. Here it works as the action class for struts and persistent class for hibernate. It calls the register method of RegisterDao class and returns success as the string.

```
1. package com.smgc;
2.
3. public class User {
4. private int id;
5. private String name;
6. //getters and setters
7.
8. public String execute(){
9.     RegisterDao.saveUser(this);
10.    return "success";
11. }
12.
13. }
```

RegisterDao.java

It is a java class that saves the object of User class using the Hibernate framework.

```
1. package com.smgc;
2.
3. import org.hibernate.Session;
4. import org.hibernate.Transaction;
5. import org.hibernate.cfg.Configuration;
6.
7. public class RegisterDao {
8.
9. public static int saveUser(User u){
10.
11. Session session=new Configuration().
12. configure("hibernate.cfg.xml").buildSessionFactory().openSession();
13.
14. Transaction t=session.beginTransaction();
15. int i=(Integer)session.save(u);
16. t.commit();
17. session.close();
18.
19. return i;
20.
21. }
22.
23. }
```

---

user.hbm.xml

This mapping file contains all the information of the persitent class.

```
1. <?xml version='1.0' encoding='UTF-8'?>
```

```
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
<class name="com.smgc.User" table="user451">
<id name="id">
<generator class="increment"></generator>
</id>
<property name="name"></property>
</class>

</hibernate-mapping>
```

---

hibernate.cfg.xml

This configuration file contains informations about the database and mapping file. Here, we are using the hb2ddl.auto property, so you don't need to create the table in the database.

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">

<!-- Generated by MyEclipse Hibernate Tools.                   -->
<hibernate-configuration>

<session-factory>
<property name="hbm2ddl.auto">update</property>
<property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
<property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
<property name="connection.username">system</property>
<property name="connection.password">oracle</property>
<property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
<mapping resource="user.hbm.xml"/>

</session-factory>

</hibernate-configuration>
```

---

struts.xml

This files contains information about the action class to be invoked. Here the action class is User.

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE struts PUBLIC "-//Apache Software Foundation
//DTD Struts Configuration 2.1//EN"
```

```
4.  "http://struts.apache.org/dtds/struts-2.1.dtd">
5.  <struts>
6.
7.  <package name="abc" extends="struts-default">
8.  <action name="register" class="com.smgc.User">
9.  <result name="success">welcome.jsp</result>
10. </action>
11. </package>
12. </struts>
```

---

welcome.jsp

It is the welcome file, that displays the welcome message with username.

```
1.  <%@ taglib uri="/struts-tags" prefix="S" %>
2.
3.  Welcome: <S:property value="name"/>
```

---

web.xml

It is web.xml file that contains the information about the controller. In case of Struts2, StrutsPrepareAndExecuteFilter class works as the controller.

```
1.  <?xml version="1.0" encoding="UTF-8"?>
2.  <web-app version="2.5"
3.      xmlns="http://java.sun.com/xml/ns/javaee"
4.      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.      xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
6.      http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">
7.    <welcome-file-list>
8.      <welcome-file>index.jsp</welcome-file>
9.    </welcome-file-list>
10.   <filter>
11.     <filter-name>struts2</filter-name>
12.     <filter-class>
13.   org.apache.struts2.dispatcher.ng.filter.StrutsPrepareAndExecuteFilter
14.     </filter-class>
15.   </filter>
16.   <filter-mapping>
17.     <filter-name>struts2</filter-name>
18.     <url-pattern>/*</url-pattern>
19.   </filter-mapping>
20. </web-app>
```

☺ Hibernate and Spring

Hibernate and Spring Integration

We can simply integrate **hibernate application with spring application**.

In hibernate framework, we provide all the database information hibernate.cfg.xml file.

But if we are going to integrate the hibernate application with spring, we don't need to create the hibernate.cfg.xml file. We can provide all the information in the applicationContext.xml file.

Advantage of Spring framework with hibernate

The Spring framework provides **HibernateTemplate** class, so you don't need to follow so many steps like create Configuration, BuildSessionFactory, Session, beginning and committing transaction etc.

So **it saves a lot of code**.

Understanding problem without using spring:

Let's understand it by the code of hibernate given below:

```
1. //creating configuration
2. Configuration cfg=new Configuration();
3. cfg.configure("hibernate.cfg.xml");
4.
5. //creating seession factory object
6. SessionFactory factory=cfg.buildSessionFactory();
7.
8. //creating session object
9. Session session=factory.openSession();
10.
11. //creating transaction object
12. Transaction t=session.beginTransaction();
13.
14. Employee e1=new Employee(111,"arun",40000);
15. session.persist(e1);//persisting the object
16.
17. t.commit();//transaction is commited
18. session.close();
```

As you can see in the code of sole hibernate, you have to follow so many steps.

Solution by using HibernateTemplate class of Spring Framework:

Now, you don't need to follow so many steps. You can simply write this:

1. Employee e1=new Employee(111,"arun",40000);
2. hibernateTemplate.save(e1);

Methods of HibernateTemplate class

Let's see a list of commonly used methods of HibernateTemplate class.

No.	Method	Description
1)	void persist(Object entity)	persists the given object.
2)	Serializable save(Object entity)	persists the given object and returns id.
3)	void saveOrUpdate(Object entity)	persists or updates the given object. If id is found, it updates the record otherwise saves the record.
4)	void update(Object entity)	updates the given object.
5)	void delete(Object entity)	deletes the given object on the basis of id.
6)	Object get(Class entityClass, Serializable id)	returns the persistent object on the basis of given id.
7)	Object load(Class entityClass, Serializable id)	returns the persistent object on the basis of given id.
8)	List loadAll(Class entityClass)	returns the all the persistent objects.

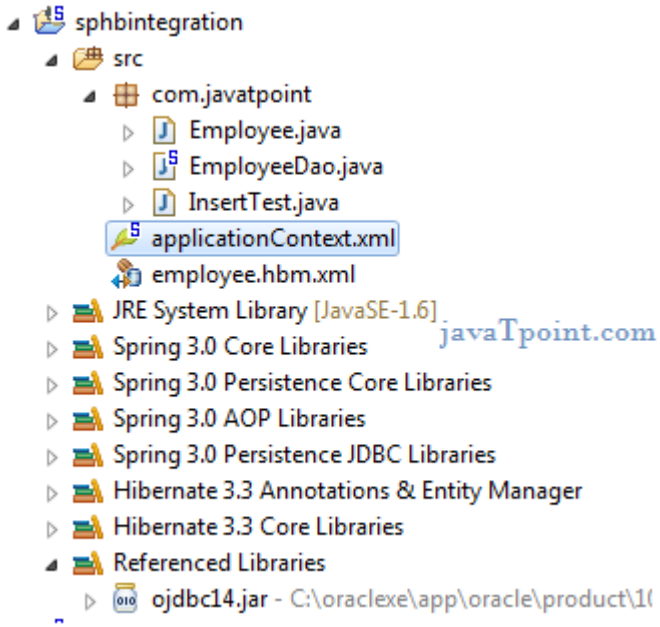
Steps

Let's see what are the simple steps for hibernate and spring integration:

1. **create table in the database** It is optional.
2. **create applicationContext.xml file** It contains information of DataSource, SessionFactory etc.
3. **create Employee.java file** It is the persistent class
4. **create employee.hbm.xml file** It is the mapping file.
5. **create EmployeeDao.java file** It is the dao class that uses HibernateTemplate.
6. **create InsertTest.java file** It calls methods of EmployeeDao class.

Example of Hibernate and spring integration

In this example, we are going to integrate the hibernate application with spring. Let's see the **directory structure** of spring and hibernate example.



---

1) create the table in the database

In this example, we are using the Oracle as the database, but you may use any database. Let's create the table in the oracle database

```
1. CREATE TABLE  "EMP558"
2.    (    "ID" NUMBER(10,0) NOT NULL ENABLE,
3.     "NAME" VARCHAR2(255 CHAR),
4.     "SALARY" FLOAT(126),
5.      PRIMARY KEY ("ID") ENABLE
6.    )
7. /
```

---

2) Employee.java

It is a simple POJO class. Here it works as the persistent class for hibernate.

```
1. package com.smgc;
2.
3. public class Employee {
4. private int id;
5. private String name;
6. private float salary;
7.
8. //getters and setters
9.
10. }
```

---

3) employee.hbm.xml

This mapping file contains all the information of the persistent class.

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
<class name="com.smgc.Employee" table="emp558">
        <id name="id">
        <generator class="assigned"></generator>
        </id>

        <property name="name"></property>
        <property name="salary"></property>
</class>

</hibernate-mapping>
```

---

4) EmployeeDao.java

It is a java class that uses the **HibernateTemplate** class method to persist the object of Employee class.

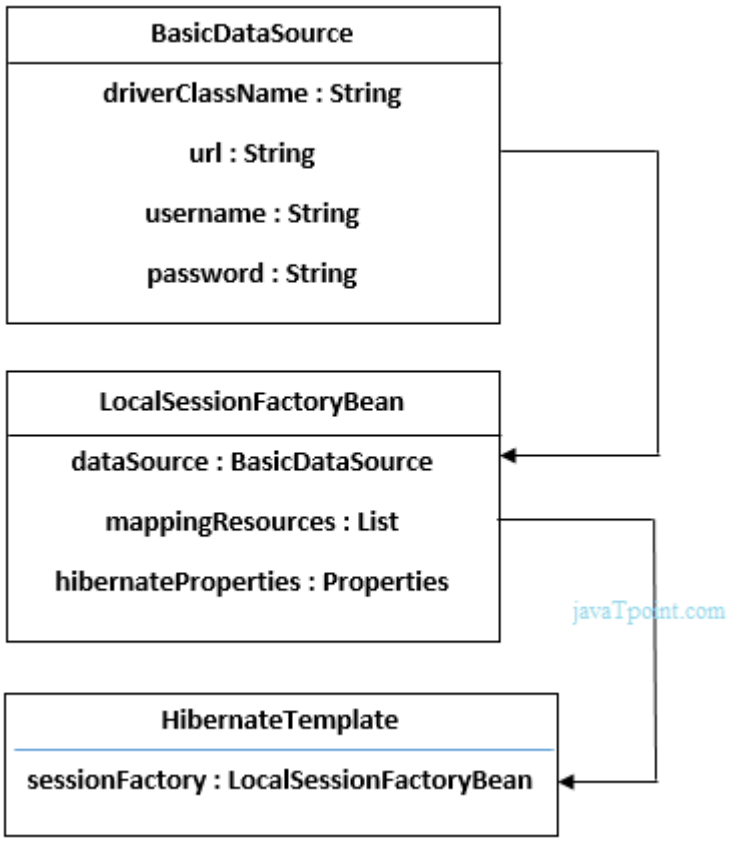
```
package com.smgc;
import org.springframework.orm.hibernate3.HibernateTemplate;
import java.util.*;
public class EmployeeDao {
HibernateTemplate template;
public void setTemplate(HibernateTemplate template) {
    this.template = template;
}
//method to save employee
public void saveEmployee(Employee e){
    template.save(e);
}
//method to update employee
public void updateEmployee(Employee e){
    template.update(e);
}
//method to delete employee
public void deleteEmployee(Employee e){
    template.delete(e);
}
//method to return one employee of given id
public Employee getById(int id){
    Employee e=(Employee)template.get(Employee.class,id);
    return e;
```

```
25. }
26. //method to return all employees
27. public List<Employee> getEmployees(){
28.     List<Employee> list=new ArrayList<Employee>();
29.     list=template.loadAll(Employee.class);
30.     return list;
31. }
32. }
```

---

5) applicationContext.xml

In this file, we are providing all the informations of the database in the **BasicDataSource** object. This object is used in the **LocalSessionFactoryBean** class object, containing some other informations such as mappingResources and hibernateProperties. The object of **LocalSessionFactoryBean** class is used in the HibernateTemplate class. Let's see the code of applicationContext.xml file.



File: applicationContext.xml

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans
3.     xmlns="http://www.springframework.org/schema/beans"
4.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.     xmlns:p="http://www.springframework.org/schema/p"
```

```
6.      xsi:schemaLocation="http://www.springframework.org/schema/beans
7.         http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
8.
9.
10.    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
11.       <property name="driverClassName"  value="oracle.jdbc.driver.OracleDriver"></property>

12.       <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"></property>
13.       <property name="username" value="system"></property>
14.       <property name="password" value="oracle"></property>
15.    </bean>
16.
17.    <bean id="mysessionFactory"   class="org.springframework.orm.hibernate3.LocalSessionFact
    oryBean">
18.       <property name="dataSource" ref="dataSource"></property>
19.
20.       <property name="mappingResources">
21.       <list>
22.       <value>employee.hbm.xml</value>
23.       </list>
24.       </property>
25.
26.       <property name="hibernateProperties">
27.          <props>
28.             <prop key="hibernate.dialect">org.hibernate.dialect.Oracle9Dialect</prop>
29.             <prop key="hibernate.hbm2ddl.auto">update</prop>
30.             <prop key="hibernate.show_sql">true</prop>
31.
32.          </props>
33.       </property>
34.    </bean>
35.
36.    <bean id="template" class="org.springframework.orm.hibernate3.HibernateTemplate">
37.    <property name="sessionFactory" ref="mysessionFactory"></property>
38.    </bean>
39.
40.    <bean id="d" class="com.smgc.EmployeeDao">
41.    <property name="template" ref="template"></property>
42.    </bean>
43.
44.
45.    </beans>
```

6) InsertTest.java

This class uses the EmployeeDao class object and calls its saveEmployee method by passing the object of Employee class.

```
1. package com.smgc;
2.
3. import org.springframework.beans.factory.BeanFactory;
4. import org.springframework.beans.factory.xml.XmlBeanFactory;
5. import org.springframework.core.io.ClassPathResource;
6. import org.springframework.core.io.Resource;
7.
8. public class InsertTest {
9. public static void main(String[] args) {
10.
11.    Resource r=new ClassPathResource("applicationContext.xml");
12.    BeanFactory factory=new XmlBeanFactory(r);
13.
14.    EmployeeDao dao=(EmployeeDao)factory.getBean("d");
15.
16.    Employee e=new Employee();
17.    e.setId(114);
18.    e.setName("varun");
19.    e.setSalary(50000);
20.
21.    dao.saveEmployee(e);
22.
23. }
24. }
```

Now, if you see the table in the oracle database, record is inserted successfully.

Enabling automatic table creation, showing sql queries etc.

You can enable many hibernate properties like automatic table creation by hbm2ddl.auto etc. in applicationContext.xml file. Let's see the code:

```
1. <property name="hibernateProperties">
2.          <props>
3.             <prop key="hibernate.dialect">org.hibernate.dialect.Oracle9Dialect</prop>
4.             <prop key="hibernate.hbm2ddl.auto">update</prop>
5.             <prop key="hibernate.show_sql">true</prop>
6.
7.          </props>
```

If you write this code, you don't need to create table because table will be created automatically.